

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Gajdoš** Jméno: **Matúš** Osobní číslo: **510946**
Fakulta/ústav: **Fakulta informačních technologií**
Zadávající katedra/ústav: **Katedra aplikované matematiky**
Studijní program: **Informatika**
Specializace: **Umělá inteligence**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Benchmark výkonu otevřených modelů LLM

Název bakalářské práce anglicky:

Performance benchmark of open LLM models

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Tomáš Vondra, Ph.D. katedra počítačových systémů FIT

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **29.01.2025**

Termín odevzdání bakalářské práce: **16.05.2025**

podpis vedoucí(ho) ústavu/katedry

podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Gajdoš Matúš
Podpis studenta

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Gajdoš** Jméno: **Matúš** Osobní číslo: **510946**
Fakulta/ústav: **Fakulta informačních technologií**
Zadávací katedra/ústav: **Katedra aplikované matematiky**
Studijní program: **Informatika**
Specializace: **Umělá inteligence**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Benchmark výkonu otevřených modelů LLM

Název bakalářské práce anglicky:

Performance benchmark of open LLM models

Pokyny pro vypracování:

V současnosti jsou moderní modely umělé inteligence typu LLM (Large Language Model). Mimo nejznámějšího ChatGPT existují i jejich otevřené implementace. Ke svému běhu potřebují grafické procesory (GPU) s vysokou mírou paralelismu. Jsou ale potřeba ty největší a nejvýkonnější GPU?

1. Prozkoumejte dostupné otevřené modely LLM o různé velikosti.
2. Prozkoumejte možnosti snížení náročnosti LLM pomocí metod jako je kvantizace.
3. Vybrané modely nasadte a změřte jejich výkon při operaci inference na vhodném standardizovaném benchmarku.
4. Podle výsledků testu latence a subjektivního posouzení kvality inference různě velkých modelů posuďte, jaké GPU je dostatečné pro provoz chatovacího robota a jaké byste použil na dávkové zpracování.

Seznam doporučené literatury:

Bakalárska práca

BENCHMARK VÝKONU OTVORENÝCH MODELOV LLM

Matúš Gajdoš, Ing. Tomáš Vondra, Ph.D.

Fakulta informačních technologií
Katedra aplikované matematiky
Vedúci: Ing. Tomáš Vondra, Ph.D.
16. mája 2025

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2025 Matúš Gajdoš, Ing. Tomáš Vondra, Ph.D.. Všetky práva vyhradené.

Táto práca vznikla ako školské dielo na FIT ČVUT v Prahe. Práca je chránená medzinárodnými predpismi a zmluvami o autorskom práve a právach súvisiacich s autorským právom. Na jej využitie, s výnimkou bezplatných zákonných licencií, je nutný súhlas autora.

Odkaz na túto prácu: Gajdoš Matúš. *Benchmark výkonu otvorených modelov LLM.*

Bakalárska práca. České vysoké učení technické v Praze, Fakulta informačních technologií, 2025.

Chcel by som na tomto mieste poďakovať vedúcemu tejto práce, pánovi Ing. Tomáš Vondra, Ph.D. za odborné vedenie práce a za cenné rady a pripomienky, ktoré mi pomohli túto prácu skompletizovať. Takisto chcem poďakovať mojím priateľom a rodičom, ktorí boli dôležitou súčasťou a oporou po celú dobu môjho štúdia.

Vyhlásenie

Prehlasujem, že som predloženú prácu vypracoval samostatne a že som uviedol všetky použité informačné zdroje v súlade s Metodickým pokynom o dodržiavaní etických princípov pri príprave vysokoškolských záverečných prác. Beriem na vedomie, že sa na moju prácu vzťahujú práva a povinnosti vyplývajúce zo zákona č. 121/2000 Sb., autorského zákona, v znení neskorších predpisov. V súlade s ust. § 2373 odst. 2 zákona č. 89/2012 Sb., občiansky zákonník, v znení neskorších predpisov, týmto udeľujem nevýhradné oprávnenie (licenciu) k užitiu tejto mojej práce, a to vrátane všetkých počítačových programov, ktoré sú jej súčasťou či prílohou a celej jej dokumentácie (ďalej súhrnne iba „Dielo“), a to všetkým osobám, ktoré si prajú Dielo použiť. Tieto osoby sú oprávnené Dielo použiť akýmkoľvek spôsobom, ktorý nesnižuje hodnotu Diela a za akýchkoľvek účelom (vrátane použitia k zárobkovým účelom). Toto oprávnenie je časové, teritoriálne a množstevne neobmedzené.

Prehlasujem, že som v priebehu príprav a písaní záverečnej práce použil nástroje umelej inteligencie. Vygenerovaný obsah som overil. Potvrdzujem, že som si vedomí, že za obsah záverečnej práce plne zodpovedám.

V Prahe dňa 16. mája 2025

Abstrakt

V posledných rokoch vzbudili veľké jazykové modely (LLM) značnú pozornosť verejnosti. Záujem sa čoraz viac sústreďuje nielen na kvalitu ich výstupov, ale aj na ich hardvérové nároky a výkonnosť. Táto práca sa zameriava na hodnotenie hardvérového výkonu open-source LLM rôznych veľkostí, pričom skúmame ich správanie pri rôznych dĺžkach vstupov a výstupov, ako aj pri rôznom počte paralelne spracovávaných požiadaviek. Testovali sme výkon modelov LLaMA-3.1-8B-Instruct, LLaMA-3.2-1B-Instruct a Qwen3-32B na grafickej karte NVIDIA A100 80GB. Pri týchto experimentoch sme sa zamerali na posúdenie vhodnosti jednotlivých modelov pre konkrétne typy úloh a záťaže. Výsledky poskytujú prehľad o pamäťovej náročnosti, latencii a priepustnosti v rôznych scenároch použitia, čím prispievajú k lepšiemu rozhodovaniu pri výbere modelu pre praktické nasadenie.

Kľúčová slova open-source LLM, testovanie LLM výkonu na GPU, GPU, benchmark

Abstract

In recent years, large language models (LLMs) have attracted significant public attention. Interest has increasingly focused not only on the quality of their outputs, but also on their hardware requirements and performance. This work focuses on evaluating the hardware performance of open-source LLMs of different parameter sizes, by investigating their behavior under different input and output lengths as well as different batch sizes. We tested the performance of LLaMA-3.1-8B-Instruct, LLaMA-3.2-1B-Instruct, and Qwen3-32B on an NVIDIA A100 80GB graphics card. In these experiments, we focused on assessing the suitability of each model for specific types of tasks and workloads. The results provide insight into memory footprint, latency, and throughput under different usage scenarios, thus contributing to better decision making when selecting a model for practical deployments.

Keywords open-source LLM, LLM performance testing on GPU, GPU, benchmark

Obsah

1	Úvod	1
1.1	Ciele	2
1.2	Súčasn�� rie��enie	2
2	��vod do LLM	3
2.1	Prehľad	3
2.2	Vyu��itie	3
2.3	Rozdiel medzi otvoren��mi a uzavret��mi modelmi	4
2.4	Architekt��ra	5
2.4.1	Transformer	6
2.4.1.1	Vkladacia (embedding) vrstva	8
2.4.1.2	Po��i��n�� k��dovanie	9
2.4.1.3	K��dova��	9
2.4.1.4	Multi-Head Attention vrstva	10
2.4.1.5	Dekod��r	10
2.4.1.6	Softmax	11
2.4.2	Encoder-decoder architekt��ra	11
2.4.3	Encoder-only architekt��ra	11
2.4.4	Decoder-only architekt��ra	12
2.4.5	Mixture-of-Experts	12
2.5	Predtr��novanie	13
2.5.1	Zber d��t	13
2.5.2	Predspracovanie d��t	15
2.5.2.1	Filtrovanie a v��ber	15
2.5.2.2	Deduplik��cia	16
2.5.2.3	Ochrana s��kromia	16
2.5.2.4	Tokeniz��cia	16
2.5.3	Data Scheduling	18
2.5.4	Tr��ning	18
2.5.4.1	Normaliza��n�� met��dy	19
2.5.4.2	Normaliza��n�� poloha	19
2.5.4.3	Aktiva��n�� funkcia	20
2.5.4.4	Po��i��n�� vlo��enia	20
2.5.4.5	Pozornos��	21
2.5.4.6	Princ��p tr��novania	22
2.5.5	Optimaliz��cia parametrov	23

2.5.6	Generovanie výstupu	23
2.6	Fine-Tuning	25
2.6.1	Dáta	25
2.6.2	Parameter efficient fine-tuning (PEFT)	25
2.6.2.1	Reparametrizácia	26
2.6.3	Ladenie inštrukcií	26
2.6.4	Reinforcement learning from human feedback (RLHF)	27
2.7	Inferencia	27
2.8	Kvantizácia	28
2.8.1	Quantization-Aware Training	29
2.8.2	Post-Training Quantization	29
3	Úvod do GPU	31
3.1	Aplikácia GPU v strojovom učení	32
3.2	GPU architektúra	32
3.3	NVIDIA A100	33
4	Porovnanie open-source LLM	37
4.1	Špecifické modely	37
4.1.1	Qwen modely	38
4.1.2	Mistral modely	42
4.1.2.1	Mistral 7B	42
4.1.2.2	Mixtral 8x7B	43
4.1.3	LLaMA Modely	44
4.1.3.1	LLaMA 1	44
4.1.3.2	LLaMA 2	46
4.1.3.3	LLaMA 3	48
4.1.4	DeepSeek	49
4.1.4.1	DeepSeek-LLM	49
4.1.4.2	DeepSeek-V2	51
4.1.4.3	DeepSeek-V3	53
5	Meranie výkonu	56
5.1	Metodika merania výkonu	57
5.2	Testovacie parametre	59
5.3	Metriky testovania	61
5.4	Výsledky	62
5.4.1	LLaMA-3.2-1B-Instruct	62
5.4.2	LLaMA-3.1-8B-Instruct	65
5.4.3	Qwen3-32B	67
5.4.4	Porovnanie	69
5.4.5	Záver merania	72
6	Záver	75

Obsah

viii

Obsah prílohy

83

Zoznam obrázkov

2.1	Evolučný strom moderných LLM sleduje vývoj jazykových modelov v posledných rokoch a zvyrazňuje niektoré z najznámejších modelov. Modely na rovnakej vetve majú užšie vzťahy. Modely založené na transformátoroch sú zobrazené v iných než šedých farbách: modely len s dekodérom v modrej vetve, modely iba s kódovacím zariadením v ružovej vetve a modely s kódovačom a dekodérom v zelenej vetve. Vertikálna poloha modelov na časovej osi predstavuje dátumy ich vydania. Open-source modely sú reprezentované plnými štvorcami, zatiaľ čo uzavreté-source modely sú reprezentované dutými. Skladaný stĺpcový graf v pravom dolnom rohu zobrazuje počet modelov od rôznych spoločností a inštitúcií. Získané z [13]	6
2.2	Transformer architektúra. Získané z [14]	8
2.3	Ilustrácia Mixture-of-Experts architektúry bloku kódovača, kde namiesto hustej dosprednej neuronovej siete je vrstva expertov so smerovačom (bledo modrá). Vrstva operuje nezávisle na jednotlivých tokenoch vo vstupnej sekvencii. Získané z [14]	13
2.4	Ilustrácia typickej pipeline na predspracovanie dát na predtréning veľkých jazykových modelov. Získané z [1]	15
2.5	Sublayer označuje podvrstvu typu FFN (Feed-Forward Network) alebo samopozorovaciu (self-attention) jednotku v transformrovej vrstve, d označuje veľkosť skrytých stavov, Norm označuje jednu z normalizačných metód, p_i označuje pozičné zapuzdrenie (embedding) na pozícii i , A_{ij} označuje skóre pozornosti medzi požiadavkou (query) a kľúčom (key), r_{i-j} označuje naučiteľný skalár na základe rozdielu medzi požiadavkou a kľúčom, a $\mathbf{R}_{\Theta,t}$ označuje rotačnú maticu s uhlom rotácie $t \cdot \Theta$. Získané z [1]	18
3.1	Prehľad architektúry a komponent v GPU. Získané z [43]	35
3.2	Prehľad komponent vnútri SM. Získané z [43]	36
5.1	Meranie latencie a priepustnosti modelu LLaMA-3.2-1B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.	63
5.2	Priepustnosť v porovnaní s latenciou pre LLaMA-3.2-1B	64

5.3	Meranie ITL a TTFT modelu LLaMA-3.2-1B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.	64
5.4	Meranie latencie a priepustnosti modelu LLaMA-3.1-8B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.	65
5.5	Priepustnosť v porovnaní s latenciou pre LLaMA-3.1-8B	66
5.6	Meranie ITL a TTFT modelu LLaMA-3.1-8B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.	67
5.7	Meranie latencie a priepustnosti modelu Qwen3-32B na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.	67
5.8	Priepustnosť v porovnaní s latenciou pre Qwen3-32B	68
5.9	Meranie ITL a TTFT modelu Qwen3-32B na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.	69
5.10	Porovnanie modelov a ich úrovní v závislosti s priepustnosťou.	70
5.11	Porovnanie modelov a ich úrovní v závislosti s priepustnosťou.	70
5.12	Porovnanie modelov a ich úrovní v závislosti s latenciou.	71
5.13	Porovnanie modelov a ich úrovní v závislosti s latenciou.	71
5.14	Meranie a porovnanie latencie všetkých modelov na rôznych úrovniach vstupných/výstupných dát. Na ose x je batch size a na ose y latencia.	73

Zoznam tabuliek

5.1	Pamäťová náročnosť jednotlivých modelov pri nasadení	62
-----	--	----

Zoznam výpisov kódu

5.1	Script testujúci rôznu záťaž pri inferencií modelu pomocou nástroju GenAI-Perf.	59
-----	---	----

5.2	Script na nasadenie LLM serveru pomocou vLLM pomocou plánovača Slurm.	60
-----	---	----

Zoznam skratiek

LLM	Large Language Model
GPU	Graphics Processing Unit
API	Application Programming Interface
IDE	Integrated Development Environment
NLP	Natural Language Processing
GPT	Generative Pre-training Transformer
VRAM	Video Random-Access Memory
BPE	Byte-Pair Encoding
SFT	Supervised fine-tuning
RLHF	Reinforcement learning from human feedback
LSH	Locality-sensitive hashing
RoPE	Rotary Positional Embedding
GQA	Grouped-query Attention
SWA	Sliding Window Attention
MoE	Mixture of Experts
HPC	High Performance Computing
SM	Streaming Multiprocessor
FP	Floating Point
DCA	Dual Chunk Attention



Kapitola 1

Úvod

Problém generovania zrozumiteľnej ľudskej reči pomocou strojového učenia začal priťahovať pozornosť vedcov a výskumníkov už v polovici 20. storočia. Napriek dlhoročnému vývoju došlo k výraznému pokroku až v poslednej dekáde, najmä vďaka úspechom veľkých jazykových modelov (Large Language Models – LLM), ktoré zaznamenali revolúciu v oblasti spracovania prirodzeného jazyka (Natural Language Processing – NLP). Tieto modely si získali nielen záujem odborníkov a akademickej obce, ale aj širokej verejnosti, čím sa stali kľúčovou technológiou v oblasti umelej inteligencie [1]. Vysoká efektivita LLM v riešení úloh NLP, ako je generovanie súvislého textu, preklad alebo sumarizácia, podnietila mnohé technologické spoločnosti, vrátane gigantov ako Meta, Google či OpenAI, aby investovali do ich vývoja a nasadenia. Avšak s rastúcou komplexnosťou modelov sa zvyšujú aj ich výpočtové nároky.

Súčasnú špičkovú LLM obsahujú miliardy až bilióny parametrov, čo vedie k extrémnym požiadavkám na hardvérové zdroje. Tréning takýchto modelov vyžaduje špecializované výpočtové jednotky, ako sú grafické procesory (Graphics Processing Units – GPU) alebo dokonca superpočítačové infraštruktúry. Prevádzka modelov, akými sú napríklad GPT-4 od OpenAI, si vyžaduje enormné investície, čo spôsobuje, že tieto modely sú často dostupné len prostredníctvom platených služieb a nie sú plne otvorené verejnosti. [2, 3] Napriek tomu existuje rastúci trend vývoja open-source LLM, ktoré umožňujú jednotlivcom a organizáciám pracovať s pokročilými modelmi bez nutnosti využívať komerčné riešenia. Príkladmi takýchto modelov sú LLaMA od spoločnosti Meta [4] alebo Qwen od Alibaba cloud. [5] Keďže hardvérové požiadavky rôznych modelov sa značne líšia v závislosti od ich veľkosti a architektúry, vzniká otázka, aké sú optimálne výpočtové zdroje potrebné na ich efektívne nasadenie. [3] Táto práca sa preto zameriava na analýzu výkonnostných požiadaviek otvorených jazykových modelov LLM pri ich inferencii a tréningu na GPU. Cieľom je preskúmať, aké výpočtové zdroje sú potrebné pre jednotlivé kategórie modelov a aké sú limity nasadenia týchto technológií na bežne dostupných hardvérových platformách.

1.1 Ciele

Cieľom tejto práce je poskytnúť hlbšie porozumenie výpočtovým požiadavkám najnovších open-source veľkých jazykových modelov (LLM) a ich výkonnosti na hardvéri. Výskum sa primárne zameria na hardvérovú záťaž a výkon LLM, pričom osobitná pozornosť bude venovaná rýchlosti inferencie modelov pri rôznych dĺžkach vstupu a výstupu. Na záver, táto práca hodnotí výkon open-source LLM nasadených na GPU poskytovanom fakultou FIT ČVUT, konkrétne na grafickej karte NVIDIA A100 s 80 GB pamäte. Hlavným cieľom je identifikovať kľúčové faktory ovplyvňujúce výpočtovú efektivitu a celkovú účinnosť týchto modelov.

1.2 Súčasné riešenie

Vzhľadom na rastúci počet open-source jazykových modelov sa v posledných rokoch objavili viaceré iniciatívy zamerané na ich systematické porovnávanie. Výkon a záťaž LLM na GPU sa dnes hodnotia prostredníctvom rôznych benchmarkových testov, ktoré sledujú viacero aspektov, ako sú rýchlosť inferencie, spotreba pamäte, efektivita využitia výpočtových zdrojov a vplyv optimalizačných techník, napríklad kvantizácie. [6]

Úvod do LLM

2.1 Prehľad

Veľký jazykový model (LLM - Large Language Model) je pokročilý typ umelej inteligencie založený na hlbokých neurónových sieťach, najčastejšie na architektúre transformera. Je trénovaný na obrovských množstvách textových dát, vďaka čomu dokáže generovať text, porozumieť jazyku, odpovedať na otázky, sumarizovať informácie a vykonávať rôzne jazykové úlohy. LLM funguje na princípe pravdepodobnostnej predikcie slov alebo viet na základe kontextu a neustále sa zdokonaľuje pomocou učenia na veľkých datasetoch.

2.2 Využitie

LLM vďaka svojim dobrým výsledkom nachádzajú široké uplatnenie v rôznych oblastiach, od automatizácie zákazníckej podpory cez tvorbu obsahu až po analýzu veľkých objemov textových dát. Využitie LLM v rôznych oblastiach priemyslu sa stáva čoraz bežnejšou skutočnosťou. Ukázalo sa, že LLM sú schopné zvládnuť rôzne úlohy v oblasti zdravotníctva, napríklad extrakciu biologických informácií, zjednodušenie správy či konzultáciu lekárskeho rád. Napriek tomu sa ich aplikácia v zdravotníctve využíva obozretne kvôli halucináciám modelov a potrebe maximálnej presnosti. Aj tak však dosiahli vysokú presnosť výstupov pod dohľadom odborníkov v danom odbore. [7]

Ďalšou dôležitou oblasťou, v ktorej má LLM sľubné možnosti uplatnenia, sú financie. Využívajú sa na rôzne úlohy súvisiace s finančným sektorom, napríklad na detekciu číselných pohľadávok, finančné uvažovanie a ďalšie. Napriek ich prínosu je nevyhnutné zvážiť potenciálne riziká pri ich aplikácii vo finančníctve, pretože generovanie nepresného alebo škodlivého obsahu by mohlo mať významné nepriaznivé dôsledky pre finančné trhy. [8]

Okrem spomenutých odvetví nachádzajú LLM uplatnenie aj v školstve a vzdelávaní, pri vývoji softvéru, korekcii programov, sumarizácii kódu, pred-

ikcii právneho úsudku, analýze právnych dokumentov a v mnohých ďalších oblastiach. Stručne povedané, využitie LLM na podporu ľudských úloh v reálnom svete sa stalo významnou oblasťou výskumu, avšak prináša aj viaceré výzvy. Zabezpečenie presnosti generovaného obsahu, riešenie predsudkov, ako aj ochrana súkromia používateľov a bezpečnosti dát, sú kľúčovými faktormi pri ich integrácii do praxe. [1]

2.3 Rozdiel medzi otvorenými a uzavretými modelmi

Rozhodovanie medzi používaním otvoreného alebo uzavretého LLM modelu má viacero aspektov, ktoré je potrebné starostlivo zvážiť. Záleží najmä na požiadavkách a podmienkach, ktoré najlepšie vyhovujú konkrétnemu use-case ¹. Preto sa v tejto časti zameriame na najväčšie rozdiely, výhody a nevýhody otvorených a uzavretých modelov.

Otvorené modely sú špecifické tým, že sú prístupné verejnosti a umožňujú používanie, úpravy a prispôbienie softvéru v súlade s licenčnými podmienkami. Prístupnosť ku zdrojovému kódu prináša množstvo výhod, z ktorých profitujú nielen vývojári, ale aj spoločnosti. Modely s otvoreným zdrojovým kódom umožňujú podnikom flexibilne ich prispôbovať a doladovať podľa svojich špecifických potrieb. Vzhľadom na ich otvorenosť môžu podniky rýchlo inovovať, integrovať ich s existujúcimi systémami a reagovať na nové výzvy bez nutnosti čakať na aktualizácie od dodávateľa. To je výhodné najmä pre spoločnosti s dostatočnými technickými znalosťami na ich úpravu a údržbu.

Významným benefitom open-source modelov je aj podpora rozsiahlych komunít, ktoré modely modifikujú, opravujú a často zlepšujú ich výkonnosť bez priamej alokácie zdrojov pôvodných tvorcov. Tento crowdsourcing ² informácií podporuje inovácie v oblasti LLM, čo vedie k rýchlejšiemu vývoju a zvyšovaniu kvality modelov. Modely ako LLaMA 3-70B navyše ponúkajú výrazne nižšie náklady na využitie – len 60 centov za milión vstupných tokenov a 70 centov za milión výstupných tokenov. [10] Táto cenová dostupnosť umožňuje širšiemu okruhu vývojárov využívať pokročilé AI možnosti bez výrazných finančných investícií. Avšak s prevádzkovaním otvorených modelov prichádza aj zodpovednosť za údržbu a zabezpečenie ich bezpečnosti. Pri nasadení v súkromnom cloude môžu spoločnosti implementovať vlastné bezpečnostné opatrenia a vykonávať audity kódu, čím sa zabezpečí ochrana citlivých dát podľa ich špecifických štandardov. [11, 3]

Na druhej strane, uzavreté modely nie sú voľne dostupné a prístup k ich zdrojovému kódu či tréningovým dátam je obmedzený. Napriek tomu ponúkajú sofistikované riešenia, ktoré často prekonávajú otvorené modely v pres-

¹Využitie v konkrétnom prípade.

²Crowdsourcing označuje spôsob zadania úlohy veľkej skupine ľudí a to predovšetkým ľuďom z online komunity namiesto využívania vlastných zamestnancov. [9]

nosti a výkone. Spoločnosti ich môžu uprednostniť kvôli špičkovým funkciám, ktoré zatiaľ nie sú dostupné v open-source alternatívach. S vysokou kvalitou však prichádzajú aj vyššie finančné náklady – licenčné poplatky môžu zahŕňať priebežné výdavky na aktualizácie, podporu a škálovanie infraštruktúry. Pre firmy bez skúseností s údržbou a správou LLM môže byť tento model výhodný, pretože dodávateľ zabezpečuje pravidelnú optimalizáciu a podporu modelu.

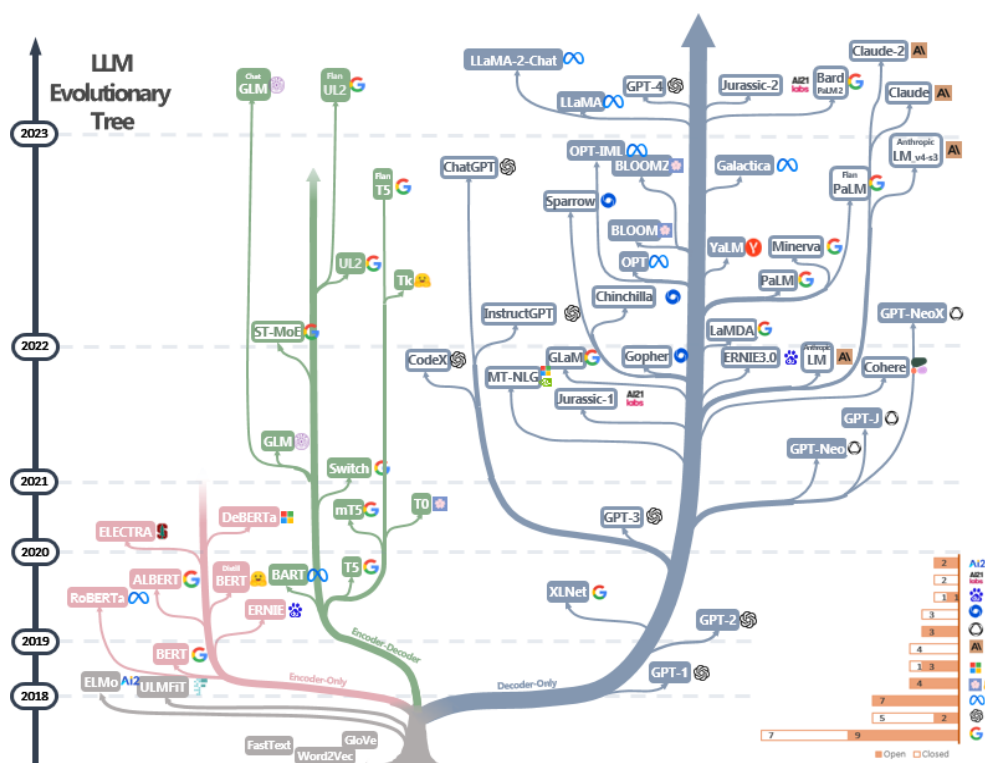
Bezpečnosť v prípade uzavretých modelov je riadená dodávateľom, čo môže byť výhodou pre spoločnosti bez veľkého IT oddelenia. Predajcovia často poskytujú certifikáty o zhode s reguláciami, čím sa zjednodušuje dodržiavanie právnych predpisov. Nevýhodou však môže byť menší prehľad o potenciálnych bezpečnostných hrozbách a závislosť na dodávateľovi pri opravách a aktualizáciách zabezpečenia. Spoliehanie sa na vendor-lock-in ³ môže tiež obmedziť flexibilitu podnikov pri budúcich technologických rozhodnutiach. [11, 13]

Výber medzi otvoreným a uzavretým modelom teda závisí od konkrétnych potrieb organizácie. Ak je cieľom nízka cena, prispôsobiteľnosť a kontrola nad bezpečnosťou, otvorené modely môžu byť ideálnym riešením. Ak spoločnosť hľadá špičkový výkon, jednoduchú správu a záruku kvality, môže sa rozhodnúť pre uzavretý model.

2.4 Architektúra

Veľké jazykové modely (LLM) sú užitočné nástroje na generovanie ľudskej reči a spracovanie prirodzeného jazyka. Využívajú pokročilé metódy umelej inteligencie na analýzu a produkciu textu, čím nachádzajú uplatnenie v rôznych oblastiach, ako sú automatizované chatboty, strojový preklad, sumarizácia textu či tvorba kreatívneho obsahu. Zatiaľ čo prvé jazykové modely stavali na rekurentných a konvolučných neurónových sieťach modely postavené na transformátorovej architektúre vďaka zlepšeniu výsledkov, či efektívnejšiemu trénovaniu predstavujú revolučný pokrok v oblasti LLM. [14] V dôsledku týchto výhod sa transformery stali základom pre väčšinu moderných LLM modelov a dnes tvoria jadro väčšiny pokročilých systémov umelej inteligencie pre prácu s textom.

³Vendor lock-in je situácia, keď je zákazník prakticky nútený využívať produkt alebo službu s ktorou už nie je spokojný. [12]



Obr. 2.1 Evolučný strom moderných LLM sleduje vývoj jazykových modelov v posledných rokoch a zvyrazňuje niektoré z najznámejších modelov. Modely na rovnakej vetve majú užšie vzťahy. Modely založené na transformátoroch sú zobrazené v iných než šedých farbách: modely len s dekodérom v modrej vetve, modely iba s kódovacím zariadením v ružovej vetve a modely s kódovačom a dekodérom v zelenej vetve. Vertikálna poloha modelov na časovej osi predstavuje dátumy ich vydania. Open-source modely sú reprezentované plnými štvorcami, zatiaľ čo uzavreté-source modely sú reprezentované dutými. Skladaný stĺpcový graf v pravom dolnom rohu zobrazuje počet modelov od rôznych spoločností a inštitúcií. Získané z [13]

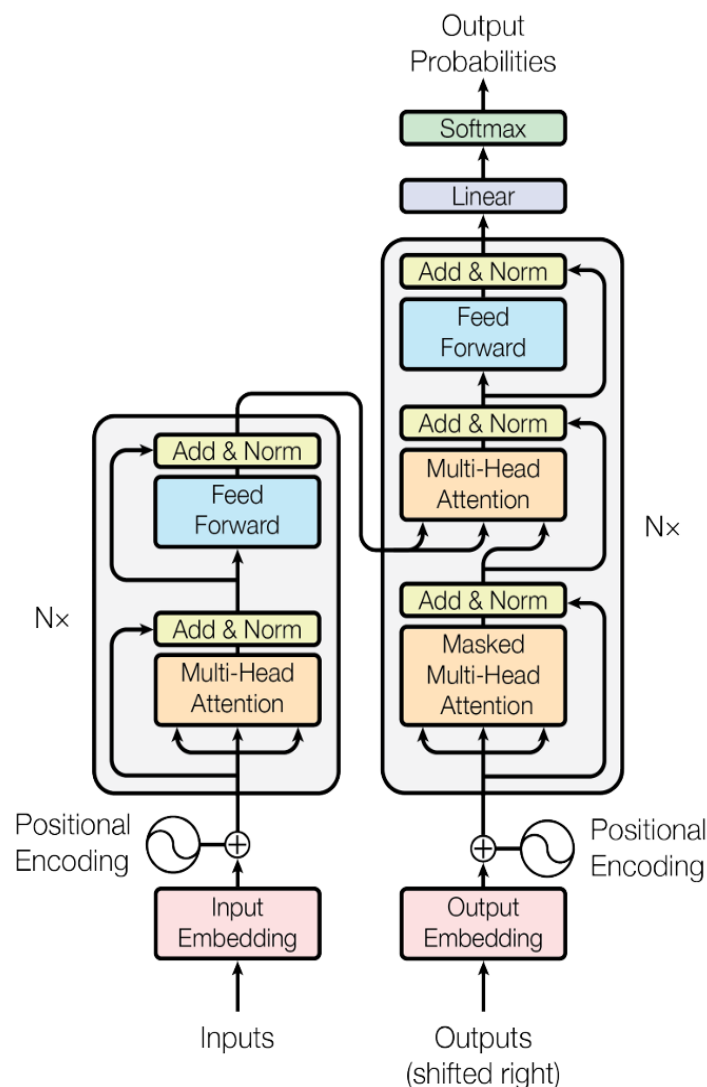
2.4.1 Transformer

Transformer je architektúra neurónových sietí, ktorá bola predstavená v práci Attention Is All You Need [14] od spoločnosti Google a zásadne ovplyvnila oblasť spracovania prirodzeného jazyka (NLP). Hlavným dôvodom ich úspechu je zlepšená výpočtová efektivita a paralelizácia v porovnaní s predchádzajúcimi sekvenčnými modelmi. [15] Transformátorová architektúra začala byť rozsiahle používaná v moderných LLM [1], pričom umožňuje modelom efektívne spracovávať veľké množstvá textových dát a učiť sa z rozsiahlych jazykových korpusov. Vďaka mechanizmu sebaopozornosti (self-attention) dokážu tieto modely rozpoznávať kontext na veľké vzdialenosti, čo výrazne zlepšuje kvalitu generovaného textu. Tento prístup nielenže umožňuje presnejšiu reprezentáciu významu slov v rôznych kontextoch, ale aj podporuje škálovateľnosť mode-

lov na masívne parametre, čo vedie k neustálemu zlepšovaniu ich schopností a praktickej využiteľnosti. [14]

Kľúčovou vlastnosťou transformera je vrstvená architektúra, ktorá sa skladá z mechanizmov sebaopozornosti (self-attention layers) a bodových dopredných neurónových sietí (point-wise feed-forward layers). Tieto vrstvy sú prítomné v kóderi aj dekodéri, ktoré spolupracujú pri transformácii vstupných dát na požadovaný výstup. Na rozdiel od rekurentných modelov, ktoré spracovávajú slová sekvenčne, transformer využíva paralelizmus a pozičné kódovanie na zachovanie poradia tokenov v sekvencii. [14]

Existujú tri hlavné varianty transformátorovej architektúry v závislosti od spôsobu využitia kódera (encoder) a dekodera (decoder): enkóder-dekóder (encoder-decoder), len enkóder (encoder-only) a len dekóder (decoder-only). Architektúra enkóder-dekóder sa používa najmä pri úlohách typu vstup-výstup, ako je napríklad strojový preklad, kde enkóder spracuje celý vstupný text a dekóder následne generuje výstup. Encoder-only modely, ako napríklad BERT, sú optimalizované na porozumenie textu. Využívajú sa na úlohy ako klasifikácia textu, analýza sentimentu alebo vyhľadávanie odpovedí. Na druhej strane, decoder-only architektúry, ako GPT-2 a GPT-3, sú prispôsobené na generovanie textu, kde model predikuje ďalší token na základe predchádzajúcich, čo je ideálne pre aplikácie ako tvorba textu, dopĺňanie viet či konverzačné asistenty. Každý z týchto variantov má svoje špecifiká a výhody v závislosti od konkrétneho typu NLP úlohy, ktorú rieši. [1, 6, 13]



■ Obr. 2.2 Transformer architektúra. Získané z [14]

2.4.1.1 Vkladacia (embedding) vrstva

Embedding vrstva je prvá vrstva, ktorá spracováva vstupnú sekvenciu. Najprv vstup prejde procesom tokenizácie, pri ktorom sa rozdelí na jednotlivé tokeny. Tokeny môžu byť slová, slabiky alebo znaky, ktoré sú následne asociované s vektorovou reprezentáciou (napríklad pomocou metód ako Word2Vec alebo GloVe). Týmto procesom vzniknú embedding vektory s vysokou dimenziou, ktoré možno interpretovať ako súradnice v vysoko dimenzionálnom priestore. Zvyčajne slová, ktoré majú podobný význam (napríklad pomenúvajú rovnakú vec), sa nachádzajú blízko seba vo vektorovom priestore. [15, 16] Tento krok

zachytáva sémantické a syntaktické informácie o vstupe.

2.4.1.2 Pozičné kódovanie

Pozičné kódovanie je mechanizmus, ktorý sa pridáva k embedding vektorom, aby poskytoval informácie o relatívnych alebo absolútnych pozíciách tokenov v sekvencii. Keďže transformátory, na rozdiel od rekurentných neurónových sietí (RNN), nespracovávajú sekvenčné dáta postupne, ale paralelne, nemajú inherentnú znalosť poradia tokenov. Pozičné kódovanie preto zabezpečuje, že model dokáže rozpoznať sekvenčné vzťahy medzi tokenmi. [15]

Najčastejšie sa používa sínusovo-kosínusové pozičné kódovanie, ktoré využíva trigonometrické funkcie na generovanie pozícií tak, aby rôzne dĺžky sekvencií mali jedinečné reprezentácie. Každý token je teda reprezentovaný kombináciou svojej embedding reprezentácie a príslušného pozičného vektora. Tento prístup umožňuje modelu generalizovať na dlhšie sekvencie, než na akých bol tréňovaný.

Alternatívou sú učené pozičné vektory, kde sa pozičné kódovanie učí ako súčasť modelu, čím sa môže lepšie prispôsobiť konkrétnym úlohám. [14]

2.4.1.3 Kódovač

Kódovač analyzuje vstupný text a vytvára množstvo skrytých stavov, ktoré zachovávajú kontext a význam textových dát. Na základe kontextu tokenov vo vstupnej sekvencii kódovač upravuje príslušné vektory na základe ich vzájomnej závislosti v tejto sekvencii. [17]

Kódovač sa skladá zo zásobníka obsahujúceho množstvo rovnakých vrstiev. Každá vrstva má dve podvrstvy – prvou je spomínaná self-attention vrstva a druhou dopredná neurónová sieť. Okolo každej z týchto podvrstiev sú aplikované reziduálne spojenia, po ktorých nasleduje normalizačná vrstva. [14]

Self-attention vrstva umožňuje modelu zvážiť závislosti a vzťahy medzi tokenmi v kontexte celej vstupnej sekvencie pomocou výpočtu skóre pozornosti. Funkciu pozornosti možno opísať ako mapovanie požiadavky a množiny párov kľúč–hodnota na výstup, pričom požiadavky, kľúče, hodnoty a výstup sú reprezentované vektormi. Výstup, nazývaný aj Scaled Dot-Product Attention, sa vypočíta, ako vážený súčet hodnôt, pričom váha priradená každej hodnote závisí od kompatibility požiadavky s príslušným kľúčom. Keďže často počítame pozornosť pre viaceré požiadavky súčasne, požiadavky sú uložené v matici Q s dimenziou d_k , pričom ten istý princíp platí pre kľúče v matici K s dimenziou d_k a hodnoty v matici V s dimenziou d_v .

Výstup je rovnakého tvaru ako matica V . Nasledujúca rovnosť je rovnaká od bežnej Dot-Product Attention s výnimkou faktora škálovania $\sqrt{d_k}$. [14]

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.1)$$

Zdroj: [14]

Dopredná (Feed-Forward) neuronová sieť je obsiahnutá v každom kóderi a dekodéri a aplikuje sa nezávisle na každý token, obdržaný z self-attention vrstvy, samostatne a rovnakým spôsobom. Táto sieť pozostáva z dvoch lineárnych transformácií, medzi ktorými sa nachádza aktivačná funkcia ReLU (REctified Linear Unit). Aj keď sú lineárne transformácie rovnaké pre všetky pozície v sekvencii, parametre sa menia medzi jednotlivými vrstvami, čo umožňuje modelu zachytiť komplexné interakcie medzi tokenmi. [14, 15]

Normalizačná vrstva sa aplikuje po každej podvrstve (self-attention a doprednej neuronovej sieti). Pomáha tak stabilizovať proces učenia a zlepšuje schopnosť modelu zovšeobecňovať rôzne vstupy. [15, 16]

2.4.1.4 Multi-Head Attention vrstva

Je rozšírením self-attention vrstvy, ktorá paralelne využíva viacero self-attention mechanizmov. Tento prístup umožňuje modelu zachytiť rôzne typy vzťahov a súčasne sa zamerať na rôzne časti vstupnej sekvencie. [15] Na rozdiel od základnej self-attention vrstvy, multi-head attention umožňuje spracovanie tokenov v rôznych reprezentačných priestoroch, čo vedie k lepšiemu pochopeniu kontextu. [14] Vzorec na výpočet je podobný tomu pri self-attention, ale je aplikovaný na viaceré hlavy (heads), pričom výsledky jednotlivých hláv sa následne spoja a ďalšou transformáciou sa prevedú na finálnu reprezentáciu.

Q - matica požiadaviek, reprezentuje aktuálny token

K - matica kľúčov, predstavuje všetky možné referenčné body pre požiadavku

V - matica hodnôt, obsahuje informácie, ktoré sa extrahujú na základe podobnosti Q a K

W_Q^i, W_K^i, W_V^i - naučené transformačné matice pre jednotlivé hlavy pozornosti, ktoré premietajú vstup do nižšej dimenzie d_k a d_v

W_O - váhová matica, ktorá transformuje spojenie všetkých hláv späť do pôvodného priestoru dimenzie d_m

h - počet hláv pozornosti

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (2.2)$$

Zdroj: [14]

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i) \quad (2.3)$$

Zdroj: [14]

2.4.1.5 Dekodér

Dekodér sa takisto skladá zo zásobníka, ktorý obsahuje niekoľko rovnakých vrstiev. Okrem dvoch podvrstiev v každej vrstve kódera pridáva dekodér tretiu vrstvu (Multi-Head Attention), ktorá vykonáva pozornosť viacerých hláv na výstupe zásobníka kódera. Toto označuje rozdiel medzi kódérom a dekodérom. Zatiaľ čo kódovač generuje kontextové vkladanie tokenov súčasne, vrstva pozornosti s viacerými hlavami dekodéra využíva autoregresívne maskovanie.

Podobne ako v kóderi, aj v dekodéri aplikujeme reziduálne spojenia okolo každej podvrstvy, následne nasleduje normalizácia.

Taktiež modifikujeme každú self-attention vrstvu v zásobníku dekodéra, aby sme zabránili ovplyvneniu budúcich tokenov pri spracovávaní aktuálneho tokenu. Toto maskovanie v kombinácii s tým, že výstupné vloženia sú posunuté o jednu pozíciu, zaisťuje, že predpovede pre token na pozícii i závisia len od známych výstupov na pozíciách tokenov menších ako i . Cieľom, ako v prípade mnohých techník umelej inteligencie, je napodobniť chápanie ľudí pri spracovaní informácií pomocou jazyka. Tento prístup k spracovaniu informácií sa nazýva autoregresný. [14]

2.4.1.6 Softmax

Posledná vrstva je zvyčajne funkcia softmax, ktorá konvertuje výstup modelu na rozdelenie pravdepodobnosti v slovníku, čo mu umožňuje vybrať najpravdepodobnejší ďalší token alebo vyplniť maskovaný token. [15]

2.4.2 Encoder-decoder architektúra

Kóder-dekodér architektúra je typ neurónovej siete, ktorá je najčastejšie spojená s architektúrou transformátora a využíva sa pri učení sa sekvencií. Modely kóder-dekodér sa používajú na spracovanie sekvenčných dát, konkrétne na mapovanie vstupných sekvencií na výstupné sekvencie rôznych dĺžok, ako je strojový preklad, sumarizácia textu, popisovanie obrázkov alebo rozpoznávanie reči. V takýchto úlohách býva mapovanie medzi tokenom na vstupe a tokenom na výstupe často nepriame. Hlavnou výhodou kóder-dekodér architektúry pri riešení NLP problému oproti iným (napríklad bag of words modelu) je vytváranie takzvaných kontextuálnych vložení (embeddings), ktoré zohľadňujú poradie slov vo vstupnej sekvencii pomocou pozičného kódovača. Sieť kóder-dekodér generuje výstupné sekvencie s premenlivou dĺžkou, ktoré sú kontextovo vhodné a zodpovedajú danej vstupnej sekvencii. Model kóder-dekodér zvyčajne obsahuje niekoľko kóderov a dekodérov. [17]

2.4.3 Encoder-only architektúra

Modely typu encoder-only, tiež označované ako automatické kódovače, používajú iba zásobník kódovačov, teda bez dekodérov. Týmto typom modelov chýba autoregresívne maskovanie a teda vrstvy pozornosti majú v každej fázy prístup k všetkým tokenom vstupnej sekvencie. Ako také sú tieto modely opísané ako obojsmerné, pretože používajú všetky okolité tokeny, predchádzajúce aj nasledujúce, na vytváranie predpovedí pre daný token. Preto predtrénovanie týchto modelov sa zvyčajne točí okolo nejakého poškodenia danej vety (napríklad maskovaním náhodných slov v nej) a poverenia modelu nájdením alebo rekonštrukciou pôvodnej vety. Tieto modely sú najvhodnejšie pre úlohy,

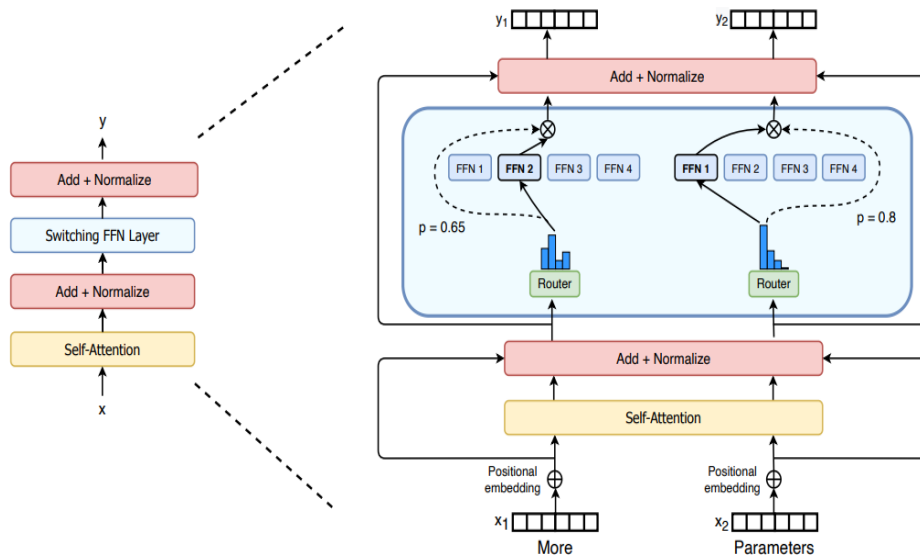
ktoré si vyžadujú pochopenie celého textového vstupu ako klasifikácia textu alebo rozpoznávanie pomenovaných entít. Príklady modelov založené na architektúre encoder-only sú od BERT rodiny, ako sú ALBERT, BERT, RoBERTa alebo aj ELECTRA. [17]

2.4.4 Decoder-only architektúra

Naopak od Encoder-only architektúry, Decoder-only architektúra používa iba zásobník dekodérov, bez akýchkoľvek kódovačov. Takže pri vytváraní predpovedí tokenov môžu vrstvy pozornosti modelu pristupovať iba k tým tokenom, ktoré predchádzajú uvažovanému tokenu. Vstupné a výstupné tokeny sú spracované rovnakým spôsobom prostredníctvom dekodéra. Modely len s dekodermi sa často používajú na úlohy generovania textu, ako je odpovedanie na otázky, písanie kódu alebo chatboty, ako je ChatGPT. Ako reprezentatívne jazykové modely tejto architektúry sú teda modely série GPT vyvinuté na základe architektúry Decoder-only. Najmä GPT-3 úspešne demonštroval účinnosť tejto architektúry, pričom tiež ukazuje úžasnú schopnosť LLM učenia sa v kontexte. [17, 1]

2.4.5 Mixture-of-Experts

Architektúra Mixture of Experts (MoE) predstavuje efektívny spôsob škálovania veľkých jazykových modelov, pri ktorom sa namiesto aktivácie celého modelu pri každom vstupe využíva len vybraná podmnožina tzv. expertov, špecializovaných podsietí. Vrstvy MoE majú určitý počet expertov, pričom každý expert je neurónová sieť. V praxi sú expertmi dopredné neuronové siete, ale môžu to byť aj zložitejšie siete. [1, 6] Kľúčovým komponentom je smerovacia sieť (router), ktorá rozhoduje, ktoré tokeny sa odošlú ktorým expertom. Týmto spôsobom možno výrazne znížiť výpočtové náklady a zároveň zvýšiť počet parametrov modelu bez zodpovedajúceho nárastu inferenčného času. Hoci MoE môže mať veľa parametrov, počas inferencie sa používajú iba niektoré z nich. To vedie k oveľa rýchlejšiemu odvodeniu v porovnaní s hustým modelom s rovnakým počtom parametrov. [18]



■ **Obr. 2.3** Ilustrácia Mixture-of-Experts architektúry bloku kódovača, kde namiesto hustej dostupnej neuronovej siete je vrstva expertov so smerovačom (bledo modrá). Vrstva operuje nezávisle na jednotlivých tokenoch vo vstupnej sekvencii. Získané z [14]

2.5 Predtrénovanie

Predtrénovanie (pre-training) vytvára základ schopností LLM. Predtrénovaním na rozsiahlych korpusoch môžu LLM získať základné jazykové znalosti a generačné zručnosti. V tomto procese sú rozsah a kvalita predtrénovacieho korpusu rozhodujúce pre LLM na dosiahnutie výkonných schopností. Okrem toho, na efektívne predtrénovanie LLM musia byť dobre navrhnuté modelové architektúry, akceleračné metódy a optimalizačné techniky. [1, 19] V tejto kapitole rozoberieme jednotlivé fázy tréningu počnúc zberom a spracovaním dát a predstavíme tréningové techniky na stabilnú a efektívnu optimalizáciu LLM.

2.5.1 Zber dát

Na rozvoj schopného LLM je kľúčové zhromaždiť veľké množstvo korpusu prirodzeného jazyka z rôznych dátových zdrojov. Existujúce LLM využívajú predovšetkým zmes rôznych verejných textových dátových súborov, ako korpus na predtrénovanie. Zdroj predtrénovacieho korpusu možno vo všeobecnosti rozdeliť do dvoch typov na všeobecné dáta a špecializované dáta. **Všeobecné dáta**, ako sú webové stránky, knihy a konverzačný text, využíva väčšina LLM kvôli ich veľkej, rôznorodej a dostupnej povahe, ktorá môže zlepšiť schopnosti LLM modelovať jazyk a zovšeobecňovať. **Špecializovanejšie súbory** dát rozširujú

ich predtrénovový korpus, ako sú viacjazyčné dáta, vedecké dáta a kód, čím poskytujú LLM špecifické schopnosti riešenia úloh.

Prevažná väčšina LLM využíva všeobecné textové dáta na predtrénovanie, ako sú webové stránky, knihy a konverzačný text, ktoré poskytujú bohaté textové zdroje na rôzne témy.

Webové stránky umožňujú získať rôznorodé lingvistické znalosti a zlepšiť ich možnosti zovšeobecňovania. Pre pohodlné používanie týchto dátových zdrojov sa prehľadáva veľké množstvo dát z webu, ako napríklad CommonCrawl ⁴. Prehľadávané webové dáta však zvyčajne obsahujú vysokokvalitný text, ale aj text nízkej kvality, preto je dôležité filtrovať a spracovávať webové stránky na zlepšenie kvality dát. [19]

Ďalším špecifickým zdrojom dát je **konverzačný text**. Dáta o konverzácií môžu zlepšiť konverzačnú kompetenciu LLM a potenciálne zlepšiť ich výkon v celom rade úloh zameraných na zodpovedanie otázok. Využívané môžu byť podmnožiny korpusu verejnej konverzácie (napr. korpus PushShift.io ⁵). Potenciálnym rizikom je, že nadmerná integrácia dát dialógu do LLM môže mať za následok vedľajší účinok a to deklaratívne pokyny a priame výsluchy sú mylne vnímané ako začiatok konverzácie, čo vedie k poklesu účinnosti pokynov.

Knihy v porovnaní s inými korpusmi poskytujú dôležitý zdroj formálnych dlhých textov, ktoré sú potenciálne prospešné pre LLM na učenie sa jazykových znalostí, modelovanie dlhodobej závislosti a vytváranie naratívnych a súvislých textov. K príkladom takýchto datasetov patrí napríklad Pile ⁶ dataset.

Špecializované súbory dát sú užitočné na zlepšenie špecifických schopností LLM. Aby sa zlepšilo porozumenie vedeckých poznatkov vrámci LLM, je užitočné začleniť **vedecký korpus** pre modelový predtrénovanie. Predtrénovanie na obrovské množstvo vedeckých textov môžu LLM dosiahnuť pôsobivý výkon vo vedeckých a logických úlohách. Na vybudovanie vedeckého korpusu existujúce snahy zbierajú najmä články arXiv, vedecké učebnice a ďalšie súvisiace vedecké zdroje. K špecifickým dátam patria aj **kódové korpusy**, ktoré môže viesť k podstatnému zlepšeniu kvality syntetizovaných programov. [20] Bežne sa používajú dva typy korpusov kódu. Prvý zdroj pochádza z programovacích komunit, ako je Stack Exchange. Druhý zdroj pochádza z verejných softvérových úložísk, ako je GitHub, kde sa zhromažďujú kódové dáta (vrátane komentárov a dokumentačných reťazcov) na použitie. [1]

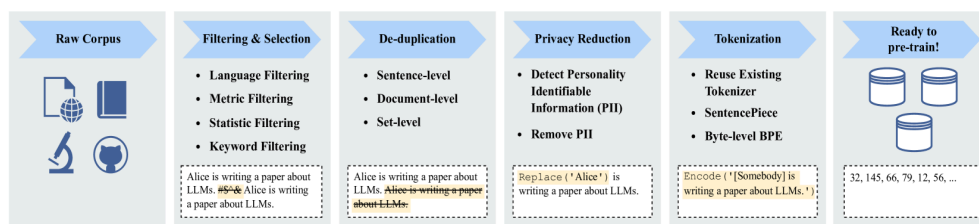
⁴CommonCrawl spravuje bezplatné, otvorené úložisko dát o prehľadávaní webu, ktoré môže používať ktokoľvek.

⁵Pushshift je nástroj, ktorý možno použiť na zhromažďovanie analýz a archiváciu dát z Redditu.

⁶The Pile je rôznorodý, open source súbor dát anglického textu vytvorený ako tréningový súbor dát pre veľké jazykové modely.

2.5.2 Predspracovanie dát

Po zozbieraní veľkého množstva textových dát je nevyhnutné dáta predspracovať na zostavenie predtrénovacieho korpusu, najmä odstránenie hlučných, nadbytočných, irelevantných a potenciálne toxických dát, ktoré môžu do značnej miery ovplyvniť kapacitu a výkon LLM. [1, 13] Typický postup predspracovania dát do predtrénovacieho korpusu pre LLM je znázornený na obrázku 2.4.



■ **Obr. 2.4** Ilustrácia typickej pipeline na predspracovanie dát na predtrén veľkých jazykových modelov. Získané z [1]

2.5.2.1 Filtrovanie a výber

Pri vytváraní veľkých jazykových modelov (LLM) je mimoriadne dôležité, aby boli tréované na kvalitných a relevantných dátach. Internetový textový obsah, z ktorého sa tieto dáta často čerpajú, však obsahuje aj množstvo nekvalitných, neúplných alebo nevhodných dát. Preto je nevyhnutné tieto dáta pred samotným tréovaním modelu dôkladne prefiltrovať. V praxi sa na tento účel využívajú dva hlavné prístupy – klasifikátorový a heuristický. [1, 19]

Klasifikátorový prístup využíva strojové učenie na vytvorenie modelu, ktorý sa naučí rozpoznávať kvalitné texty. Tento model (klasifikátor) sa trénuje na vzorkách kvalitných dát – napríklad textoch z Wikipédie alebo synteticky vytvorených vzorových dátach – a potom sa používa na odhad kvality ostatných dát. Na základe toho, ako model skóruje konkrétny text, sa rozhoduje, či sa text ponechá alebo vyradí. Výhodou tohto prístupu je schopnosť odhaliť nekvalitné alebo nezmyselné texty, avšak má aj nevýhody. Napríklad môže omylom vyradiť texty v nárečiach, hovorovom jazyku alebo texty s odlišným štýlom, čo môže viesť k zníženiu jazykovej a štýlovej rôznorodosti v tréovacích dátach. [1]

Heuristický prístup sa naopak nespolieha na tréovaný model, ale na ručne navrhnuté pravidlá. Napríklad sa odstraňujú texty, ktoré nie sú v cieľovom jazyku (jazykové filtrovanie), obsahujú nezmyselné alebo chaotické vety (napr. na základe metriky zmätenosti), majú zvláštnu štatistickú štruktúru (napr. príliš veľa interpunkcie, krátke vety bez významu), alebo obsahujú neželané prvky, ako HTML značky, odkazy, nevhodné výrazy a pod. Tento prístup je jednoduchší na implementáciu a zrozumiteľnejší, ale môže byť menej flexibilný a nie vždy dostatočne presný pri odhaľovaní nekvalitného obsahu. [1]

Okrem týchto dvoch prístupov sa v poslednej dobe objavuje aj možnosť použiť samotné jazykové modely na výber kvalitných dát. Takéto modely môžu byť explicitne vyzvané, aby odhadli dôležitosť alebo informatívnosť konkrétneho textového úseku. Táto technika však vyžaduje značné výpočtové zdroje a preto zatiaľ nie je bežne používaná pri filtrovaní veľkých objemov dát.

Celkovo je filtrovanie dát nevyhnutnou súčasťou procesu tréningu LLM, ktorá priamo ovplyvňuje ich výkonnosť, neustrannosť a schopnosť správne generalizovať naprieč rôznymi jazykmi a štýlmi. Výber vhodnej metódy filtrovania závisí od požiadaviek projektu, dostupných výpočtových prostriedkov a charakteru cieľovej úlohy. [1, 19]

2.5.2.2 Deduplikácia

Duplicitné dáta v korpuse by znížili rôznorodosť jazykových modelov, čo môže spôsobiť, že tréningový proces sa stane nestabilným, a tým ovplyvní výkon modelu. Preto je potrebné predtréningový korpus deduplikovať. Špeciálne, deduplikáciu možno vykonávať na rôznych úrovniach podrobnosti, vrátane deduplikácie na úrovni viet, dokumentu a množiny dát. Najprv by sa mali odstrániť vety nízkej kvality, ktoré obsahujú opakujúce sa slová a frázy, pretože môžu zaviesť opakujúce sa vzory pri modelovaní jazyka. Na úrovni dokumentov sa sústreďuje na pomer prekrývania povrchových znakov (napr. prekrývanie slov) medzi dokumentmi s cieľom odhaliť a odstrániť duplicitné dokumenty obsahujúce podobný obsah. [19] Okrem toho, aby sa predišlo problému kontaminácie množiny dát, je tiež veľmi dôležité zabrániť prekrývaniu medzi trénujúcou a hodnotiacou množinou, a to odstránením možných duplicitných textov z trénujúcej množiny. Ukázalo sa, že tri úrovne odstraňovania duplikátov sú užitočné na zlepšenie tréningu LLM, ktoré by sa mali spoločne využívať v praxi. [1, 19, 21]

2.5.2.3 Ochrana súkromia

Je potrebné z predtréningového korpusu odstrániť informácie umožňujúce identifikáciu osôb. Jedným z priamych a účinných prístupov je použitie metód založených na pravidlách, ako je napríklad vyhľadávanie kľúčových slov, na zistenie a odstránenie súkromných údajov, ako sú mená, adresy a telefónne čísla. Zraniteľnosť LLM pri útokoch na súkromie možno pripísať prítomnosti duplicitných súkromných údajov v predtréningovom korpuse. [21] Preto môže odstránenie duplikácie do určitej miery znížiť aj riziká ohrozujúce súkromie. [1]

2.5.2.4 Tokenizácia

Tokenizácia je tiež kľúčovým krokom pri predbežnom spracovaní dát. [22] Jej cieľom je segmentovať surový text na sekvencie jednotlivých tokenov, ktoré sa následne používajú ako vstupy LLM. V tradičnom výskume NLP prevláda tokenizácia založená na slovách, ktorá je viac v súlade s poznávaním ľudského

jazyka. Tokenizácia založená na slovách však môže v niektorých jazykoch pri-niesť rôzne výsledky segmentácie pre ten istý vstup. [1] Preto sa v jazykových modeloch založených na transformátore široko používajú tokenizátory podslov, ktoré zvyčajne zahŕňajú tokenizáciu v kódovaní Byte-Pair Encoding, tokeni-záciu WordPiece a tokenizáciu Unigram. [1, 22]

Byte-Pair Encoding (BPE) bol pôvodne navrhnutý ako všeobecný al-goritmus kompresie dát a potom bol prispôbosený na NLP na tokenizáciu. Začína sa súborom základných symbolov (napr. abecedy a hraničných znakov) a iteratívne sa kombinujú časté dvojice dvoch po sebe idúcich tokenov v kor-puse ako nové tokeny (tzv. merge). Pre každé zlúčenie je výberové kritérium založené na frekvencii spoločného výskytu dvoch susediacich tokenov: vyberie sa najfrekventovanejšia dvojica. Proces zlučovania pokračuje, kým nedosiahne vopred definovanú veľkosť. Ďalej sa BPE na úrovni bajtov použil na zlepše-nie kvality tokenizácie pre viacjazyčný korpus (napr. text obsahujúci iné ako ASCII znaky) tým, že sa bajty považujú za základné symboly na zlučovanie. Medzi reprezentatívne jazykové modely s týmto tokenizačným prístupom pat-ria GPT-2, BART, RoBERTa a LLaMA. [22, 1]

WordPiece bol interný algoritmus spoločnosti Google na tokenizáciu čiast-kových slov. Pôvodne ho navrhla spoločnosť Google pri vývoji systémov hla-sového vyhľadávania, no neskôr bol prijatý ako tokenizátor slov pre BERT. [23] WordPiece má veľmi podobnú myšlienku s BPE iteratívnym spájaním po sebe idúcich tokenov, pričom pre spájanie berie trochu iné výberové kritérium. Na vykonanie zlučovania najprv vycvičí jazykový model a použije ho na vyhod-notenie všetkých možných dvojíc. Potom pri každom zlúčení vyberie dvoj-icu, ktorá vedie k najväčšiemu zvýšeniu pravdepodobnosti trénovaných údajov. Dvojica je ohodnotená vydelením počtu spoluvýskytov súčinom počtu výsky-tov dvoch tokenov v dvojici na základe tréningového korpusu. [22, 1]

Unigram na rozdiel od BPE a WordPiece začína s dostatočne veľkou mno-žinou možných podreťazcov alebo podtextov pre korpus a iteratívne odstraňuje tokeny v aktuálnom slovníku, kým sa nedosiahne očakávaná veľkosť slovníka. Ako výberové kritérium počíta získané zvýšenie pravdepodobnosti trénova-ného korpusu za predpokladu, že z aktuálneho slovníka bol odstránený nejaký token. Tento krok sa vykonáva na základe natrénovaného unigramového jazy-kového modelu. Na odhadnutie unigramového jazykového modelu používa al-goritmus maximalizácie očakávaní pri každej iterácii najprv nájdeme aktuálne optimálnu tokenizáciu slov na základe starého jazykového modelu a potom znovu odhadneme pravdepodobnosti unigramov na aktualizáciu jazykového modelu. Počas tohto postupu sa používajú algoritmy dynamického programo-vania na efektívne nájdenie optimálneho spôsobu rozkladu slova vzhľadom na model jazyka. [22, 1]

2.5.3 Data Scheduling

Po predspracovaní dát je ďalším krokom v trénovaní veľkých jazykových modelov (LLM) nastavenie poradia a pomeru rôznych typov dát – tzv. data scheduling. Nájsť optimálne usporiadanie nie je jednoduché, preto sa často najprv trénujú menšie modely s rôznymi variantmi plánov, z ktorých sa následne vyberie ten najvhodnejší. Počas tréningu je možné sledovať výkonnosť modelu na vybraných úlohách a podľa toho dynamicky upravovať zloženie a distribúciu tréningových dát. Dôležitou súčasťou je aj skúmanie vzťahu medzi typom dát a schopnosťami modelu, čo pomáha lepšie navrhnúť stratégiu učenia. [1]

2.5.4 Trénig

Trénig veľkého jazykového modelu (LLM) zahŕňa optimalizáciu hlbokéj neurónovej siete na predpovedanie ďalšieho tokenu v sekvencii. Od spustenia transformera boli navrhnuté rôzne vylepšenia na zlepšenie jeho tréningovej stability, výkonu a výpočtovej efektivity. [1] Na základe modelu sa pri tréningu používajú rôzne aktivačné funkcie, normalizačné metódy, metódy pozornosti, či pozičné metódy vloženia. Pre lepšiu prehľadnosť rôznych techník ponúkame obrázok 2.5, ktorý zohľadňuje najpoužívanéjšie metódy na úrovni rôznych vrstiev. Ďalej si jednotlivé metódy stručne popíšeme.

Configuration	Method	Equation
Normalization position	Post Norm	$\text{Norm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$
	Pre Norm	$\mathbf{x} + \text{Sublayer}(\text{Norm}(\mathbf{x}))$
	Sandwich Norm	$\mathbf{x} + \text{Norm}(\text{Sublayer}(\text{Norm}(\mathbf{x})))$
Normalization method	LayerNorm	$\frac{\mathbf{x} - \mu}{\sigma} \cdot \gamma + \beta, \quad \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$
	RMSNorm	$\frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \gamma, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$
	DeepNorm	$\text{LayerNorm}(\alpha \cdot \mathbf{x} + \text{Sublayer}(\mathbf{x}))$
Activation function	ReLU	$\text{ReLU}(\mathbf{x}) = \max(\mathbf{x}, \mathbf{0})$
	GeLU	$\text{GeLU}(\mathbf{x}) = 0.5\mathbf{x} \otimes [1 + \text{erf}(\mathbf{x}/\sqrt{2})], \quad \text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$
	Swish	$\text{Swish}(\mathbf{x}) = \mathbf{x} \otimes \text{sigmoid}(\mathbf{x})$
	SwiGLU	$\text{SwiGLU}(\mathbf{x}_1, \mathbf{x}_2) = \text{Swish}(\mathbf{x}_1) \otimes \mathbf{x}_2$
	GeGLU	$\text{GeGLU}(\mathbf{x}_1, \mathbf{x}_2) = \text{GeLU}(\mathbf{x}_1) \otimes \mathbf{x}_2$
Position embedding	Absolute	$\mathbf{x}_i = \mathbf{x}_i + \mathbf{p}_i$
	Relative	$A_{ij} = \mathbf{W}_q \mathbf{x}_i \mathbf{x}_j^T \mathbf{W}_k^T + r_{i-j}$
	RoPE	$A_{ij} = \mathbf{W}_q \mathbf{x}_i \mathbf{R}_{\Theta, i-j} \mathbf{x}_j^T \mathbf{W}_k^T = (\mathbf{W}_q \mathbf{x}_i \mathbf{R}_{\Theta, i})(\mathbf{W}_k \mathbf{x}_j \mathbf{R}_{\Theta, j})^T$
	ALiBi	$A_{ij} = \mathbf{W}_q \mathbf{x}_i \mathbf{x}_j^T \mathbf{W}_k^T - m(i-j)$

■ **Obr. 2.5** Sublayer označuje podvrstvu typu FFN (Feed-Forward Network) alebo samopozorováciu (self-attention) jednotku v transformerovej vrstve, d označuje veľkosť skrytých stavov, Norm označuje jednu z normalizačných metód, p_i označuje pozičné zapuzdrenie (embedding) na pozícii i , A_{ij} označuje skóre pozornosti medzi požiadavkou (query) a kľúčom (key), r_{i-j} označuje naučiteľný skalár na základe rozdielu medzi požiadavkou a kľúčom, a $\mathbf{R}_{\Theta, t}$ označuje rotačnú maticu s uhlom rotácie $t \cdot \Theta$. Získané z [1]

2.5.4.1 Normalizačné metódy

Nestabilita tréningu a časová náročnosť je jedným z problémov pre predtréninové LLM. Jedným zo spôsobov, ako skrátiť čas tréningu, je normalizovať činnosť neurónov. Normalizácia je široko prijatou stratégiou na stabilizáciu a skrátenie času tréningu neurónových sietí. [24] Vo transformátore sa používa LayerNorm alebo niekoľko pokročilých normalizačných techník ako alternatív k LayerNorm napr. RMSNorm a DeepNorm.

LayerNorm na vykonanie vrstvenej normalizácie špecificky vypočíta priemer a rozptyl použitý na normalizáciu zo všetkých sumárnych vstupov do neurónov vo vrstve na jednom tréňovanom prípade. LayerNorm dokáže zvládnuť opätovné centrovanie a zmenu mierky vstupov aj váhovej matice. Dá sa jednoducho aplikovať aj na rekurentné neurónové siete tým, že sa štatistika normalizácie vypočíta samostatne v každom časovom kroku. [24]

RMSNorm regularizuje sumárne vstupy do neurónu v jednej vrstve podľa kvadratického priemeru (root-mean-square — RMS), čím dáva modelu vlastnosť invariantnosti opätovného škálovania a implicitnú schopnosť adaptácie miery učenia. RMSNorm je výpočtovo jednoduchší kvôli nahradeniu priemeru a rozptylu za kvadratický priemer, a teda efektívnejší ako LayerNorm. [25] **DeepNorm** navrhla spoločnosť Microsoft na stabilizáciu tréňovania hlbokých transformátorov. Pomocou DeepNormu ako zvyškových spojení možno transformátory škálovať až na 1 000 vrstiev, čo preukázalo výhody stability a dobrého výkonu. [26]

2.5.4.2 Normalizačná poloha

Okrem metód normalizácie zohráva v LLM kľúčovú úlohu aj normalizačná poloha. Vo všeobecnosti existujú tri možnosti normalizačnej pozície, a to post-LN, pre-LN a sandwich-LN, ktoré rôzne používajú jednotlivé normalizačné metódy.

Post-LN sa používa v transformátore, ktorý je umiestnený medzi zvyškovými blokmi. V existujúcich prácach sa však zistilo, že tréňovanie transformátorov s post-LN má tendenciu byť nestabilné kvôli veľkým gradientom v blízkosti výstupnej vrstvy. Preto sa post-LN v existujúcich LLM používa len zriedka, okrem kombinácie s inými stratégiami (napr. kombinácia post-LN s pre-LN).

Na rozdiel od post-LN sa **pre-LN** aplikuje pred každou podvrstvou a pred konečnou predikciou sa umiestni dodatočný LN. V porovnaní s post-LN sú transformátory s pre-LN pri tréňovaní stabilnejšie. Dosahuje však horšie výsledky ako varianty s post-LN. Napriek klesajúcemu výkonu väčšina LLM stále používa pre-LN kvôli stabilite pri tréňovaní.

Na základe pre-LN pridáva **sandwich-LN** pred reziduálne spojenia ďalší extra LN, aby sa predišlo problémom hodnôt vo výstupoch vrstvy transformera. Zistilo sa však, že sandwich-LN niekedy nedokáže stabilizovať tréňovanie LLM a môže viesť ku kolapsu tréňovania.

2.5.4.3 Aktivačná funkcia

Na dosiahnutie dobrého výkonu je potrebné správne nastaviť aktivačné funkcie aj v sieťach typu feed-forward. V existujúcich LLM sa široko používajú GeLU aktivácie. [27] Špeciálne v najnovších LLM sa využívajú aj varianty aktivácie GLU [28], najmä varianty SwiGLU a GeGLU, ktoré v praxi často dosahujú lepší výkon. V porovnaní s GeLU si však vyžadujú dodatočné parametre (približne 50%) v sieťach s doprednou aktiváciou. [1]

2.5.4.4 Pozičné vloženia

Keďže moduly vlastnej pozornosti v transformeri sú permutačne ekvivariantné, na vnášanie informácií o absolútnej alebo relatívnej polohe pri modelovaní sekvencií sa používajú pozičné vloženia.

Absolute position embedding. V transformátore sa používajú absolútne vložené polohy. V dolnej časti kodéra a dekodéra sa k vstupným embeddingom pridávajú absolútne pozičné embeddingy. V transformeri sú navrhnuté dve varianty absolútnych pozičných embeddingov, sínusový a naučený pozičný embedding, pričom druhý z nich sa bežne používa v existujúcich predtrénovaných jazykových modeloch. [1]

Relative position embedding. Na rozdiel od absolútnych pozičných embeddingov sa relatívne pozičné embeddingy generujú podľa posunov medzi kľúčmi a požiadavkami. Výpočet skóre pozornosti medzi kľúčmi a požiadavkami bol upravený tak, aby sa zaviedli naučiteľné embeddingy zodpovedajúce relatívnym pozíciám. Konkrétne pridáva k skóre pozornosti učenlivé skaláre, pričom skaláre sú vypočítané na základe vzdialeností medzi pozíciami požiadavky a kľúča. [29]

Rotary position embedding (RoPE) nastavuje špecifické rotačné matice na základe absolútnej polohy každého kľúča alebo požiadavky. Skóre medzi kľúčmi a požiadavkami možno vypočítať pomocou informácií o relatívnej polohe. [29] RoPE kombinuje každú po sebe idúcu dvojicu prvkov vo vektoroch požiadaviek a kľúčov ako dimenziu, takže pre pôvodné vloženie s dĺžkou d existuje $d/2$ dimenzií. Pre každú dimenziu $i \in \{1, \dots, d/2\}$, sa dvojica zapojených prvkov otáča na základe rotačného uhla $t \cdot \theta_i$, kde t označuje index pozície a θ_i je báza v danej dimenzii. Podľa sínusoidného pozičného zakódovania [14], RoPE definuje bázu θ_i ako umocnenie základu b (štandardne nastaveného na 10000): Množina báz Θ je definovaná ako:

$$\Theta = \left\{ \theta_i = b^{-\frac{2(i-1)}{d}} \mid i \in \left\{ 1, 2, \dots, \frac{d}{2} \right\} \right\} \quad \text{Zdroj: [1]} \quad (2.4)$$

Okrem toho definuje vzdialenosť potrebnú na otočenie o jeden celý cyklus (2π) pre každú dimenziu ako vlnovú dĺžku:

$$\lambda_i = 2\pi \cdot b^{\frac{2(i-1)}{d}} = \frac{2\pi}{\theta_i} \quad \text{Zdroj: [1]} \quad (2.5)$$

ALiBi sa navrhuje na zlepšenie extrapolácie transformátora. Podobne ako relatívne vkladanie pozícií skresľuje skóre pozornosti penalizáciou na základe vzdialeností medzi kľúčmi a požiadavkami. Na rozdiel od metód relatívneho pozičného vnorenia sú penalizačné skóre v ALiBi vopred definované bez akýchkoľvek trénovateľných parametrov. [30]

2.5.4.5 Pozornosť

Mechanizmus pozornosti je dôležitou súčasťou systému Transformer. Umožňuje tokenom v celej sekvencii vzájomne interagovať a vypočítať reprezentácie vstupnej a výstupnej sekvencie. [1, 29]

Full attention v transformátore je mechanizmus pozornosti vykonáva párovým spôsobom, pričom sa zohľadňujú vzťahy medzi všetkými dvojicami tokenov v sekvencii. Využíva škálovanú bodovo-produktovú pozornosť, v ktorej sú skryté stavy mapované na požiadavky, kľúče a hodnoty. Okrem toho transformer využíva namiesto jednoduchej pozornosti pozornosť viacerých hláv, pričom do rôznych hláv premieta požiadavky, kľúče a hodnoty s rôznymi projekciami. [14, 1]

Sparse attention. Zásadnou výzvou úplnej pozornosti (full attention) je kvadratická výpočtová zložitosť, ktorá sa stáva záťažou pri práci s dlhými sekvenciami. Preto sa navrhujú rôzne efektívne varianty transformátora na zníženie výpočtovej náročnosti mechanizmu pozornosti. Namiesto celej sekvencie sa každá požiadavka môže venovať len podmnožine tokenov na základe pozícií. [31]

Multi-query/grouped-query attention. Multi-query sa vzťahuje na variant pozornosti, kde rôzne hlavy zdieľajú rovnaké lineárne transformačné matice na kľúčoch a hodnotách. Dosahuje vyššiu rýchlosť vyvodzovania len s menšou stratou kvality modelu. Na dosiahnutie kompromisu medzi pozornosťou založenou na multi-query attention a multi-head attention sa preskúmala pozornosť grouped-query attention (GQA). V GQA sú hlavy priradené do rôznych skupín a tie hlavy, ktoré patria do rovnakej skupiny, budú zdieľať rovnaké transformačné matice. [1, 32]

PagedAttention. Keď sú LLM nasadené na servery, pamäť GPU je z veľkej časti obsadená tenzormi kľúča pozornosti a hodnôt uložených vo vyrovnávacej pamäti (nazývané KV cache). Hlavným dôvodom je, že vstupné dĺžky sú často rôzne, čo vedie k fragmentácii a problémom s nadmernou rezerváciou. Inšpirovaný klasickou technikou stránkovania v operačných systémoch, PagedAttention bol navrhnutý na zlepšenie efektivity pamäte a priepustnosti nasadených LLM. Podrobne, PagedAttention rozdeľuje každú sekvenciu na podsekvencie a zodpovedajúce vyrovnávacie pamäte KV týchto podsekvencií sú alokované do nesúvislých fyzických blokov. Technika stránkovania zvyšuje využitie GPU a umožňuje efektívne zdieľanie pamäte pri paralelnom vzorkovaní. [1]

Súhrn. Aby sme dali všetky tieto diskusie dokopy, zhrnieme návrhy z

existujúcej literatúry na podrobnú konfiguráciu. Pre silnejšie zovšeobecnenie a stabilitu tréningu sa odporúča zvoliť pred RMSNorm pre normalizáciu vrstvy a SwiGLU alebo GeGLU ako aktivačnú funkciu. Okrem toho sa LN nesmie použiť ihneď po vložení vrstiev, čo pravdepodobne spôsobí zhoršenie výkonu. Pokiaľ ide o vkladanie pozícií, RoPE alebo ALiBi je lepšia voľba, pretože funguje lepšie pri dlhých sekvenciách.

2.5.4.6 Princíp tréningovania

Kľúčovú úlohu zohráva tréning, ktorý zakóduje všeobecné znalosti z rozsiahleho korpusu do masívnych parametrov modelu. Tréning prebieha iteratívne v dvoch fázach. V doprednom priecho model robí predpovede pre dávku vzorových vstupov z tréningového súboru dát a stratová funkcia meria rozdiel alebo stratu medzi predikciami modelu pre každý vstup a možnými odpoveďami. Počas spätného šírenia sa používa optimalizačný algoritmus – zvyčajne gradientový zostup – na úpravu váh modelov v celej sieti, aby sa znížili straty. Tieto úpravy váh modelu sú spôsob, akým sa model učí. Proces sa opakuje vo viacerých tréningových epochách, kým sa model nepovažuje za dostatočne natrénovaný. [33] Na tréning LLM sa bežne používajú dve tréningové úlohy, konkrétne jazykové modelovanie (language modeling) a maskovacie modelovanie (masked modeling).

Language modeling úloha je najbežnejšie používaným cieľom na predbežné tréningovanie LLM iba s dekodérom. Vzhľadom na postupnosť tokenov $\mathbf{x} = \{x_1, \dots, x_n\}$ je cieľom úlohy autoregresne predpovedať cieľové tokeny x_i na základe predchádzajúcich tokenov $x < i$ v sekvencii. [1] Oproti masked modeling, ktorý náhodne maskuje jednotlivé tokeny a učí sa na základe predikcií maskovaného tokenu pomocou tokenov nachádzajúcich sa pred alebo za maskovaným, language modeling predikuje budúci token iba na základe predchádzajúcich. Všeobecným cieľom tréningovania je maximalizovať nasledujúcu pravdepodobnosť:

$$\mathcal{L}_{\text{LM}}(\mathbf{x}) = \sum_{i=1}^n \log P(x_i | \mathbf{x}_{<i}) \quad \text{Zdroj: [1]} \quad (2.6)$$

log-pravdepodobnosť každého tokenu x_i , podmieneného všetkými predchádzajúcimi tokenmi $x_{<i}$.

Masked modeling je takzvané obojsmerné, pretože vyžívajú okolité (nie len predošle) tokeny na predikciu zamaskovaného slova. Aby sme trénovali obojsmernú sieť, jednoducho náhodne maskujeme určité percento vstupných tokenov a potom tieto maskované tokeny predpovedáme. [23] Okrem konvenčného jazykového modelovania sa na predtrénovanie jazykových modelov široko využíva aj úloha masked modeling. [23] Vstupy $\tilde{x}$ pre úlohu sú narušené texty s náhodne maskovnými úsekmi. [34] Jazykové modely sú následne trénované na rekonštrukciu pôvodných tokenov x . Formálne je tréningová funkcia definovaná

nasledovne:

$$\mathcal{L}_{\text{MM}}(\mathbf{x}) = \log P(\tilde{\mathbf{x}} \mid \mathbf{x} \setminus \tilde{\mathbf{x}}) \quad \text{Zdroj: [1]} \quad (2.7)$$

2.5.5 Optimalizácia parametrov

Optimalizácia parametrov v LLM modeloch zohráva kľúčovú úlohu v procese učenia – jej cieľom je nájsť také hodnoty váh (parametrov siete), ktoré minimalizujú chybu (stratovú funkciu) na tréningových dátach. Efektívna optimalizácia zabezpečuje, že model dokáže správne generalizovať – teda nielen si zapamätať tréningové dáta, ale naučiť sa z nich vzory a pravidlá, ktoré aplikuje aj na neznáme vstupy. Optimalizačné algoritmy ako Adam, gradientný zostup (SGD) alebo ich varianty tak priamo ovplyvňujú kvalitu, stabilitu a rýchlosť tréningovania modelu. [35, 36, 34]

Adam (Adaptive Moment Estimation) je optimalizačný algoritmus, ktorý kombinuje výhody dvoch iných rozšírení stochastického gradientného zostupu, AdaGrad, ktorý dobre pracuje s sprasou gradientom a RMSProp, ktorý funguje dobre v on-line a nestacionárnom prostredí. Tento algoritmus sa považuje za jeden z najefektívnejších pri tréningu hlbokých neurónových sietí. Adam používa adaptívne momenty prvej a druhej derivácie gradientu, čím umožňuje optimalizáciu s rôznymi rýchlosťami učenia pre rôzne parametre modelu. Niektoré z výhod Adama sú, že veľkosti aktualizácií parametrov sú invariantné voči zmene mierky gradientu, jeho veľkosti krokov sú približne ohraničené hyperparametrom veľkosti kroku. [36]

Predtým ako sa pozrieme na optimalizátor AdamW najprv ujasníme, čo je *weight decay*. Weight decay je regularizačná technika, ktorá penalizuje veľké hodnoty váh v neurónovej sieti, čím pomáha predchádzať preučeniu (overfittingu). Funguje tak, že pri aktualizácii parametrov pridáva k stratovej funkcii penalizáciu. [35]

AdamW je rozšírenie optimalizátora Adam, ktoré bolo navrhnuté na zlepšenie regularizácie váh počas tréningovania modelov. Zatiaľ čo pôvodný Adam implementuje weight decay ako súčasť gradientu, AdamW rozdeľuje (decouples) weight decay od aktualizácie gradientu, čo vedie k efektívnejšiemu tréningu a lepšej generalizácii modelu. [35]

2.5.6 Generovanie výstupu

Po tréningu LLM je nutné použiť špecifickú stratégiu dekodovania na generovanie vhodného výstupu. S prevládajúcou architektúrou iba dekodéra popíšeme mechanizmus automatického regresného dekodovania. Keďže takéto LLM sú vopred tréňované na základe úlohy modelovania jazyka 2.6, základnou metódou dekodovania je nenásytné vyhľadávanie, ktoré predpovedá najpravdepodobnejší token v každom kroku na základe predtým vygenerovaných tokenov, formálne modelovaných ako:

$$x_i = \arg \max_x P(x \mid \mathbf{x}_{<i}) \quad \text{Zdroj: [1]} \quad (2.8)$$

Kde x_i je token s najvyššou pravdepodobnosťou v i -tom kroku generovania, podmienený kontextom $\mathbf{x}_{<i}$. Greedy vyhľadávanie vyberie token, ktorý má v danom kroku najvyššiu pravdepodobnosť. Greedy vyhľadávanie môže dosahovať uspokojivé výsledky v úlohách generovania textu (napr. strojový preklad alebo sumarizácia textu), kde je výstup silne závislý od vstupu. [1]

Avšak pri otvorených generovacích úlohách (napr. tvorba príbehov alebo dialógov) má greedy vyhľadávanie tendenciu generovať neprirodzené a opakujúce sa vety. [1]

Ako alternatívna dekodovacia stratégia boli navrhnuté metódy založené na vzorkovaní (sampling), ktoré náhodne vyberajú ďalší token na základe pravdepodobnostného rozdelenia s cieľom zvýšiť náhodnosť a rozmanitosť počas generovania:

$$x_i \sim P(x \mid \mathbf{x}_{<i}) \quad \text{Zdroj: [1]} \quad (2.9)$$

Tieto dve dekodovacie metódy nie sú obmedzené len na architektúru založenú výhradne na dekóderi. Vo všeobecnosti ich možno podobným spôsobom aplikovať aj na modely typu encoder-decoder a prefixové dekóderové modely. [1]

Greedy vyhľadávanie alebo aj hladové vyhľadávanie, ktorý vyberá token s najvyššou pravdepodobnosťou v každom kroku môže viesť k prehliadnutiu vety s vyššou celkovou pravdepodobnosťou, ale nižším odhadom kontextu vety. Preto sa vyvinulo niekoľko stratégií na zmiernenie tohto problému. [37]

Beam vyhľadávanie zachová vety s n najvyššími pravdepodobnosťami v každom kroku počas procesu dekódovania a nakoniec vyberie vygenerovanú odpoveď s najvyššou pravdepodobnosťou. [37]

Length penalty je bežne používanou technikou, ktorá penalizuje za dĺžku (normalizácia dĺžky). Normalizuje pravdepodobnosť vety podľa dĺžky vety (vydelenej exponenciálnou mocninou dĺžky α). [38]

Sampling metóda založené na vzorkovaní vzorkujú token z celej slovnej zásoby, čo môže vybrať nesprávne alebo irelevantné tokeny na základe kontextu. Na zlepšenie kvality generovania bolo navrhnutých niekoľko stratégií na zmiernenie alebo zabránenie výberu slov s mimoriadne nízkou pravdepodobnosťou. [1]

Temperature sampling v praxi často používa úprava tzv. teplotného koeficientu v softmax funkcii, ktorá sa používa na výpočet pravdepodobnosti výberu j -teho tokenu zo slovníka:

$$P(x_j \mid \mathbf{x}_{<i}) = \frac{\exp(l_j/t)}{\sum_{j'} \exp(l_{j'}/t)}, \quad \text{Zdroj: [1]} \quad (2.10)$$

kde $l_{j'}$ je skóre jednotlivých slov a t je teplotný koeficient. Zníženie hodnoty t zvyšuje pravdepodobnosť výberu tokenov s vyšším skóre a zároveň znižuje pravdepodobnosť výberu menej pravdepodobných tokenov.

Ak je $t = 1$, ide o štandardné náhodné vzorkovanie. Keď sa t blíži k nule, vzorkovanie sa správa ako greedy search. Naopak, keď t smeruje k nekonečnu, prechádza do uniformného vzorkovania. [1]

Top-k sampling na rozdiel od vzorkovania teploty priamo odsekne tokeny s nižšou pravdepodobnosťou a iba vzorkuje z tokenov s najvyššími k najvyššími pravdepodobnosťami.

2.6 Fine-Tuning

Predtrénované modely majú určité základné znalosti a sú plne schopné vykonávať širokú škálu úloh, no nemajú žiadnu špecializáciu. Na zvládnutie úrovne odbornosti v konverzačných zručnostiach, generovaní textu alebo vytváraní iného obsahu na požiadanie si LLM vyžaduje niekoľko ďalších etáp učenia. Preto predtrénované LLM ešte nie sú vhodné na použitie vo vysoko špecializovaných oblastiach, pretože nemajú hlboké kontextové znalosti v určitých oblastiach. Doladenie pod dohľadom (supervised fine-tuning) a posilnenie tréningu z ľudskej spätnej väzby sú často potrebné na vytvorenie predtrénovaného modelu vhodného pre takéto špecifické použitie. [39, 13]

Fine-tuning je teda proces učenia pod dohľadom (supervised learning), ktorý si vyžaduje relatívne malý súbor dát. Použitie modelu, ktorý je už predtrénovaný, je presne to, čo umožňuje použiť relatívne malý súbor špecifických dát na dodatočné ladenie. Keďže model už má prvotné znalosti, je ľahké ho doladiť na menšom množstve dát. Využitím minulých skúseností sú jazykové modely schopné efektívne riešiť nové úlohy počas doladovania (fine-tuningu). [39]

Fine-tuning používa váhy vopred trénovaného modelu ako východiskový bod pre ďalšie tréningy na menšom súbore dát príkladov, ktoré priamejšie odrážajú špecifické úlohy a prípady použitia, na ktoré sa model použije, napríklad na inštrukcie alebo zdrojový kód. Zvyčajne zahŕňa učenie pod dohľadom (SFT), ale môže zahŕňať aj učenie sa posilňovaním (reinforcement learning).

2.6.1 Dáta

Súbory dát používané na fine-tuning sprostredkujú špecifické znalosti domény, štýl, úlohy alebo prípady použitia, pre ktoré sa vopred trénovaný model doladuje. Napríklad LLM vopred natrénovaný na všeobecný jazyk môže byť doladený na kódovanie pomocou novej množiny dát obsahujúcej relevantné programovacie požiadavky a zodpovedajúce úryvky kódu pre každý z nich. [33, 13]

2.6.2 Parameter efficient fine-tuning (PEFT)

Fine-tuning je možné použiť na aktualizáciu váh celej siete, ale z praktických dôvodov to tak nie je vždy. Existuje široká škála alternatívnych metód fine-tuning, ktoré sa často označujú pod zastrešujúcim pojmom parametricky efek-

tívne ladenie (PEFT), ktoré aktualizujú iba vybranú podmnožinu parametrov modelu. Metódy PEFT môžu znížiť výpočtové nároky a znížiť zabúdanie, jav, pri ktorom ladenie spôsobuje stratu alebo destabilizáciu základných znalostí modelu, často bez zmysluplných kompromisov vo výkone. PEFT zahŕňa celý rad metód na zníženie počtu trénovateľných parametrov, ktoré je potrebné aktualizovať, aby bolo možné efektívne prispôsobiť veľký predtrénovaný model špecifickým následným aplikáciám. Pritom PEFT výrazne znižuje výpočtové zdroje a pamäťové úložisko potrebné na získanie efektívne vyladeného modelu. [33, 1]

2.6.2.1 Reparametrizácia

Metódy založené na reparametrizácii, ako je Low Rank Adaptation (LoRA), využívajú nízkorozmernú transformáciu vysokorozmerných matíc (ako masívna matica vopred pripravených modelových váh v modeli transformera). Tieto nízkorozmerné reprezentácie vynechávajú nepodstatné vyššie dimenzionálne informácie, aby zachytili základnú nízkorozmernú štruktúru modelových váh, čo výrazne znižuje počet trénovateľných parametrov. To výrazne urýchľuje ladenie a znižuje pamäť potrebnú na ukladanie aktualizácií modelu.

LoRA sa vyhýba priamej optimalizácii matice modelových váh a namiesto toho optimalizuje maticu aktualizácií modelových váh, ktorá je vložená do modelu. Táto matica aktualizácií váh je zas reprezentovaná ako dve menšie matice, čo výrazne znižuje počet parametrov, ktoré sa majú aktualizovať, čo zase dramaticky urýchľuje ladenie a znižuje pamäť potrebnú na ukladanie aktualizácií modelu. Samotné predtrénované modelové váhy zostávajú zmrazené. Ďalšou výhodou LoRA je, že keďže to, čo sa optimalizuje a ukladá, nie sú nové modelové váhy, ale skôr rozdiel medzi pôvodnými vopred natrénovanými a jemne vyladenými váhami, je možné podľa potreby zameniť rôzne LoRA špecifické pre danú úlohu, aby sa predtrénovaný model, ktorého skutočné parametre zostali nezmenené, na daný prípad použitia. Boli vyvinuté rôzne deriváty LoRA, ako napríklad QLoRA, ktorá ďalej znižuje výpočtovú zložitosť kvantizáciou modelu transformera pred LoRA. [33, 1]

2.6.3 Ladenie inštrukcií

Ladenie inštrukcií je podmnožinou SFT, ktoré sa často používa na doladovanie LLM na používanie chatbotov, ktoré pripravuje LLM na generovanie odpovedí, ktoré priamo reagujú na potreby používateľov. Označené príklady podľa formátu (výzva, odpoveď), v ktorých príklady výziev obsahujú úlohy orientované na pokyny, ako napríklad „klasifikujte nasledujúcu vetu ako pozitívnu alebo negatívnu“, demonštrujú, ako reagovať na výzvy predstavujúce rôzne prípady použitia, ako je zodpovedanie otázok, sumarizácia alebo preklad. Pri aktualizácii modelových váh, aby sa minimalizovala strata medzi výstupmi modelu a označenými vzorkami, sa LLM naučí pridávať text k výzvam užitočnejším

spôsobom a lepšie dodržiavať pokyny vo všeobecnosti. Súbor dát použitý pre SFT by mohol obsahovať niekoľko párov (výzva, odpoveď), ktoré demonštrujú, že požadovaným spôsobom, ako reagovať na výzvy je poskytnúť návrhy krok za krokom, a nie iba dokončiť vetu. [33, 1]

2.6.4 Reinforcement learning from human feedback (RLHF)

Zatiaľ čo ladenie inštrukcií môže naučiť model hmatateľnému a priamemu správaniu, ako napríklad štruktúrovať jeho odpovede, môže byť namáhavé a ťažké naučiť abstraktné ľudské vlastnosti, ako je ústretovosť, faktická presnosť, humor alebo empatia prostredníctvom označených príkladov. Na lepšie zosúladenie výstupov modelu s ideálnym ľudským správaním, najmä pre prípady konverzačného použitia, ako sú chatboty, môže byť SFT doplnený o posilňovacie učenie, konkrétnejšie učenie sa posilňovaním z ľudskej spätnej väzby (RLHF). RLHF, tiež nazývaná posilňovacie učenie z ľudských preferencií, pomáha doladovať modely pre kvality, ktoré sú zložité, zle definované alebo ťažko špecifikovateľné prostredníctvom samostatných príkladov. RLHF v podstate poskytuje matematicky crowdsourcingovú alternatívu: požiadať LLM, aby generovala vtipy a nechajte testerov hodnotiť ich kvalitu. Tieto hodnotenia možno použiť na tréning modelu odmeňovania na predpovedanie druhov vtipov, ktoré získajú pozitívnu spätnú väzbu, a tento model odmeňovania možno následne použiť na tréning LLM prostredníctvom posilňovacieho učenia. Praktickejšie sa RLHF zameriava na riešenie existenčných výziev LLM, ako sú halucinácie, odrážajúce spoločenské predsudky obsiahnuté v tréningových údajoch alebo narábanie s hrubými alebo nepriateľskými používateľskými vstupmi. [33, 13]

2.7 Inferencia

LLM inferencia je kritickým aspektom rôznych moderných aplikácií, ktorý odkazuje na použitie predtrénovaného LLM na generovanie odpovedí. Dnes je efektívna inferencia nevyhnutná pre možnosti generovania v rôznych aplikáciách, ako sú chatboty, jazykové preklady a systémy na vyhľadávanie informácií. [6]

Typická úloha generatívnej inferencie LLM pozostáva z dvoch fáz, fáza prefill, ktorá preberá sekvenciu požiadaviek na generovanie vyrovnávacej pamäte kľúč-hodnota (KV cache) pre každú transformačnú vrstvu LLM, a fáza dekódovania, ktorá využíva a aktualizuje KV cache na postupné generovanie tokenov, pričom aktuálne generovanie tokenov závisí od predtým vygenerovaných tokenov. Počas inferencie definujeme niekoľko dôležitých veličín, ako je veľkosť dávky, dĺžky vstupných a výstupných sekvencií, skryté rozmery transformovaných vrstiev a vrstiev neuronovej siete. Každá tranformerová vrstva používa špecifické váhové matice pre kľúč, požiadavky, hodnotu, výstupnú projekciu a

dve transformácie feedforward. [40]

Vo fáze prefill model spracúva celú vstupnú sekvenciu naraz. Pre každú vrstvu vypočíta kľúče a hodnoty použitím príslušných váhových matíc na vstup. Požiadavky sa počítajú podobne. Výstup pozornosti sa potom generuje aplikovaním operácie pozornosti softmax nad požiadavkami a kľúčmi, vynásobením hodnotami, premietnutím pomocou výstupnej váhy a pridaním reziduálneho spojenia. Výstup ďalej prechádza cez doprednú sieť s aktiváciou ReLU, po ktorej nasleduje ďalšie reziduálne spojenie na vytvorenie vstupu pre ďalšiu vrstvu. [40]

Vo fáze dekodovania model spracováva jeden token v čase. Pre každý nový token aktualizuje kľúče a hodnoty uložené v medzipamäti spojením nových projekcií. Potom vypočíta dopyt, vykoná pozornosť pomocou aktualizovanej vyrovnávacej pamäte, premietne výstup a odovzdá ho cez doprednú sieť podobne, ako vo fáze prefill. Výsledok sa stáva vstupom pre ďalšiu vrstvu. [40]

Po dekodovacej fáze sa model dostane do autoregresnej generačnej slučky. V každom kroku použije posledný vygenerovaný token na výpočet nových projekcií kľúčov, hodnôt a dopytov. Tie sa pripájajú k existujúcej vyrovnávacej pamäti, čo umožňuje efektívne opakované použitie minulých výpočtov. Model predpovedá ďalší token na základe aktualizovanej vyrovnávacej pamäte a svojich aktuálnych skrytých stavov. Tento proces sa opakuje iteratívne, pričom sa každý nový token vkladá späť do modelu, až kým sa nedosiahne maximálna dĺžka výstupu alebo sa nevytvorí špeciálny token konca sekvencie. [40] Ďalej na zvolenie najvhodnejšieho tokenu pri predikcii sa používajú rôzne techniky, ktoré sme opísali s časti 2.5.6.

2.8 Kvantizácia

Kvôli obrovskému množstvu parametrov modelu zaberajú LLM značnú pamäťovú stopu pri inferencií, takže ich nasadenie v reálnych aplikáciách je nákladné. V tejto časti sa zameriame na to, ako znížiť pamäťovú stopu LLM pomocou technických prístupov. [1] Predovšetkým opíšeme prístup kvantizácie modelu.

Vo všeobecnosti existujú dva hlavné prístupy kvantifikácie modelu, a to tréning zameraný na kvantovanie (QAT) (quantization-aware training) (vyžadujúci dodatočné úplné pretrenovanie modelu) a kvantovanie po tréningu (PTQ) (post-training quantization) (nevyžaduje žiadne preškolenie modelu). V porovnaní s malými jazykovými modelmi je potrebné pri navrhovaní alebo výbere metód kvantovania pre LLM zvážiť dva hlavné rozdiely. LLM pozostávajú z obrovského množstva parametrov, a preto sú metódy PTQ preferovanejšie kvôli oveľa nižším výpočtovým nákladom, ako metódy QAT. [41, 1]

2.8.1 Quantization-Aware Training

Pri aplikácii kvantizácie na už natrénovaný model môže dôjsť k narušeniu jeho parametrov, čo vedie k odchýleniu od konvergenčného bodu dosiahnutého pri tréňovaní s plávajúcou desatinnou čiarkou. Tento problém možno zmierniť opätovným tréňovaním modelu s kvantizovanými parametrami, aby sa priblížil k novému optimu s čo najnižšou stratou. Jednou z najpoužívanejších techník je *Quantization-Aware Training* (QAT). Pri QAT sa používa štandardný dopredný a spätný prechod v režime plávajúcej desatinnej čiarky, pričom parametre modelu sa po každej aktualizácii gradientu kvantizujú (analogicky k plánovanému zostupu gradientu). Kľúčovým krokom je aplikácia kvantizačnej projekcie až po aktualizácii váh v plnej presnosti, čo znižuje akumuláciu chyby. Spätný prechod sa uskutočňuje v plávajúcej presnosti, pretože kvantizovaná presnosť môže viesť k nulovým alebo výrazne nepresným gradientom, obzvlášť pri nízkych bitových šírkach. Hlavnou výzvou je zaobchádzanie s nediferencovateľnou kvantizačnou operáciou, najmä zaokrúhľovaním, ktoré má takmer všade nulový gradient. Na tento problém sa často používa *Straight Through Estimator* (STE), ktorý aproximuje gradient kvantizačného operátora ako identitu. Napriek svojej jednoduchej povahe STE v praxi funguje prekvapivo dobre, najmä ak sa nepoužíva extrémne nízka presnosť (napr. binárna kvantizácia). Aj keď niektoré novšie prístupy ponúkajú jemnejšie aproximácie, STE ostáva v praxi najrozšírenejším riešením. [41]

2.8.2 Post-Training Quantization

Post-Training Quantization (PTQ) je alternatívou ku QAT, ktorá umožňuje kvantizáciu modelu bez ďalšieho tréňovania. Vďaka tomu je PTQ výpočtovo nenáročné riešenie, vhodné aj v situáciách s obmedzeným alebo neoznačeným množstvom dát. Táto výhoda však často prichádza za cenu nižšej presnosti, najmä pri nízkej bitovej kvantizácii. Preto boli navrhnuté rôzne metódy na zmiernenie presnostnej degradácie. Napríklad úpravy rozptylu a priemeru kvantizovaných váh, alebo metódy zarovnávanie rozsahov medzi vrstvami s cieľom znížiť kvantizačné chyby. Metóda *ACIQ* (Analytical Clipping for Integer Quantization) analyticky určuje optimálny rozsah pre orezávanie hodnôt a bitovú šírku pre jednotlivé kanály. Hoci poskytuje dobré výsledky, jej aplikácia na reálny hardvér je zložitá kvôli potrebe kanálovej kvantizácie. Na riešenie tohto problému bola navrhnutá metóda *OMSE*, ktorá namiesto kanálovej kvantizácie optimalizuje kvantizáciu minimalizáciou L2 vzdialenosti medzi kvantizovaným a pôvodným tenzorom. Ďalej, metóda delenia odľahlých kanálov navrhuje duplikovať a rozčleniť kanály s extrémnymi hodnotami, čím sa znižuje ich negatívny vplyv na PTQ. Zaujímavým prístupom je aj *AdaRound*, ktorý upravuje klasické zaokrúhľovanie na najbližšiu hodnotu a nahrádza ho adaptívnou stratégiou, ktorá lepšie optimalizuje stratu modelu. Rozširujúca metóda *AdaQuant* ide ešte ďalej a povoľuje väčšiu flexibilitu v úprave kvanti-

zovaných váh. PTQ možno aplikovať dokonca aj bez použitia trénovacích alebo testovacích dát ide o tzv. *zero-shot* scenáre, ktoré sú extrémne, no v niektorých prípadoch užitočné. [41]

■ ■

Úvod do GPU

Grafický procesor (GPU) sa stal neoddeliteľnou súčasťou dnešných bežných počítačových systémov. Moderný GPU nie je len výkonný grafický engine, ale aj vysoko paralelný programovateľný procesor so špičkovou aritmetikou a šírkou pásma pamäte, ktorý podstatne prevyšuje jeho procesorový náprotivok. [42]

Rozmanitosť výpočtovo náročných aplikácií bežiacich v moderných cloudových dátových centrách podnietila explóziu cloud computingu s akceleráciou GPU. K takýmto intenzívnym aplikáciám patrí tréning a inferencia hlbokého učenia, analýza dát, vedecké výpočty, genomika, vykresľovanie grafiky, cloudové hry a mnohé ďalšie.

GPU poskytujú potrebnú silu na zrýchlenie mnohých zložitých a nepredvídateľných zariadení, ktoré prebiehajú v dnešných cloudových dátových centrách, od škálovania tréningov a vedeckých výpočtov, cez škálovanie inferenčných aplikácií až po umožnenie konverzačnej umelej inteligencie v reálnom čase.

GPU NVIDIA sú popredné výpočtové enginy, ktoré poháňajú revolúciu umelej inteligencie a poskytujú ohromné zrýchlenie tréningu a inferencie. Okrem toho urýchľujú mnoho typov aplikácií a systémov HPC (high performance computing) a analýzy dát, čo zákazníkovi umožňuje efektívne analyzovať, vizualizovať. Zrýchlené počítačové platformy NVIDIA sú základom mnohých najdôležitejších a najrýchlejších rastúcich odvetví na svete. HPC prerástlo za hranice superpočítačov prevádzkujúcich výpočtovo náročné aplikácie, ako je predpoveď počasia, prieskum ropy a zemného plynu a finančné modelovanie. V súčasnosti milióny GPU zrýchľujú mnoho typov HPC aplikácií bežiacich v cloudových dátových centrách, serveroch, systémoch na okraji a dokonca aj na pracovných staniciach, ktoré obsluhujú stovky priemyselných odvetví a vedeckých oblastí. [43]

3.1 Aplikácia GPU v strojovom učení

Siete umelej inteligencie neustále rastú čo do veľkosti, zložitosti a rozmanitosti a používanie aplikácií a služieb založených na umelej inteligencii sa rýchlo rozširuje. GPU urýchľujú množstvo systémov a aplikácií umelej inteligencie vrátane systémov odporúčaní hlbokého učenia, autonómnych strojov (samozajazdiace autá, továrenské roboty atď.), spracovania prirodzeného jazyka (chatboty, preklad jazykov v reálnom čase atď.), analýzy videa v inteligentných mestách, molekulárne simulácie, analýza obrazu, drony a ďalšie. Všetky tieto aplikácie majú spoločné to, že vyžadujú rýchle a paralelné spracovanie rozsiahlych vstupov pomocou lineárnej algebry (najmä maticových a tenzorových operácií), ktorú GPU zvládajú efektívne vďaka svojmu architektonickému návrhu. [43]

Moderné GPU, ako napríklad tie od spoločnosti NVIDIA, sú vybavené tisíckami paralelných výpočtových jadier, vrátane CUDA jadier na základné aritmetické výpočty a špecializovaných Tensor jadier optimalizovaných pre výpočty maticových operácií, typicky používané pri tréňovaní a inferencii hlbokých neurónových sietí. Tieto jadrá umožňujú vykonávať masívny počet operácií v krátkom čase, čím výrazne skracujú trvanie tréňovania a zvyšujú efektivitu nasadzovania modelov.

Zvlášť dôležitou oblasťou sú LLM. Tieto modely obsahujú miliardy až stovky miliárd parametrov a pri tréňovaní aj inferencii vyžadujú enormné množstvo výpočtových zdrojov. GPU poskytujú základnú infraštruktúru na tréňovanie takýchto modelov v distribuovanom prostredí, kde je výpočtová záťaž rozdelená medzi desiatky až stovky GPU pomocou techník ako rôzne typy paralelizmu. GPU teda nie sú len výpočtovým prostriedkom, ale základným pilierom moderného ekosystému umelej inteligencie. Ich význam v oblasti LLM je značný, bez GPU by nebolo možné tréňovať modely v rozumne dlhom čase ani ich nasadzovať v reálnom čase do produkcie. [42, 44, 45, 46]

3.2 GPU architektúra

Grafický procesor (GPU) je špecializovaný čip navrhnutý na vysokovýkonné paralelné výpočty. Skladá sa z miliárd tranzistorov a jeho architektúra je optimalizovaná na masívne paralelné spracovanie dát.

Základom GPU je jeho čip, ktorý je tvorený viacerými výpočtovými a riadiacimi komponentmi usporiadanými hierarchicky. Dominantnú časť čipu zaberajú výpočtové jadrá, ktoré sú zoskupené do väčších celkov nazývaných grafické výpočtové klastre (Graphics Processing Clusters, GPC). Každý GPC obsahuje niekoľko streaming multiprocesorov (Streaming Multiprocessors, SM), ktoré predstavujú základnú výpočtovú jednotku GPU. Každý SM je schopný súčasne vykonávať tisíce vlákien, ktoré sú organizované do warpov, skupín 32 vlákien, ktoré vykonávajú rovnakú inštrukciu paralelne. Vo vnútri SM sa nachádzajú špecializované výpočtové jadrá, ako CUDA jadrá (na základné

aritmetické a logické operácie v plávajúcej alebo celočíselnej presnosti), ako aj Tensor jadrá, ktoré sú navrhnuté na efektívne vykonávanie tenzorových a maticových operácií potrebných v strojovom učení. Okrem toho moderné GPU integrujú aj RT Cores (Ray Tracing jadrá) a Tensor Cores ako súčasť akcelerácie pre špecifické grafické alebo AI úlohy. [43]

Dôležitou súčasťou GPU architektúry je tiež pamäťový subsystém. GPU obsahuje viacero kontrolérov grafickej pamäte, ktoré spravujú prístup do globálnej pamäte typu HBM2e alebo GDDR6. Táto pamäť má vysokú priepustnosť, čo je kľúčové pre efektívne spracovanie rozsiahlych dátových dávok. Medzi ďalšie pamäťové vrstvy patrí L2 cache (zdieľaná medzi všetkými SM) a L1/shared memory, ktorá je lokálna pre každý SM a umožňuje rýchlu komunikáciu medzi vláknami. [43]

GPU je ďalej vybavený NVLink kontrolérmi, ktoré umožňujú vysoko-rýchlostnú komunikáciu medzi viacerými GPU v rámci jedného systému, čo je nevyhnutné pre tréňovanie veľkých modelov v distribuovanom prostredí. PCIe rozhranie zabezpečuje komunikáciu medzi GPU a CPU. [43]

Na vrchole architektúry sa nachádza GigaThread Engine, čo je komponent zodpovedný za rozdeľovanie a plánovanie úloh naprieč všetkými SM a GPC. Tento engine umožňuje súbežné vykonávanie viacerých jadier na viacerých dátach, čím maximalizuje využitie výpočtových zdrojov GPU. [43]

Celá táto architektúra je navrhnutá tak, aby maximalizovala priepustnosť (throughput), počet operácií za jednotku času, na rozdiel od CPU, ktoré sú optimalizované skôr na nízku latenciu. Vďaka tejto odlišnej filozofii návrhu sú GPU extrémne efektívne pri úlohách, ako paralelné maticové výpočty, ktoré sú základom moderného strojového učenia a veľkých jazykových modelov. [46]

3.3 NVIDIA A100

Grafický procesor NVIDIA A100 vychádza z možností predchádzajúceho grafického procesora NVIDIA Tesla V100, pridáva mnoho nových funkcií a zároveň poskytuje výrazne vyšší výkon pre pracovné zaťaženia v oblasti HPC, umelej inteligencie a dátovej analýzy. Model A100, poháňaný GPU GA100 na báze architektúry NVIDIA Ampere, poskytuje veľmi silné škálovanie pre GPU výpočty a aplikácie hlbokého učenia prevádzkované v serveroch, klastroch, cloudových dátových centrách, systémoch na okraji a superpočítačoch. GPU A100 umožňuje budovať pružné, všestranné a vysoko priepustné dátové centrá. A100 pridáva výkonné nové jadro Tensor Core tretej generácie, ktoré zvyšuje priepustnosť oproti V100 a zároveň pridáva komplexnú podporu pre typy dát hlbokého učenia a HPC spolu s novou funkciou Sparsity, ktorá prináša ďalšie zdvojnásobenie priepustnosti. Nové operácie TensorFloat-32 (TF32) Tensor Core v A100 poskytujú jednoduchú cestu k akcelerácii vstupných/výstupných dát FP32 v rámcach hlbokého učenia a HPC, pričom bežia 10x rýchlejšie ako operácie FP32 vo V100 alebo 20x rýchlejšie so sparsity. [43]

GPU NVIDIA GA100 sa skladá z viacerých GPU Processing Clusters

(GPC), Texture Processing Clusters (TPC), Streaming Multiprocessors (SM) a pamäťových radičov HBM2. Úplná implementácia GPU GA100 zahŕňa nasledujúce jednotky:

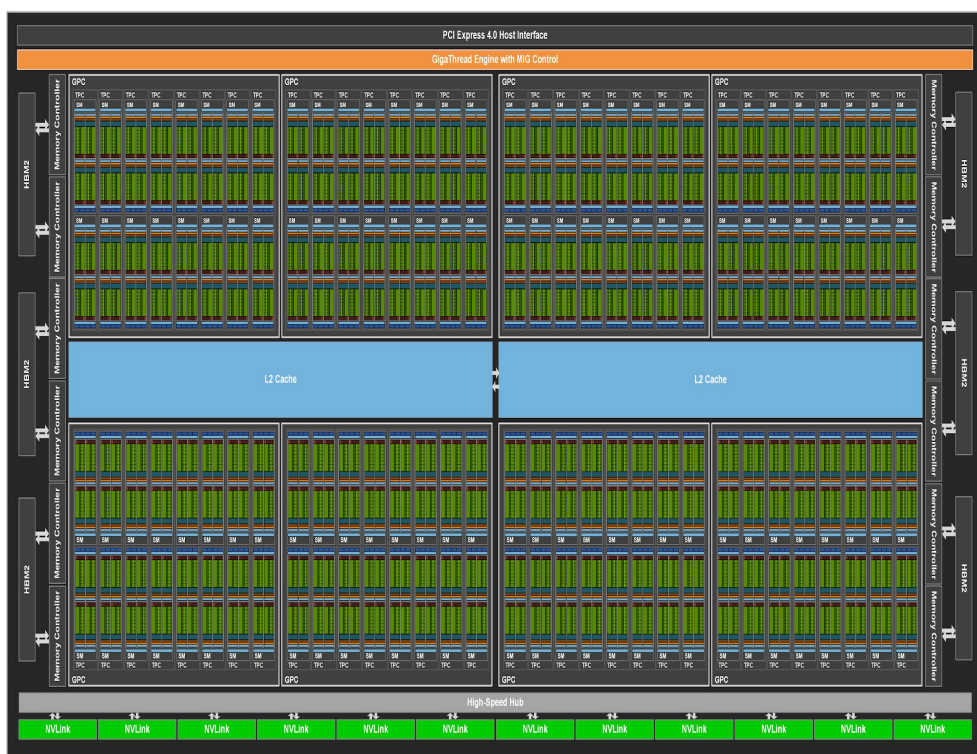
- 8 GPC, 8 TPC/GPC, 2 SM/TPC, 16 SM/GPC, 128 SM na celý GPU
- 64 jadier FP32 CUDA/SM, 8192 jadier FP32 CUDA na celý GPU
- 4 jadrá Tensor Cores/SM tretej generácie, 512 jadier Tensor Cores tretej generácie na celý GPU
- 6 zásobníkov HBM2, 12 512-bitových pamäťových radičov

Implementácia NVIDIA A100 Tensor Core GPU GA100 zahŕňa nasledujúce jednotky:

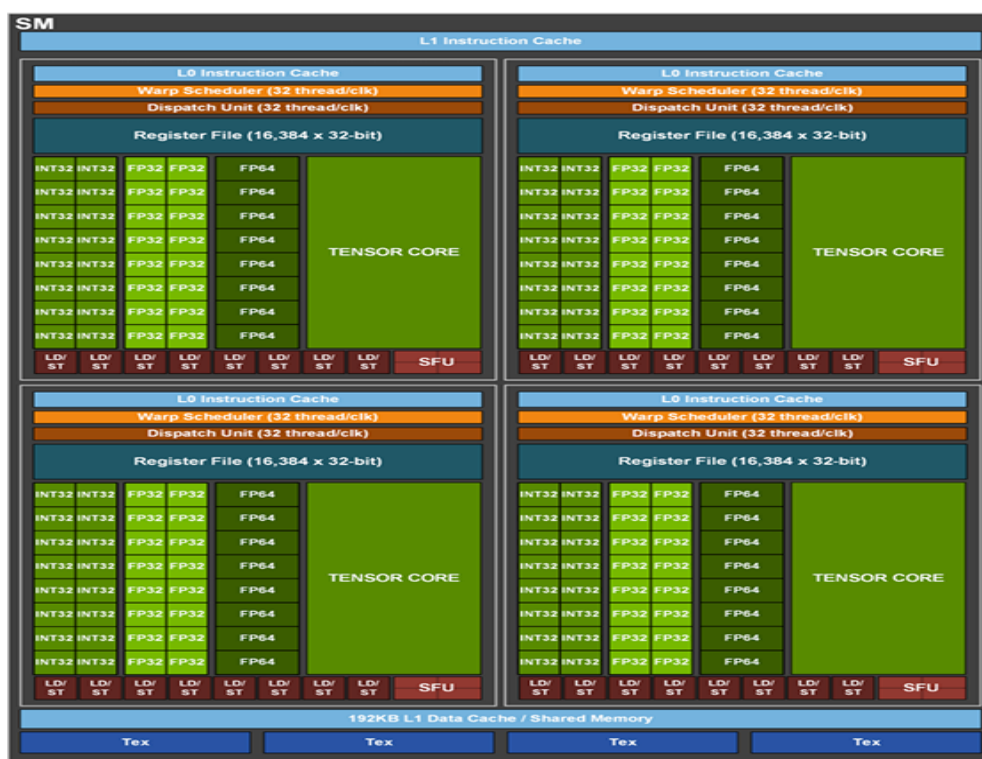
- 7 GPC, 7 alebo 8 TPC/GPC, 2 SM/TPC, až 16 SM/GPC, 108 SM
- 64 FP32 CUDA jadier/SM, 6912 FP32 CUDA jadier na GPU
- 4 jadrá Tensor Cores/SM tretej generácie, 432 jadier Tensor Cores tretej generácie na GPU
- 5 zásobníkov HBM2, 10 512-bitových pamäťových radičov

Streaming Multiprocessor: Každé Tensor Core vykonáva 64 FP16/FP32 zmiešaných presných fúzovaných multiply-add (FMA) operácií za jednotku času. A100 SM obsahuje novú tretiu generáciu Tensor Cores, z ktorých každé vykonáva 256 FP16/FP32 FMA operácií za jednotku času. A100 má štyri jadrá Tensor na SM, ktoré spolu poskytujú 1024 hustých operácií FP16/FP32 FMA za jednotku času. [43]

Tensor Core: Tensor Cores sú špecializované vysokovýkonné výpočtové jadrá pre maticové matematické operácie, ktoré poskytujú prelomový výkon pre aplikácie AI a HPC. Tensor Cores vykonávajú výpočty násobenia a akumulácie matice (matrix multiply and accumulate, MMA). Stovky jadier Tensor pracujúcich paralelne v jednom GPU NVIDIA umožňujú masívne zvýšenie priepustnosti a efektivity. [43]



■ **Obr. 3.1** Prehľad architektúry a komponent v GPU. Získané z [43]



■ Obr. 3.2 Prehľad komponent vnútri SM. Získané z [43]

Porovnanie open-source LLM

Ako sme už v predchádzajúcich častiach naznačili, vývoj veľkých jazykových modelov (LLM) je dynamickou oblasťou, v ktorej sa neustále objavujú nové prístupy, architektúry a trénovacie stratégie. Rozdiely v návrhu modelov, vo výbere a rozsahu trénovacích dát, ako aj v použitých optimalizačných technikách, môžu výrazne ovplyvniť nielen výkonnosť, ale aj použiteľnosť a efektívnosť daného modelu. Vzhľadom na túto rozmanitosť vyvstáva prirodzená otázka, aký konkrétny vplyv majú jednotlivé rozhodnutia o dizajne modelu na jeho schopnosti. Porovnaním niekoľkých popredných open-source LLM modelov sa pokúsime identifikovať kľúčové faktory, ktoré stoja za ich úspechom, ako aj ich prípadné obmedzenia. V tejto kapitole sa preto zameriame na detailnú analýzu štyroch významných a aktuálne veľmi výkonných open-source LLM: Qwen [47], LLaMA [32], Mistral [48] a DeepSeek [49]. Tieto modely reprezentujú rôzne prístupy k návrhu architektúry, využívajú odlišné trénovacie korpory a často zavádzajú vlastné inovatívne optimalizačné techniky. Našou ambíciou je porovnať tieto modely v niekoľkých kľúčových aspektoch:

- **Architektúra:** ako je navrhnutý samotný model, aké špecifické modifikácie alebo vrstvy používa.
- **Trénovacie dáta:** aké množstvo, typ a zdroje dát boli využité pri predtrénovaní.
- **Optimalizačné techniky:** aké algoritmy a metódy boli použité na zrýchlenie a stabilizáciu tréningu.

Cieľom tejto kapitoly je nielen porovnať konkrétne modely, ale aj načrtnúť širší kontext a trendy vo vývoji open-source LLM, ktoré môžu slúžiť ako inšpirácia alebo východisko pre ďalší výskum a vývoj v tejto oblasti.

4.1 Špecifické modely

4.1.1 Qwen modely

QWEN je komplexná séria jazykových modelov, ktorá zahŕňa odlišné modely s rôznym počtom parametrov. Modelová séria zahŕňa základné predtrénované jazykové modely, chatové modely doladené technikami ľudského zarovnania, t. j. supervizované doladenie (SFT), posilňovacie učenie s ľudskou spätnou väzbou (RLHF) atď., ako aj špecializované modely v kódovaní a matematike. QWEN, sú predtrénované na masívnych súboroch dát obsahujúcich miliardy tokenov. Používané sú techniky SFT (supervised fine-tuning) a RLHF (reinforcement learning from human feedback) na zosúladienie QWEN s ľudskými preferenciami, a tak existujú QWEN-CHAT a konkrétne jeho vylepšená verzia QWEN-CHAT-RLHF, ktoré sú doladené na lepšie výsledky v chatovacích úlohách. Okrem toho boli tiež vyvinuté špecializované modely na kódovanie a matematiku, ako sú CODE-QWEN, CODE-QWEN-CHAT a MATH-QWEN-CHAT založené na QWEN s podobnými fine-tuningovými technikami. Qwen je dostupný vo viacerých veľkostiach, ktoré sa líšia podľa počtu parametrov. Tieto varianty poskytujú flexibilitu pre rôzne použitia, od ľahších aplikácií na okrajových zariadeniach, až po náročné výpočtové úlohy na výkonných serveroch. Medzi najznámejšie varianty patria:

- **Qwen-1.8B** – menší model vhodný na rýchle inferencie a úlohy s obmedzenými zdrojmi.
- **Qwen-7B** – stredne veľký model určený pre general-purpose jazykové úlohy s dobrým kompromisom medzi výkonom a nárokmí.
- **Qwen-14B** – väčší model s vyššou kapacitou a presnosťou, vhodný pre komplexnejšie jazykové porozumenie.
- **Qwen-72B** – najväčší open-source variant určený na náročné generatívne úlohy, dosahujúci špičkové výsledky v benchmarkoch.

Každý z týchto modelov je dostupný v základnej (pretraining only) forme, ako aj vo verzii **Qwen-Chat**, o ktorej si povieme nižšie. [47]

Neskôr v priebehu 2 rokov vyvinuli novšie modely ako Qwen2, Qwen2.5 a najnovšie Qwen3, ktoré inovovali pôvodné modely k lepším výsledkom. Zmeny sa týkali, či už pri počte parametrov daných modelov, ale čo je podstatnejšie pri jednotlivých vylepšeniach architektúry alebo tréningových dát. Stručne rozoberieme hlavné zmeny a inovácie.

Qwen2 došiel s modelmi s rovnakými parametrami no s pár zmenami oproti Qwen. Prvou zmenou bola pozornosť, kde z multi-head attention (MHA) prechádza nová séria modelov na Grouped Query Attention (GQA). GQA optimalizuje využitie vyrovnávacej pamäte KV počas inferencie, čím výrazne zvyšuje priepustnosť. Ďalej využívajú takzvanú Dual Chunk Attention (DCA) s YARN na rozšírenie kontextového okna. Qwen2 využíva Dual Chunk Attention, ktorý segmentuje dlhé sekvencie na kúsky zvládnuteľnej dĺžky. Ak sa

vstup dá spracovať v kuse, DCA produkuje rovnaký výsledok ako pôvodná pozornosť. V opačnom prípade DCA uľahčuje efektívne zachytenie relatívnej pozičnej informácie medzi tokenmi v rámci a medzi jednotlivými kusmi, čím zlepšuje výkonnosť dlhých kontextov. Okrem toho využíva aj YARN na zmenu váhy pozornosti na lepšiu extrapoláciu dĺžky. [50] V porovnaní s predošlou verziou zhromaždili podstatne väčší objem vysokokvalitného kódu, matematických a viacjazyčných dát, čím rozšírili možnosti modelu v príslušných oblastiach. Tento nový súbor údajov podporuje približne 30 jazykov, napríklad angličtinu, čínštinu, španielčinu, francúzštinu, nemčinu, arabčinu, ruštinu, kórejštinu, japončinu, thajčinu a vietnamčinu. Predtréninové dáta preto boli rozšírené z 3 biliónov tokenov na 7 biliónov tokenov.

Qwen2.5 v porovnaní s Qwen2 prináša okrem modelov 0.5B, 1.5B, 7B a 72B modely 3B, 14B a 32B, ktoré sú nákladovo efektívnejšie pre scenáre s obmedzenými zdrojmi a sú nedostatočne zastúpené v súčasnej oblasti modelov otvorených zdrojov. Predtréninové dáta sa zvýšili zo 7 biliónov tokenov na 18 biliónov tokenov so zameraním na znalosti, kódovanie a matematiku. Sú stupňovité, aby umožňovali prechod medzi rôznymi zmesami. Bolo odstránených niekoľko kľúčových obmedzení Qwen2 pri používaní vrátane väčšej dĺžky generovania (z 2K tokenov na 8K tokenov), lepšej podpory štruktúrovaných vstupov a výstupov, (napr. tabuliek a JSON) a jednoduchšieho používania nástrojov. [51]

Qwen modely pršli s podobnými modelmi, ako je to pri Qwen2.5. Opäť výrazná zmena nastala v predtréninových dátach, kde súbor dát pre Qwen3 bol v porovnaní s Qwen2.5 výrazne rozšírený. Zatiaľ čo Qwen2.5 bol predtrénovaný na 18 biliónoch tokenov, Qwen3 využíva takmer dvojnásobok tohto množstva s približne 36 biliónmi tokenov pokrývajúcich 119 jazykov a dialektov. Na vytvorenie tejto veľkej sady údajov zozbierali údaje nielen z webu, ale aj z dokumentov podobných PDF. Na extrakciu textu z týchto dokumentov použili Qwen2.5 a na zlepšenie kvality extrahovaného obsahu Qwen2.5. Na zvýšenie množstva matematických a kódových údajov použili Qwen2.5-Math a Qwen2.5-Coder na generovanie syntetických údajov. Tie zahŕňajú dvojice otázok a odpovedí a úryvky kódu. Zaujímavá novinka pri nových modeloch je, že zavádzajú hybridný prístup k riešeniu problémov. Podporujú dva režimy. Režim myslenia. V tomto režime si model pred poskytnutím konečnej odpovede vyžaduje čas na postupné uvažovanie. Tento režim je ideálny pre zložité problémy, ktoré si vyžadujú hlbšie premýšľanie. Režim bez premýšľania. V tomto prípade model poskytuje rýchle, takmer okamžité odpovede, vhodné pre jednoduchšie otázky, pri ktorých je dôležitejšia rýchlosť ako hĺbka. [52]

Ďalším rozdelením sú rôzne doladované modely na základe ich použitia.

- **QWEN-CHAT:** boli vyladené na súbore dát relevantných pre vykonávanie úloh, chat, používanie nástrojov, agenta, bezpečnosť atď. Vycvičili modely odmeňovania tak, aby napodobňovali ľudské preferencie a použili ich v RLHF pre modely chatu, ktoré môžu produkovať reakcie preferované ľuďmi. Modely QWEN-CHAT trénované pomocou RLHF (Reinforcement

Learning from Human Feedback) sú vysoko konkurencieschopné.

- **CODE-QWEN:** CODE-QWEN bol predtrénovaný na rozsiahlych súboroch dát kódu a ďalej doladený tak, aby zvládal konverzácie súvisiace s generovaním kódu, ladením a interpretáciou. Výsledky experimentov uskutočnených na porovnávacích súboroch dát, ako sú HumanEval, MBPP a HumanEvalPack, demonštrujú vysokú úroveň odbornosti CODE-QWEN v porozumení a generovaní kódu.
- **MATH-QWEN-CHAT:** MATH-QWEN-CHAT, ako z názvu môže plynúť, je špeciálne navrhnutý na riešenie matematických problémov. Prekonávajú modely s otvoreným zdrojom v rovnakých veľkostiach s veľkými rezervami a približujú sa GPT-3.5 na matematických referenčných súboroch, ako je GSM8K.
- **QWEN-VL:** QWEN-VL majú všestrannú schopnosť porozumieť vizuálnym a jazykovým pokynom. Podporujú rozpoznávanie textu a vizuálne uzemnenie v čínskom aj anglickom jazyku. Okrem toho tieto modely umožňujú konverzácie a rozprávanie príbehov s viacerými obrázkami.
- **Dáta:** Ukázalo sa, že početnosť dát je kľúčovým faktorom pri vývoji robustného LLM. Na vytvorenie efektívneho súboru dát predtrénovania je nevyhnutné zabezpečiť, aby boli dáta rôznorodé a pokrývali širokú škálu typov, domén a úloh. Súbor dát je navrhnutý tak, aby spĺňal tieto požiadavky a zahŕňa verejné webové dokumenty, encyklopédiu, knihy, kódy atď. Okrem toho je súbor dát viacjazyčný, pričom významná časť dát je v angličtine a čínštine. Aby zabezpečili kvalitu predtrénových dát, vyvinuli komplexný postup predspracovania dát. Pre verejné webové dáta extrahujú text z HTML a na určenie jazyka používajú nástroje na identifikáciu jazyka. Na zvýšenie rozmanitosti dát využívajú techniky deduplikácie vrátane deduplikácie presnej zhody po normalizácii a fuzzy deduplikácie pomocou algoritmov MinHash a LSH (Locality-sensitive hashing). Na odfiltrovanie dát nízkej kvality používajú kombináciu metód založených na pravidlách a metód strojového učenia. Konkrétne na hodnotenie obsahu používajú viacero modelov vrátane jazykových modelov, modelov hodnotenia kvality textu a modelov na identifikáciu potenciálne urážlivého alebo nevhodného obsahu. Tiež ručne vzorkujú texty z rôznych zdrojov a kontrolujú ich, aby zaistili ich kvalitu. Aby ešte viac zvýšili kvalitu dát, selektívne preberajú vzorky údajov z určitých zdrojov, aby zaistili, že modely budú trénované na rôznorodom rozsahu vysokokvalitného obsahu. Na zlepšenie výkonu modelu, začlenili do predtrénového procesu vysokokvalitné inštruktážne dáta. Aby zabezpečili integritu referenčného hodnotenia, starostlivo odstránili všetky vzorky pokynov, ktoré vykazujú prekrytie s akýmkoľvek údajmi prítomnými v testovacích súboroch použitých pri hodnotení. Vzhľadom na veľký počet následných úloh nie je možné opakovať

tento proces filtrovania pre všetky úlohy. Namiesto toho sa uistili, že dáta pokynov pre nahlásené úlohy prešli procesom filtrovania, aby zabezpečili ich presnosť a spoľahlivosť. Nakoniec vytvorili súbor dát s až 3 miliárdami tokenov. [47]

- **Tokenizácia:** Dizajn slovnej zásoby výrazne ovplyvňuje efektivitu tréningu a plnenie následných úloh, preto používa kódovanie bajtových párov (BPE) ako metódu tokenizácie. Začiatkom je open source rýchly tokenizér BPE, tiktokenom a ako východiskový bod je volený základ slovnej zásoby cl100k. Na zlepšenie výkonu modelu pri viacjazyčných nadväzujúcich úlohách, najmä v čínštine, rozširujú slovnú zásobu o bežne používané čínske znaky a slová, ako aj o tie v iných jazykoch. Konečná veľkosť slovnej zásoby je približne 152 kB. QWEN oproti niekoľkým ďalším tokenizérom dosahuje vyššiu účinnosť kompresie ako jeho konkurenti vo väčšine jazykov. To znamená, že náklady na obsluhu sa môžu výrazne znížiť, pretože menší počet tokenov dokáže poskytnúť viac informácií ako jej konkurenti. Okrem toho zabezpečili, že škálovanie veľkosti slovnej zásoby QWEN negatívne neovplyvní následnú výkonnosť predtrénovaného modelu. Teda napriek nárastu veľkosti slovnej zásoby si QWEN zachováva svoju úroveň výkonnosti pri následnom hodnotení. [47]
- **Trénovanie:** QWEN sa riadi štandardným prístupom autoregresívneho jazykového modelovania. To zahŕňa trénovanie modelu na predpovedanie ďalšieho tokenu na základe kontextu poskytnutého predchádzajúcimi tokenmi. Modely rénujú s dĺžkami kontextu 2048. Aby vytvorili dávky dát, dokumenty premieša a zlúči a potom ich skráti na zadané dĺžky kontextu. Na zlepšenie výpočternej efektivity a zníženie využitej pamäte, využíva Flash Attention v moduloch pozornosti. Na optimalizáciu predtrénovania používa štandardný optimalizátor AdamW [35] s nastavenými hyperparametrami $\beta_1 = 0.9$, $\beta_2 = 0.95$ a $\epsilon = 10^{-8}$. Používa kosínusový plán rýchlosti učenia so špecifikovanou špičkovou rýchlosťou učenia pre každú veľkosť modelu. Rýchlosť učenia sa zníži na minimálnu rýchlosť učenia, ktorá je 10% maximálnej rýchlosti učenia. Všetky modely sú cvičené so zmiešanou presnosťou BFloat16 pre stabilitu tréningu.
- **Architektúra:** QWEN je navrhnutý s použitím upravenej verzie architektúry transformer. Konkrétne prijali nedávny open-source prístup trénovania veľkých jazykových modelov, LLaMA, ktorý je všeobecne považovaný za top open-source LLM. Úpravy architektúry zahŕňajú vkladáciu vrstvy, pozičné kódovanie, bias, Pre-Norm, RMSNorm a aktivačnú funkciu. [47]
Vkladacia vrstva: Použitý prístup neviazaného vkladania namiesto viazania váh vstupného vkladania a výstupnej projekcie. Toto rozhodnutie bolo prijaté s cieľom dosiahnuť lepší výkon s cenou nákladov na pamäť. [47]
Pozičné kódovanie: Používa RoP 2.5.4.4 (Rotary Positional Embedding) ako preferovanú možnosť na začlenenie polohových informácií do modelu.

RoPE bol široko prijatý a preukázal úspech v súčasných veľkých jazykových modeloch, najmä PaLM a LLaMA. Predovšetkým je použitá presnosť FP32 pre inverznú frekvenčnú maticu, a nie BF16 alebo FP16, aby uprednostnili výkon modelu a dosiahli vyššiu presnosť. [47]

Bias: Pre väčšinu vrstiev odstraňuje odchýlky, ale pridáva odchýlky vo vrstve pozornosti, aby zvýšili extrapoláčnú schopnosť modelu. [47]

Pre-Norm a RMSNorm: V modeloch modern Transformer je prednormalizácia najpoužívanejším prístupom, ktorý preukázateľne zlepšuje stabilitu tréningu v porovnaní s postnormalizáciou. Nahradili tradičnú techniku normalizácie vrstiev [24] za RMSNorm. Zmena viedla k rovnakému výkonu a zároveň k zlepšeniu účinnosti. [47]

Aktivačná funkcia: Ako aktivačnú funkciu vybrali SwiGLU. Aktivačné funkcie založené na GLU vo všeobecnosti prevyšujú iné základné možnosti, ako je GeLU. Ako je bežnou praxou v predchádzajúcom výskume, zmenšili rozmer doprednej siete (FFN) zo 4-násobku skrytej veľkosti na 8/3 skrytej veľkosti. [47]

4.1.2 Mistral modely

Prvým veľkým jazykovým modelom od mistral.ai je model Mistral 7B, ktorý bol publikovaný v roku 2023. [48] Od neho sa následne vyvíjali aj robustnejšie ale aj modely s menším počtom parametrov, základ modelu ale zostal rovnaký. Odlišnosti v modeloch nezostali iba pri počte parametrov, ale aj finetuningu k zlepšeniu výkonnosti v odlišných sférach. Vzniklo tak Mistral 7B instruct, ktorý sa zameriaval na plnenie inštrukcií alebo neskôr Codestral, ktorý sa hlavne zameriava na generovanie a vyhodnocovanie kódu. Mistral 7B, ukazuje, že starostlivo navrhnutý jazykový model môže poskytnúť vysoký výkon pri zachovaní efektívnej inferencie. Mistral 7B prekonáva predchádzajúci najlepší 13B model LLaMA 2 vo všetkých testovaných benchmarkoch a prekonáva najlepší 34B model LLaMA 34B v matematike a generovaní kódu. Okrem toho sa Mistral 7B približuje výkonu kódovania Code-LLaMA 7B bez toho, aby obetoval výkon na benchmarkoch nesúvisiacich s kódom. [48, 53]

4.1.2.1 Mistral 7B

Mistral 7B využíva skupinovú pozornosť (GQA) 2.5.4.5 a pozornosť pri posuvnom okne (SWA). GQA výrazne zrýchľuje rýchlosť inferencie a tiež znižuje požiadavky na pamäť počas dekódovania, čo umožňuje väčšie veľkosti dávok a tým vyššiu priepustnosť, čo je rozhodujúci faktor pre aplikácie v reálnom čase. Okrem toho je SWA navrhnutý tak, aby efektívnejšie zvládol dlhšie sekvenencie pri znížených výpočtových nákladoch, čím sa zmierni bežné obmedzenia v LLM. [48]

- **Architektúra:** Mistral 7B je založený na architektúre transformera. V porovnaní s Llamou prináša niekoľko zmien, ktoré zhrnieme nižšie. **Posúvne okno (SWA):** Posuvné okno využíva naskladané vrstvy transformátora na sledovanie informácií nad veľkosťou okna W . Skrytý stav v polohe i vrstvy k , h_i sa stará o všetky skryté stavy z predchádzajúcej vrstvy s polohami medzi i a W . Rekurzívne môže h_i pristupovať k tokenom zo vstupnej vrstvy na vzdialenosť až $k * W$ tokenov. V praxi pri dĺžke sekvencie 16 kB a $W = 4096$ zmeny vykonané v FlashAttention prinesú 2-násobné zlepšenie rýchlosti oproti základnej línii pozornosti. [48]

Rolling Buffer Cache: Pevné rozpätie pozornosti znamená, že môžeme obmedziť veľkosť vyrovnávacej pamäte pomocou rolovacej vyrovnávacej pamäte. Táto vyrovnávacia pamäť má pevnú veľkosť W a kľúče a hodnoty pre časový krok i sú uložené na pozícii $i \bmod W$ vyrovnávacej pamäte. V dôsledku toho, keď je pozícia i väčšia ako W , minulé hodnoty v medzipamäti sa prepíšu a veľkosť medzipamäte sa prestane zväčšovať. Pri sekvencii s dĺžkou 32 tisíc tokenov sa tým zníži využitie pamäte vyrovnávacej pamäte 8x bez vplyvu na kvalitu modelu. [48]

Predvypĺňanie a rozdeľovanie na časti: Pri generovaní sekvencie musíme predpovedať tokeny jeden po druhom, pretože každý token je podmienený predchádzajúcimi. Výzvu však poznáme vopred a môžeme vopred naplniť vyrovnávaciu pamäť (k, v) výzvou. Ak je výzva veľmi veľká, môžeme ju rozdeliť na menšie časti a vopred naplniť vyrovnávaciu pamäť každou časťou. Na tento účel môžeme zvoliť veľkosť okna ako veľkosť našej časti. Pre každý kus teda musíme vypočítať pozornosť nad vyrovnávacou pamäťou a nad kusom. [48]

4.1.2.2 Mixtral 8x7B

Mixtral 8x7B, model riedkej zmesi expertov s otvorenými váhami. Mixtral dosahuje lepšie výsledky ako LLaMA 2 70B a GPT-3.5 na väčšine benchmarkov. Keďže pre každý token používa len podmnožinu svojich parametrov, Mixtral umožňuje vyššiu rýchlosť inferencie pri malých veľkostiach dávok a vyššiu priepustnosť pri veľkých veľkostiach dávok. Je to model založený len na dekodéri, v ktorom blok feedforward vyberá z množiny 8 rôznych skupín parametrov. V každej vrstve pre každý token vyberie sieť smerovača dve z týchto skupín expertov, ktoré spracujú token a aditívne skombinujú ich výstup. Táto technika zvyšuje počet parametrov modelu a zároveň kontroluje náklady a latenciu, keďže model používa len časť z celkovej sady parametrov na jeden token. Model Mixtral je predtrénovaný s viacjazyčnými údajmi s použitím kontextu o veľkosti 32 tisíc tokenov. V rámci niekoľkých referenčných kritérií sa vyrovná alebo prekoná výkonnosť modelov LLaMA 2 70B a GPT-3.5. Mixtral dokáže úspešne načítať informácie zo svojho kontextového okna s 32 tisíci tokenmi bez ohľadu na dĺžku sekvencie a umiestnenie informácie v sekvencii. [53]

- **Architektúra:** Mixtral je založený na tranformerovej architektúre a používa rovnaké modifikácie, ako sú opísané v Mistral 7B, s významnými výnimkami, že Mixtral podporuje plne hustý kontext s dĺžkou 32 tisíc tokenov a napájacie bloky sú nahradené vrstvami Mixture-of-Expert, ktorý sme v kapitole 2.4.5 vysvetlili. V transformačnom modeli sa vrstva MoE aplikuje nezávisle na token a nahrádza podbloky doprednej neuronovej siete transformačného bloku. V prípade modelu Mixtral používa rovnakú architektúru SwiGLU a nastavujeme $K = 2$. To znamená, že každý token je smerovaný do dvoch SwiGLU subblokov s rôznymi sadami váh. [53]

4.1.3 LLaMA Modely

Rodina modelov LLaMA (Large Language Model Meta AI) predstavuje sériu veľkých jazykových modelov vyvinutých spoločnosťou Meta AI s cieľom poskytnúť efektívne a výkonné open-source alternatívy k uzavretým modelom. LLaMA modely sú známe svojou efektivitou vo výpočte aj pri menšom počte parametrov, čím sú prístupnejšie pre výskumnú komunitu a vývojárov. LLaMA modely sa zameriavajú na dosiahnutie najlepších možných výkonov pri rôznych rozpočtoch na inferenciu, a to trénovaním na väčšom počte tokenov, ako sa zvyčajne používa. Výsledné modely majú rozsah parametrov od 1B do 405B s konkurencieschopným výkonom v porovnaní s najlepšími existujúcimi LLM. Využíva len verejne dostupné dáta, vďaka čomu je kompatibilná s otvoreným zdrojovaním, zatiaľ čo väčšina existujúcich modelov sa spolieha na údaje, ktoré buď nie sú verejne dostupné, alebo nie sú zdokumentované. Veľké transformátory trénuje na veľkom množstve textových dát pomocou štandardného optimalizátora. [32, 54, 55]

4.1.3.1 LLaMA 1

Prvá verzia modelu LLaMA [32] bola vydaná v roku 2023 a obsahovala modely rôznych veľkostí: 7B, 13B, 33B a 65B parametrov. Modely boli trénované na rozsiahlych dátach (1,4T tokenov) z otvorených zdrojov, ako sú CommonCrawl, Wikipedia, Project Gutenberg či arXiv. Hlavnou výhodou LLaMA bolo, že aj menšie modely (napr. 13B) dosahovali výkon porovnateľný s oveľa väčšími modelmi ako GPT-3 (175B). [32]

- **Architektúra:** Založená na architektúre transformera. Využíva rôzne vylepšenia, ktoré boli následne navrhnuté a použité v rôznych modeloch.

Prednormalizácia: Na zlepšenie stability tréningu normalizuje vstup každej tranformerovej podvrstvy namiesto normalizácie výstupu. Používa normalizačnú funkciu RMSNorm 2.5.4.1.

Aktivačná funkcia: Nelinearitu ReLU nahrádza aktivačnou funkciou SwiGLU s cieľom zlepšiť výkonnosť.

Rotary position embedding: Odstránili absolútne pozičné vloženia a namiesto toho pridali rotačné pozičné vloženia (RoPE 2.5.4.4) v každej vrstve siete.

- **Optimalizátor:** modely sú natrénované pomocou optimalizátora AdamW [35] s nasledujúcimi hyperparametrami $\beta_1 = 0.9$, $\beta_2 = 0.95$. Používa kosínusový plán miery učenia tak, že konečná miera učenia sa rovná 10% maximálnej miery učenia. Používa rozpad váh 0.1 a orezávanie gradientu 1.0. Používa 2000 zahrievacích krokov a rýchlosť učenia a veľkosť dávky mení s veľkosťou modelu. Urobili niekoľko optimalizácií na zlepšenie rýchlosti tréningu modelov. Používa efektívnu implementáciu kauzálnej viachlavej pozornosti, aby znížili spotrebu pamäte a čas behu. Dosahuje sa to tým, že sa neukladajú váhy pozornosti a nevypočítavajú sa skóre kľúčov/požiadaviek, ktoré sú maskované vzhľadom na kauzálnu povahu úlohy modelovania jazyka. Na ďalšie zlepšenie efektivity tréningu znížili množstvo aktivácií, ktoré sa prepočítavajú počas spätného prechodu s kontrolným bodom. Presnejšie povedané, ušetrili aktivácie, ktoré sú nákladné na výpočet, ako napríklad výstupy lineárnych vrstiev. Dosiahli to ručnou implementáciou spätnej funkcie pre transformačné vrstvy namiesto toho, aby sa spoliehali na autogram PyTorch. Aby túto optimalizáciu plne využili, znížili spotrebu pamäte modelu pomocou modelového a sekvenčného paralelizmu. Pri tréningu 65B-parametrového modelu kód spracúva približne 380 tokenov/sec/GPU na 2048 GPU A100 s 80G, to znamená, že tréning nad dátovou sadou obsahujúcou 1.4 T tokenov trvá približne 21 dní. [32]
- **Dáta:** Súbor tréningových dát je zmesou niekoľkých zdrojov, ktoré pokrývajú rôznorodé oblasti. Z väčšej časti opätovne používa zdroje dát, ktoré boli využité na tréning iných LLM, s obmedzením, že používa len dáta, ktoré sú verejne dostupné a kompatibilné s otvorenými zdrojmi. To vedie k nasledujúcej zmesi dát a percentuálnemu podielu, ktorý predstavujú v tréningovej množine. Dokopy má súbor 4.749TB. [32]

Anglický CommonCrawl [67%]: Predbežne spracúvame päťicu výpisov CommonCrawl z rokov 2017 až 2020. Proces deduplikuje dáta na úrovni riadkov, vykonáva identifikáciu jazyka pomocou lineárneho klasifikátora na odstránenie neanglických stránok a filtruje nekvalitný obsah pomocou jazykového modelu ngramov. Okrem toho natrénovali lineárny model na klasifikáciu stránok použitých ako odkazy vo Wikipédii oproti náhodne vybraných stránok a vyradili stránky, ktoré neboli klasifikované ako odkazy. [32]

C4 [15%]: Použitie rôznych vopred spracovaných súborov CommonCrawldataset zlepšuje výkon. Preto zahrnuli verejne dostupný súbor dát C4. Predspracovanie C4 obsahuje aj kroky deduplikácie a identifikácie jazyka. [32]

Github [4.5%]: Používa sa verejný súbor dát GitHub, ktorý je k dis-

pozícii na Google BigQuery. Ponechali len projekty, ktoré sú šírené pod licenciami Apache, BSD a MIT. Okrem toho odfiltrovali nekvalitné súbory pomocou heuristiky založenej na dĺžke riadku alebo podiele alfanumerických znakov a pomocou regulárnych výrazov odstránili napríklad hlavičky. Nakoniec výsledný súbor dát deduplikovali na úrovni súborov s presnými zhodami. [32]

Wikipedia [4.5%]: Pridáva výpisy z Wikipédie z obdobia jún - august 2022, ktoré pokrývajú 20 jazykov, ktoré používajú latinku alebo cyriliku. Dáta spracúva tak, aby odstránili hypertextové odkazy, komentáre a iné formátovacie prvky. [32]

Gutenberg a Books3 [4.5%]: Do tréningového súboru dát zahrnuli dva knižné korpusy: projekt Gutenberg, ktorý obsahuje knihy, ktoré sú verejnou doménou, a časť Books3. [32]

ArXiv [2.5%]: Spracúva súbory arXiv Latex, aby do súboru dát pridali vedecké dáta. Odstránili všetko pred prvou časťou, ako aj bibliografiu. Odstránili aj komentáre zo súborov .tex a inline-rozšírené definície a makrá napísané používateľmi, aby zvýšili konzistenciu medzi jednotlivými dokumentmi. [32]

Stack Exchange [2%]: Zahŕňa výpis Stack Exchange, webovej stránky s vysokokvalitnými otázkami a odpoveďami, ktorá pokrýva rôznorodé oblasti, od informatiky až po chémiu. Získali dáta z 28 najväčších webových stránok, z textu odstránili značky HTML a odpovede zoradili podľa skóre (od najvyššieho po najnižšie). [32]

- **Tokenizácia:** Dáta tokenizujú pomocou algoritmu byte-pair encoding (BPE), pričom používa implementáciu z programu SentencePiece. [56] Najmä všetky čísla rozdeľuje na jednotlivé číslice a na rozklad neznámych znakov UTF-8 sa vracia k bajtom. Celkovo celá tréningová množina dát obsahuje po tokenizácii približne 1.4 T tokenov. Pri väčšine tréningových dát sa každý token použije len raz počas tréningovania, s výnimkou domén Wikipedia a Books, nad ktorými vykonáva približne dve epochy. [32]

4.1.3.2 LLaMA 2

Druhá generácia modelov LLaMA [55] priniesla vylepšenia v oblasti architektúry, tréningovania aj generalizácie. Bola uvedená v júli 2023 v spolupráci so spoločnosťou Microsoft. LLaMA 2 je dostupný vo verziách 7B, 13B a 70B a je tréňovaný na väčšej množine dát (2T tokenov). Pribudli aj inštrukčne ladené verzie, tzv. *LLaMA 2-Chat*, optimalizované na generovanie odpovedí v konverzačných úlohách.

- **Predtréningovanie:** S použitím optimalizovaného auto-regresívneho transformátora, vykonali niekoľko zmien na zlepšenie výkonu oproti LLaMA 1.

Konkrétne vykonali robustnejšie čistenie dát, aktualizovali mixy dát, trénovali o 40% viac celkových tokenov, zdvojnásobili dĺžku kontextu a použili skupinovú pozornosť (GQA) 2.5.4.5, aby zlepšili škálovateľnosť inferencií pre naše väčšie modely. Tréningový korpus obsahuje nový mix dát z verejne dostupných zdrojov, ktorý nezahŕňa údaje z produktov alebo služieb Meta. Odstránili dáta z určitých stránok, o ktorých je známe, že obsahujú veľké množstvo osobných informácií o súkromných osobách. Trénovali na 2 biliónoch tokenov dát, pretože to poskytuje dobrý kompromis medzi výkonom a nákladmi, pričom prevzorkovali tie najaktuálnejšie zdroje v snahe zvýšiť znalosti a zmierniť halucinácie. [55] Väčšinu predtréningového nastavenia modelu prebrali z LLaMA 1. [55]

- **Architektúra:** Väčšinu architektúry modelu prebrali z LLaMA 1. Používa štandardnú architektúru transformera, aplikuje prednormalizáciu pomocou RMSNorm, používa funkciu aktivácie SwiGLU a rotačné pozičné vloženia (RoPE 2.5.4.4). Medzi hlavné architektonické rozdiely oproti LLaMA 1 patrí zvýšená dĺžka kontextu a skupinovú pozornosť dopytu (GQA) 2.5.4.5. [55]

Prednormalizácia: Na zlepšenie stability trénovania normalizuje vstup každej tranformerovej podvrstvy namiesto normalizácie výstupu. Používa normalizačnú funkciu RMSNorm 2.5.4.1, ako to je aj v LLaMA 1.

Aktivačná funkcia: Používa funkciu aktivácie SwiGLU.

- **Optimalizátor:** Trénovali pomocou optimalizátora AdamW [35], s hyperparametrami $\beta_1 = 0.9$, $\beta_2 = 0.95$ a $\epsilon = 10^{-5}$. Používa kosínusový plán rýchlosti učenia, so zahriatím 2 000 krokov a konečnou rýchlosťou učenia až na 10% maximálnej rýchlosti učenia. Používa pokles hmotnosti 0.1 a orezanie gradientu 1.0. [55]
- **Tokenizácia:** Používa rovnaký tokenizér ako LLaMA1. Využíva algoritmus kódovania bajtových párov (BPE) s použitím implementácie od Sentence-Piece [56]. Rovnako ako v prípade LLaMA1 rozdeľujeme všetky čísla na jednotlivé číslice a používa bajty na rozklad neznámych znakov UTF-8. Celková veľkosť slovnej zásoby je 32 000 tokenov. [55]

LLaMA2-Chat je výsledkom niekoľkomesačného výskumu a opakovaných aplikácií zoraďovacích techník, vrátane ladenia inštrukcií a RLHF, ktoré si vyžadujú značné výpočtové a anotačné zdroje. Model bol doladovaný pomocou doladenia pod dohľadom, ako aj počiatočného a opakovaného modelovania odmien a RLHF. Využíva tiež novú techniku, Ghost Attention, o ktorej zistili, že pomáha kontrolovať tok dialógov vo viacerých otáčkach. [55]

GAtt metóda Predpokladajme, že máme prístup k viacotáčkovej dialógovej množine medzi dvoma osobami (napr. užívateľ a asistent) so zoznamom správ $[u_1, a_1, u_2, a_2, \dots, u_n, a_n]$, kde u_n a a_n zodpovedajú správam užívateľa a asistenta pre poradie n . Potom definujeme pokyn *inst*, ktorý by mal byť rešpektovaný počas celého dialógu. Napríklad, *inst* môže byť „konať ako“ (napr.

”konať ako historik”). Potom môžeme synteticky zrefaziť túto inštrukciu ku všetkým užívateľským správam konverzácie, t.j. každý vstup užívateľa u_n nahradíme ako: $u'_n = \text{inst} \| u_n$, kde $\|$ označuje operáciu zrefazovania textu. Týmto spôsobom zabezpečí, že kontext inštrukcie sa zachová počas celého dialógu. [55]

4.1.3.3 LLaMA 3

Ide o sadu jazykových modelov (1B, 3B, 8B, 70B, 90B, 405B), ktoré natívne podporujú viacjazyčnosť, kódovanie, uvažovanie a používanie nástrojov. Najväčší model je hustý Transformer s parametrami 405B a kontextovým oknom až 128 000 tokenov. LLaMA 3 [54] poskytuje porovnateľnú kvalitu s poprednými jazykovými modelmi, ako je GPT-4, pri množstve úloh. LLaMA 3 integruje schopnosti obrazu, videa a reči prostredníctvom kompozičného prístupu. [54]

- **Predtrénovanie:** Trénuje model v oveľa väčšom rozsahu ako predchádzajúce modely LLaMA. Vlajkový jazykový model bol vopred natrénovaný pomocou $3,8 \times 10^{25}$ FLOP, takmer 50 krát viac ako najväčšia verzia LLaMA 2. Konkrétne predtrénovali model vlajkovej lode s trénovateľnými parametrami 405B na 15.6 T textových tokenoch. Ako sa očakávalo podľa škálovacích zákonov pre základné modely, vlajkový model prekonáva menšie modely trénované pomocou rovnakého postupu. Aj keď zákony o škálovaní naznačujú, že vlajkový model má približne výpočtovo optimálnu veľkosť pre rozpočet na tréning, trénuje aj menšie modely oveľa dlhšie, než je výpočtovo optimálne. Výsledné modely fungujú lepšie ako výpočtovo optimálne modely pri rovnakom inferenčnom rozpočte. [54]
- **Dáta:** V porovnaní s predchádzajúcimi verziami LLaMA zlepšili kvantitu aj kvalitu dát, ktoré sa používajú na predtréning. Tieto vylepšenia zahŕňajú vývoj starostlivejších kanálov na predbežné spracovanie a úpravu dát pred tréňovaním. Predtrénuje Llamu 3 na korpuse s približne 15 T viacjazyčnými tokenmi. [54] Podobne ako v predchádzajúcich modeloch vytvára súbor dát z rôznych zdrojov dát obsahujúcich znalosti do konca roku 2023. Na každý zdroj dát aplikuje niekoľko metód deduplikácie a mechanizmov čistenia dát, aby získal vysokokvalitné tokeny. Odstraňuje domény, ktoré obsahujú veľké množstvo informácií umožňujúcich identifikáciu osôb. [54]
- **Architektúra:** LLaMA 3 používa štandardnú, hustú architektúru transformera. Z hľadiska architektúry modelu sa výrazne neodchyľuje od LLaMA a LLaMA 2. Zvýšenie výkonnosti je primárne poháňané zlepšením kvality a rozmanitosti dát, ako aj zvýšeným rozsahom školení. V porovnaní s Llamou 2 urobili niekoľko malých úprav. Používa skupinovú pozornosť pri dopyte (GQA) 2.5.4.5 s 8 hlavičkami kľúč-hodnota na zlepšenie rýchlosti inferencie a na zníženie veľkosti vyrovnávacích pamätí kľúč-hodnota počas dekodovania. Používa masku pozornosti, ktorá zabraňuje sebaupozorňovaniu medzi rôznymi dokumentmi v rovnakej sekvencii. Táto zmena mala obmedzený

vplyv počas štandardného predtréningu, ale považuje sa za dôležitú pri pokračovaní predtréningu vo veľmi dlhých sekvenciách. Zvýšili hyperparameter základnej frekvencie RoPE 2.5.4.4 na 500 000. To umožnilo lepšie podporovať dlhšie kontexty. Táto hodnota je účinná pre dĺžky kontextu do 32 768. LLaMA 3 405B používa architektúru so 126 vrstvami, rozmer reprezentácie tokenu 16 384 a 128 hláv pozornosti. LLaMA 3 8B používa architektúru so 32 vrstvami, rozmer reprezentácie tokenu 4096 a 32 hláv pozornosti. LLaMA 3 70B používa architektúru so 80 vrstvami, rozmer reprezentácie tokenu 8192 a 64 hláv pozornosti. [54]

Aktivačná funkcia: Takisto, ako predošlé LLaMA modely používa funkciu aktivácie SwiGLU.

- **Optimalizátor:** Predtrénuje Llamu 3 405B pomocou AdamW, lineárnym zahrievaním 8 000 krokov a kosínusovým rozvrhom rýchlosti učenia sa klesajúcim. Na začiatku tréningu používa nižšiu veľkosť dávky na zlepšenie stability tréningu a následne ju zvyšuje, aby zlepšili efektivitu. Konkrétne používa počiatočnú veľkosť dávky 4 miliónov tokenov a sekvencie s dĺžkou 4 096 a tieto hodnoty zdvojnásobí na veľkosť dávky 8 miliónov sekvencií 8 192 tokenov po predbežnom natrénovaní 252 miliónov tokenov. Po predtrénovaní na tokenoch 2.87 T veľkosť dávky opäť zdvojnásobí na 16M. [54]
- **Tokenizácia:** Slovná zásoba tokenov kombinuje 100 000 tokenov z tokenizéra tiktoken3 s 28 000 dodatočnými tokenmi na lepšiu podporu neanglických jazykov. V porovnaní s tokenizérom LLaMA 2 nový tokenizér zlepšuje mieru kompresie na vzorke anglických dát z 3,17 na 3,94 znakov na token. To umožňuje modelu „čítať“ viac textu pri rovnakom množstve tréningových výpočtov. Pridanie 28 000 tokenov z vybraných neanglických jazykov zlepšilo kompresný pomer aj následný výkon bez vplyvu na anglickú tokenizáciu. [54]

4.1.4 DeepSeek

Predstavuje sériu veľkých jazykových modelov, ktoré vyvíja spoločnosť DeepSeek AI. Tieto modely sú navrhnuté s cieľom poskytovať otvorené a vysoko výkonné riešenia pre generovanie prirodzeného jazyka, kódovania a ďalšie úlohy umelej inteligencie. DeepSeek sa vyznačuje dôrazom na efektívnu architektúru, robustné trénovacie postupy a vysokú dostupnosť výskumným aj priemyselným komunitám. Rozoberieme si vývoj LLM od DeepSeek od prvého open-source LLM DeepSeek-LLM až po najmodernejší DeepSeek-V3 a ukážeme rozdiely a inovácie, ktoré priniesli.

4.1.4.1 DeepSeek-LLM

Prvým modelom sa zaoberalo DeepSeek-AI v 2024, kde motivácia plynula z skúmania škálovacieho správania jazykových modelov a aplikuje zistenia v

dvoch široko používaných konfiguráciách veľkých modelov, konkrétne DeepSeek-7B a DeepSeek-67B. Cieľom štúdie bolo položiť základy pre budúce škálovanie LLM s otvoreným zdrojovým kódom, čím sa pripraví pôda pre ďalší pokrok v tejto oblasti. Konkrétne najprv preskúmali zákony škálovania veľkosti dávky a rýchlosti učenia a našli ich trendy s veľkosťou modelu. Na základe toho vykonali komplexnú štúdiu zákonov škálovania dát a modelovej škály, pričom úspešne odhalili optimálnu stratégiu pridelovania škálovania modelov/dát a predpovedali očakávaný výkon spomínaných modelov. [57]

- **Architektúra:** Mikrodizajn DeepSeek LLM do veľkej miery kopíruje dizajn LLaMA [32], pričom prijíma štruktúru Pre-Norm s funkciou RMS-Norm 2.5.4.1 a používa SwiGLU, ako aktivačnú funkciu pre doprednú neuronovú sieť (FFN). Zahŕňa aj rotačné vloženia na pozičné kódovanie (RoPE 2.5.4.4). Na optimalizáciu nákladov na inferenciu model 67B používa Grouped-Query Attention (GQA) namiesto tradičného Multi-Head Attention (MHA). Z hľadiska makroprojektu sa však DeepSeek LLM mierne líši. Konkrétne, DeepSeek LLM 7B je 30-vrstvová sieť, zatiaľ čo DeepSeek LLM 67B má 95 vrstiev. Tieto úpravy vrstiev pri zachovaní konzistentnosti parametrov s inými modelmi s otvoreným zdrojovým kódom tiež uľahčujú rozdelenie modelovej pipeline s cieľom optimalizovať tréning a inferenciu. Na rozdiel od väčšiny prác využívajúcich Grouped-Query Attention (GQA) parametre modelu 67B rozšírili v hĺbke siete namiesto bežnej praxe rozširovania medzipriestoru vrstiev FFN s cieľom dosiahnuť lepší výkon. [57]
- **Dáta:** Cieľom výberu dát je zvýšiť bohatosť a rozmanitosť súboru dát. Na dosiahnutie týchto cieľov proces zberu dát rozdelili do troch základných fáz: deduplikácia, filtrovanie a remixovanie. Etapy deduplikácie a remixovania zabezpečujú rozmanitú reprezentáciu dát výberom jedinečných prípadov. Etapa filtrovania zvyšuje hustotu informácií, čím umožňuje efektívnejšie a účinnejšie tréning modelov. Zvolili agresívnu stratégiu deduplikácie, čím rozšírili rozsah deduplikácie. Deduplikácia celého korpusu Common Crawl vedie k vyššiemu odstráneniu duplicitných prípadov v porovnaní s deduplikáciou v rámci jedného výpisu. Vo fáze filtrovania sa zameriava na vývoj spoľahlivých kritérií na hodnotenie kvality dokumentov. To zahŕňa podrobnú analýzu zahŕňajúcu lingvistické aj sémantické hodnotenia, ktorá poskytuje pohľad na kvalitu dát z individuálneho aj globálneho hľadiska. Vo fáze remixovania upravuje prístup s cieľom riešiť nerovnováhu dát, pričom sa zameriava na zvýšenie zastúpenia nedostatočne zastúpených domén. Cieľom tejto úpravy je dosiahnuť vyváženejší a inkluzívnejší súbor dát a zabezpečiť, aby boli primerane zastúpené rôzne perspektívy a informácie. [57]
- **Optimalizácia:** DeepSeek LLM je natrénovaný pomocou optimalizátora AdamW [35] s hyperparametrami $\beta_1 = 0.9$, $\beta_2 = 0.95$ a *weight_decay* =

0, 1. Počas predtrénovania sa namiesto typického kosínusového plánovača používa viacstupňový plánovač miery učenia. Konkrétne, rýchlosť učenia modelu dosiahne maximálnu hodnotu po 2000 zahrievacích krokoch a potom klesne na 31,6% maximálnej hodnoty po spracovaní 80% trénujúcich tokenov. Ďalej sa znižuje na 10% maximálnej hodnoty po 90% tokenov. Orežanie gradientu počas fázy trénovania je nastavené na hodnotu 1,0. Na základe zistení pozorovali, že napriek rozdielom v trende znižovania strát počas trénovania je konečný výkon pri použití viackrokového plánovača miery učenia v podstate konzistentný s plánovačom kosínusovým. Pri úprave rozsahu trénovania pri zachovaní fixnej veľkosti modelu umožňuje viacstupňový plánovač miery učenia opätovné použitie trénovania z prvej fázy, čo ponúka jedinečný komfort pri kontinuálnom trénovaní. Preto zvolili viacstupňový plánovač miery učenia ako predvolené nastavenie. V záujme vyváženia pomerov opätovného použitia pri nepretržitom trénovaní a výkonnosti modelu sa však rozhodli pre uvedené rozdelenie 80%, 10% a 10% pre tri etapy. [57]

- **Tokenizácia:** Ako tokenizér implementovali algoritmus BBPE (Byte-level Byte-Pair Encoding) založený na knižnici tokenizérov z Hugging Face. Na zabránenie spájaniu tokenov z rôznych kategórií znakov, ako sú nové riadky, interpunkčné znamienka a čínsko-japonsko-korejské symboly, sa použila predtokenizácia, podobne ako v prípade GPT-2. Rozdelili čísla na jednotlivé číslice podľa prístupu použitého v LLaMA [32]. Stanovili počet konvenčných tokenov v slovníku na 100 000. Tokenizér bol natrénovaný na viacjazyčnom korpuse s veľkosťou približne 24 GB a konečný slovník rozšírili o 15 špeciálnych tokenov, čím dosiahli celkovú veľkosť 100 015. Aby zabezpečili výpočtovú efektívnosť počas trénovania a vyhradili priestor pre prípadné ďalšie špeciálne tokeny, ktoré by mohli byť potrebné v budúcnosti, nakonfigurovali veľkosť slovníka modelu na 102400 na trénovanie. [57]

4.1.4.2 DeepSeek-V2

DeepSeek-V2 je silný open-source jazykový model Mixture-of-Experts (MoE), ktorý sa vyznačuje ekonomickým trénovaním a efektívnou inferenciou prostredníctvom architektúry transformer. Je vybavený celkom 236B parametrami, z ktorých 21B je aktivovaných pre každý token, a podporuje dĺžku kontextu 128K tokenov. Optimalizujte moduly pozornosti a siete Feed-Forward Networks (FFN) v rámci transformera pomocou navrhovaného Multi-head Latent Attention (MLA) a DeepSeekMoE [58]. V kontexte mechanizmov pozornosti predstavuje vyrovnávacia pamäť kľúč-hodnota (KV) viachlavovej pozornosti (MHA) 2.4.1.4 významnú prekážku efektívnosti inferencie LLM. Na vyriešenie tohto problému boli preskúmané rôzne prístupy, vrátane Grouped-Query Attention (GQA) a Multi-Query Attention (MQA). Tieto metódy však zmierňujú kompromisný výkon pri ich pokuse o zníženie vyrovnávacej pamäte KV. Aby dosiahli to najlepšie z oboch svetov, predstavili MLA, mechanizmus

pozornosti vybavený nízko hodnotovou kompresiou kľbu kľúč-hodnota. Empiricky MLA dosahuje lepší výkon v porovnaní s MHA a medzitým výrazne znižuje KV vyrovnávaciu pamäť počas inferencie, čím zvyšuje efektívnosť inferencie. [49]

- **Architektúra:** DeepSeek-V2 je vo všeobecnosti stále v architektúre transformer, kde každý blok transformera pozostáva z modulu pozornosti a siete Feed-Forward (FFN). Pre modul pozornosti aj FFN však navrhuje a používa inovatívne architektúry. Pre pozornosť navrhuje Multi-head Latent Attention (MLA), ktorý využíva nízkoúrovňovú spoločnú kompresiu kľúčov a hodnôt na elimináciu úzkeho hrdla v podobe vyrovnávacej pamäte kľúčov a hodnôt v čase inferencie, čím podporuje efektívnu inferenciu. Pre sieť FFN sa riadi architektúrou DeepSeekMoE [58], ktorá využíva jemnú odbornú segmentáciu a zdieľanú odbornú izoláciu pre vyšší potenciál v špecializácii odborníkov. Architektúra DeepSeekMoE [58] demonštruje veľké výhody v porovnaní s konvenčnými architektúrami MoE, čo umožňuje trénovať silné modely za ekonomické náklady. Keďže pri tréovaní využíva expertný paralelizmus, navrhuje aj doplnkové mechanizmy na riadenie réžie komunikácie a zabezpečenie vyváženia záťaže. Kombináciou týchto dvoch techník sa DeepSeek-V2 vyznačuje silným výkonom, ekonomickými nákladmi na tréovanie a efektívnou priepustnosťou záverov súčasne. Pokiaľ ide o ostatné drobné detaily normalizáciu vrstiev a aktivačnú funkciu vo FFN, DeepSeek-V2 sa riadi nastaveniami DeepSeek 67B [57, 49].

Multi-head Latent Attention: Konvenčné modely transformerov zvyčajne využívajú Multi-Head Attention (MHA) 2.4.1.4, ale počas generovania sa ich ťažká vyrovnávacia pamäť Key-Value (KV) stane úzkym miestom, ktoré obmedzuje účinnosť inferencie. S cieľom znížiť vyrovnávaciu pamäť KV sa navrhuje Multi-Query Attention (MQA) 2.5.4.5 a Grouped-Query Attention (GQA) 2.5.4.5. Vyžadujú menšiu veľkosť vyrovnávacej pamäte KV, ale ich výkonnosť sa nevyrovná MHA. DeepSeek-V2 navrhuje inovatívny mechanizmus pozornosti nazvaný Multi-head Latent Attention (MLA). Vybavený nízkoúrovňovou spoločnou kompresiou kľúčov a hodnôt dosahuje MLA lepší výkon ako MHA, ale vyžaduje podstatne menšie množstvo vyrovnávacej pamäte KV. [49]

Vloženie oddelenej rotačnej polohy: Alebo Decoupled Rotary Position Embedding je modifikácia klasického RoPE 2.5.4.4 kvôli kompatibiliti s MLA. V nadväznosti na DeepSeek 67B sa plánovalo pre DeepSeek-V2 použiť Rotary Position Embedding (RoPE). RoPE je však nekompatibilný s kompresiou KV s nízkym rangom a je citlivý na pozíciu pre kľúče aj požiadavky. Ak aplikuje RoPE pre kľúče bude spojená s maticou RoPE citlivou na pozíciu. Týmto spôsobom sa počas inferencie už nemôže absorbovať. V dôsledku toho musí počas inferencie prepočítavať kľúče pre všetky prefixové tokeny, čo výrazne zníži efektívnosť inferencie. Ako riešenie navrhuje oddelenú stratégiu RoPE, ktorá využíva dodatočné viachlavové požiadavky a

zdieľaný kľúč na vykonanie RoPE. [49]

- **Dáta:** Vysokokvalitný a viaczdrojový predtréningový korpus pozostávajúci z 8.1T tokenov. V porovnaní s korpusom používaným v DeepSeek67B tento korpus obsahuje rozšírené množstvo dát, najmä čínskych, a vyššiu kvalitu dát. Pri zachovaní rovnakých fáz spracovania dát ako v prípade DeepSeek 67B predlžuje množstvo dát a zvyšuje ich kvalitu. S cieľom zväčšiť predtréningový korpus skúma potenciál internetových dát a optimalizuje procesy čistenia, čím obnoví veľké množstvo omylom vymazaných dát. Okrem toho začleňuje viac čínskych dát s cieľom lepšie využiť korpus dostupný na čínskom internete. Okrem množstva dát sa zameriava aj na ich kvalitu. Predtréningový korpus obohacuje o vysokokvalitné údaje z rôznych zdrojov a medzitým zlepšuje algoritmus filtrovania založený na kvalite. Vylepšený algoritmus zabezpečuje, že sa odstráni veľké množstvo neprínosných dát, zatiaľ čo hodnotné údaje sa väčšinou zachovávajú. Okrem toho z predtréningového korpusu odfiltruje sporný obsah, aby zmiernili skreslenie dát spôsobené špecifickými regionálnymi kultúrami. [49]
- **Optimalizácia:** Používa optimalizátor AdamW [35] s hyperparametrami nastavenými na $\beta_1 = 0.9$, $\beta_2 = 0.95$ a $weight_decay = 0.1$. Rýchlosť učenia je naplánovaná pomocou stratégie zahrievania a postupného rozpadu. Na začiatku sa rýchlosť učenia lineárne zvyšuje z 0 na maximálnu hodnotu počas prvých 2K krokov. Následne sa miera učenia po natrénovaní približne 60% tokenov vynásobí hodnotou 0.316 a po natrénovaní približne 90% tokenov opäť hodnotou 0.316. Maximálna rýchlosť učenia je nastavená na 2.4×10^{-4} a norma orezania gradientu je nastavená na 1.0. Používa tiež stratégiu plánovania veľkosti dávky, kde sa veľkosť dávky postupne zvyšuje z 2304 na 9216 pri tréňovaní prvých 225B tokenov a potom sa pri zvyšnom tréňovaní zachováva 9216. Maximálna dĺžka sekvencie je nastavená na 4K a DeepSeek-V2 trénuje na 8.1T tokenov. Využíva paralelnosť pipeline na nasadenie rôznych vrstiev modelu na rôznych zariadeniach a pre každú vrstvu budú smerovaní experti rovnomerne nasadení na 8 zariadeniach. Pokiaľ ide o smerovanie obmedzené na zariadenie, každý token bude odoslaný najviac na 3 zariadenia. [49]
- **Tokenizácia:** Prijíma rovnaký tokenizér, aký sa používa v DeepSeek 67B, ktorý je vytvorený na základe algoritmu BBPE (Byte-level Byte-Pair Encoding) a má veľkosť slovníka 100K. Tokenizovaný predtréningový korpus obsahuje 8.1T tokenov, pričom čínskych tokenov je približne o 12% viac ako anglických. [49]

4.1.4.3 DeepSeek-V3

DeepSeek-V3, veľký model Mixture-of-Experts (MoE) s parametrami 671B, z ktorých 37B je aktivovaných pre každý token. S perspektívou do budúcnosti

sa neustále snaží o silný výkon modelu a ekonomické náklady. Preto, pokiaľ ide o architektúru, DeepSeek-V3 stále používa Multi-head Latent Attention (MLA) ako v jeho predchádzajúcej verzii DeepSeek-V2 [49] na efektívnu inferenciu a DeepSeekMoE [58] na nákladovo efektívne trenovanie. Okrem základnej architektúry implementuje dve ďalšie stratégie na ďalšie vylepšenie možností modelu. Po prvé, DeepSeek-V3 je priekopníkom bezstratovej stratégie (auxiliary-loss-free strategy) [59] na vyrovňovanie záťaže s cieľom minimalizovať nepriaznivý vplyv na výkonnosť modelu. Po druhé, DeepSeek-V3 využíva multi-tokenový predikčný tréningový cieľ. Aby dosiahli efektívny tréning, podporuje zmiešaný presný tréning FP8 a implementujeme komplexné optimalizácie tréningového rámca. Tréning s nízkou presnosťou sa ukázal, ako sľubné riešenie efektívneho tréningu. [49]

- **Architektúra:** Základnú architektúru DeepSeek-V3 predstavuje Multi-head Latent Attention (MLA) pre efektívnu inferenciu a DeepSeekMoE pre ekonomický tréning. Tréningový cieľ Multi-Token Prediction (MTP), ktorý prináša zlepšenie celkového výkonu. Pokiaľ ide o ďalšie zmeny, DeepSeek-V3 dodržiava nastavenia DeepSeekV2. Základná architektúra DeepSeek-V3 je stále v rámci transformer. V porovnaní s DeepSeek-V2 je výnimkou, že dodatočne zavádza pomocnú bezstratovú stratégiu vyrovňovania záťaže (auxiliary-loss-free strategy) pre DeepSeekMoE, aby zmiernil degradáciu výkonu spôsobenú pevnosťou, aby sa zabezpečilo vyváženie záťaže. Pre pozornosť, DeepSeek-V3 prijíma architektúru Multi-head Latent Attention (MLA). Jadrom MLA je nízkohodnotná spoločná kompresia pre kľúče a hodnoty pozornosti na zníženie vyrovnávacej pamäte kľúča a hodnoty (KV) počas inferencie. Pre dopredné neurovnové siete (FFN) využíva DeepSeek-V3 architektúru DeepSeekMoE. V porovnaní s tradičnými architektúrami MoE, DeepSeekMoE využíva jemnejších odborníkov a izoluje niektorých odborníkov ako zdieľaných. [49]

Auxiliary-Loss-Free Load Balancing: V prípade modelov MoE povedie nevyvážená expertná záťaž ku kolapsu smerovania a zníži výpočtovú efektivitu v scenároch s expertným paralelizmom. Konvenčné riešenia sa zvyčajne spoliehajú na pomocnú stratu, aby sa predišlo nevyváženému zaťaženiu. Príliš veľká pomocná strata však zhorší výkon modelu. K dosiahnutiu lepšieho kompromisu medzi vyvážením záťaže a výkonom modelu, zavádza pomocnú bezstratovú stratégiu vyrovňovania záťaže, aby zabezpečil vyváženie záťaže. Zavádza skreslený výraz pre každého experta a pridá ho k zodpovedajúcim skóre afinity na určenie smerovania top-K. Výraz skreslenia sa používa iba na smerovanie. Konečná hodnota, ktorá sa vynásobí výstupom FFN, je stále odvodená od pôvodného skóre afinity. [49]

Multi-Token Prediction (MTP) rozširuje rozsah predikcie na viacero budúcich tokenov na každej pozícii. Na jednej strane cieľ MTP zahusťuje tréningové signály a môže zlepšiť efektivitu dát. Na druhej strane, MTP môže umožniť modelu vopred naplánovať svoje reprezentácie pre lepšiu

predikciu budúcich tokenov. Konkrétne implementácia MTP používa D sekvenčný modul ako predpovedať D ďalšie tokeny. [49]

- **Dáta:** V porovnaní s DeepSeek-V2 optimalizuje korpus predtrénungu zvýšením pomeru matematických a programovacích dát a zároveň rozširuje viacjazyčné pokrytie nad rámec angličtiny a čínštiny. Pipelina na spracovanie dát je tiež prepracovaná, aby sa minimalizovala nadbytočnosť pri zachovaní rozmanitosti korpusov. Implementuje metódu balenia dokumentov pre integritu dát, ale nezahŕňa maskovanie pozornosti medzi vzorkami počas tréningovania. Napokon, tréningový korpus pre DeepSeek-V3 pozostáva z 14.8T vysoko kvalitných a rôznorodých tokenov. Stratégia Fill-in-Middle (FIM) neohrozuje schopnosť predikcie ďalšieho tokenu a zároveň umožňuje modelu presne predpovedať stredný text na základe kontextových podnetov. V súlade s DeepSeekCoder-V2 tiež začleňuje stratégiu FIM do predtrénungu DeepSeek-V3. Používame rámec Prefix-Suffix-Middle (PSM) na štruktúrovanie dát. [49]
- **Optimalizácia:** Používa optimalizátor AdamW [35] s hyperparametrami nastavenými na $\beta_1 = 0.9$, $\beta_2 = 0.95$ a $weight_decay = 0.1$. Počas predtrénungu je nastavená maximálna dĺžka sekvencie na 4K a predtrénovaný DeepSeek-V3 na 14.8T tokenoch. Čo sa týka plánovania rýchlosti učenia, najprv lineárne zvyšuje z 0 na 2.2×10^{-4} počas prvých 2K krokov. Potom udržiava konštantnú rýchlosť učenia 2.2×10^{-4} , kým model nespotrebuje 10T tréningových tokenov. Následne postupne znižuje rýchlosť učenia na 2.2×10^{-5} v 4.3T tokenoch podľa kosínusovej krivky rozpadu. Počas tréningu posledných 500B tokenov udržiava konštantnú rýchlosť učenia 2.2×10^{-5} v prvých 333B tokenoch a prepne na ďalšiu konštantnú rýchlosť učenia zo 7.3×10^{-6} v zostávajúcich 167B tokenoch. Stratégia plánovania veľkosti dávky, kde sa veľkosť dávky postupne zvyšuje z 3072 na 15360 počas tréningovania prvých 469B tokenov a potom si udržiava 15360 počas zostávajúceho tréningovania. [49]
- **Tokenizácia:** Tokenizér pre DeepSeek-V3 využíva BPE na úrovni bajtov s rozšírenou slovnou zásobou 128 000 tokenov. Pretokenizer a tréningové dáta pre tokenizér sú upravené tak, aby optimalizovali efektivitu viacjazyčnej kompresie. Okrem toho, v porovnaní s DeepSeek-V2, nový pretokenizer zavádza tokeny, ktoré kombinujú interpunkciu a zalomenie riadkov. Rozdelili určitú časť takýchto kombinovaných tokenov, čo vystavuje model širšiemu spektru špeciálnych prípadov a zmierňuje zaujatosť. [49]

[illegible]

Meranie výkonu

Benchmarkovanie výkonu LLM je kľúčové pri analýze ich možností a pri rozhodovaní o ich nasadení v reálnych aplikáciách. Rôzne použitia umelej inteligencie, ako sú chatboty, automatizované generovanie textu, sumarizácia dokumentov alebo komplexné analytické nástroje, majú rozdielne nároky na latenciu, priepustnosť a hardvérové zdroje. Aby sme mohli efektívne zvoliť vhodný model a infraštruktúru, je nevyhnutné najskôr identifikovať konkrétne požiadavky daného use-casu, napríklad požadovaný čas odozvy alebo objem dát na spracovanie. Na základe týchto požiadaviek si treba stanoviť cieľ merania a definovať výkonnostné metriky, ako sú napríklad latencia generovania, spotreba pamäte alebo priepustnosť (throughput). Tieto ukazovatele umožňujú objektívne porovnanie medzi rôznymi modelmi a zároveň slúžia ako vstup pre rozhodovanie o optimalizácii infraštruktúry.

V tejto časti práce sa preto sústreďíme na meranie výkonu viacerých open-source LLM rôznych veľkostí (konkrétne 1B, 8B, 32B parametrov) pri nasadení na výpočtových jednotkách NVIDIA A100 80GB GPU. Táto výpočtová architektúra predstavuje štandard v oblasti výkonného inferenčného aj tréningového prostredia a poskytuje dostatočnú pamäť aj pre stredne veľké modely. Pri experimentoch budeme skúmať vplyv rôznych premenných, ktoré ovplyvňujú výkon modelu. Prvou z nich je **veľkosť modelu** – počet parametrov a jeho dopad na latenciu a spotrebu pamäte, ďalej **počet dávok** (batch size) – teda koľko požiadaviek model obsluhuje paralelne alebo dá sa prirovnať k počtu užívateľov v jednej chvíli, **dĺžka vstupného a výstupného textu** – ktorá výrazne ovplyvňuje čas spracovania. Cieľom je vytvoriť jasný a kvantifikovateľný prehľad o tom, ako sa rôzne modely správajú pri rozdielnych nastaveniach a záťaži, čo umožní informované rozhodovanie o ich použití v praxi.

5.1 Metodika merania výkonu

S rastúcim záujmom jazykových modelov rastie aj pozornosť na ich testovanie výkonu, kde za posledné roky vznikli nástroje a knižnice, ktoré túto úlohu uľahčujú. Mnohé testy sa sústreďujú len na kvalitatívne hodnotenie, ako je presnosť odpovedí alebo schopnosť porozumieť otázkam, no v praxi je rovnako dôležité sledovať aj výkonnostné aspekty, ako sú latencia, priepustnosť a využitie pamäte. Testovanie výkonnosti LLM má preto viacero dimenzií, od časovej náročnosti generovania odpovedí pri rôznych typoch vstupov, cez spotrebu výpočtových zdrojov, až po schopnosť modelu zvládať veľký počet súbežných požiadaviek. Tieto faktory sú rozhodujúce pri nasadení modelov do produkčných systémov. Budeme používať moderné benchmarkové nástroje, ktoré umožňujú merať kľúčové parametre generovania a efektivity modelov v realistických podmienkach.

Výkon rôznych LLM budeme merať na platforme **ClusterFIT**. ClusterFIT je výpočetná platforma, ktorá umožňuje užívateľovi priehľadne využiť voľné výpočetné prostriedky FIT ČVUT. Je zložená z niekoľkých frontendov, ktoré komunikujú s výpočtovými uzlami rôznej konfigurácie. Frontend, ktorý sa v tejto práci využíva je založený na CPU architektúre amd64/x86_64, operačný systém Ubuntu LTS 8GB RAM 8xCPU. Na frontende a výpočetných uzloch sú nainštalované nástroje, ako CUDA, gcc, gdb, git, Lmod, nano, python3-venv, python3-pip, Slurm a iné, aby sa uľahčila práca s platformou. Ďalej bol v práci využívaný výpočetný uzol s NVIDIA Tesla A100 80GB. Na tomto uzle s týmto GPU sa testovali všetky modely obsiahnuté v tejto práci.

Na meranie výkonu LLM modelov používame nástroj **GenAI-Perf** [60], ktorý umožňuje konzistentné porovnanie rôznych modelov pomocou jednotného rozhrania. Je to nástroj príkazového riadka na meranie priepustnosti a latencie generatívnych modelov umelej inteligencie, ktoré sú obsluhované prostredníctvom inferenčného servera. Pre veľké jazykové modely GenAI-Perf poskytuje metriky, ktoré sú opísané v sekcii 5.3. GenAI-Perf vygeneruje zadanú záťaž, zmeria výkon inferenčného servera a oznámi metriky v jednoduchej tabuľke ako výstup konzoly. Nástroj tiež zaznamenáva všetky výsledky v súbore csv a json, ktoré možno použiť na odvodenie ďalších metrík a vizualizácií.

Príklad testovacieho skriptu, ktorý testuje záťaž na základe testovacích parametrov. V tomto konkrétnom prípade testuje model Qwen3-32B na úrovni 3, čo sa týka vstupov, cez všetky možnosti dávky. Využívame framework GenAI-Perf 2.57.0, ktorý vyžaduje Ubuntu 24.04, Python 3.10+ a CUDA 12. Príkaz `genai-perf profile` spustí proces benchmarkovania s požadovanými parametrami. Požadované parametre majú nasledujúci význam:

- *model (-m)* - meno modelu, ktorý meriame.
- *-stability-percentage* - Povolená odchýlka v meraniach latencie pri určovaní, či je výsledok stabilný. Meranie sa považuje za stabilné, ak je pomer max / min z posledných 3 meraní v rámci percenta stability, pokiaľ ide o odvodenie

za sekundu aj latenciu.

- *measurement-interval* - časový interval použitý pre každé meranie v milisekundách. Perf Analyzer bude vzorkovať zadaný časový interval a vykonávať merania nad požiadavkami dokončenými v rámci tohto časového intervalu. Pri použití predvoleného percenta stability bude GenAI-Perf vykonávať referenčné meranie počas $3 \times (\text{measurement-interval})$ milisekúnd.
- *service-kind* - druh služby, pre ktorú bude perf-analyzer generovať záťaž.
- *synthetic-input-tokens-mean* - priemerný počet tokenov v generovaných požiadavkách pri použití syntetických dát.
- *synthetic-input-tokens-stddev* - smerodajná odchýlka počtu tokenov v generovaných požiadavkách pri použití syntetických dát.
- *output-tokens-mean* - priemerný počet tokenov v každom generovanom výstupe.
- *output-tokens-stddev* - smerodajná odchýlka v každom generovanom výstupe.
- *endpoint-type* - typ koncového bodu, na ktorý sa majú odosielať požiadavky na serveri. Pri použití inferenčného serveru pomocou vLLM ponúka napríklad `/v1/chat/completions` alebo `/v1/completions`
- *url* - URL koncového bodu, na ktorý sa má zamerať benchmark.
- *concurrency* - hodnota súbežnosti, ktorú benchmarkuje.

Aby testovanie prebehlo pomocou nástroja `genai-perf`, je potreba nasaadiť inferenčný server. Na obsluhu inferenčného serveru využívame vLLM [61]. vLLM, čo je skratka pre virtual large language model, je knižnica s otvoreným zdrojovým kódom, ktorú spravuje komunita vLLM. Pomáha veľkým jazykovým modelom vykonávať výpočty efektívnejšie a vo väčšom rozsahu. Konkrétne vLLM je inferenčný server, ktorý urýchľuje výstup generatívnych aplikácií umelej inteligencie lepším využitím pamäte GPU. Inferenčný server je softvér, ktorý pomáha modelu umelej inteligencie vyvodzovať nové závery na základe jeho predchádzajúceho tréningu. Inferenčné servery posielajú vstupné požiadavky cez model strojového učenia a vracajú výstup.

Systém vLLM možno nasaadiť ako server, ktorý implementuje protokol OpenAI API. V predvolenom nastavení sa server spúšťa na adrese `http://localhost:8000`. Server hostí jeden model v danom čase a implementuje koncové body, ako `chat-completion` alebo `completions`. V predvolenom nastavení server použije súbor `generation-config.json` z úložiska modelu huggingface, ak existuje. To znamená, že predvolené hodnoty niektorých parametrov môžu byť nahradené hodnotami odporúčanými tvorcom modelu. V predvolenom nastavení server používa preddefinovanú šablónu chatu uloženú v tokenizéri. Na

```
#!/bin/bash

con=(1 4 16 32 64 128)
a=(20000 20000 30000 30000 40000)

for i in "${!con[@]}" ; do
    concurenc=${con[$i]}
    interval=${a[$i]}

    echo "Testing concurrency=$concurenc, interval=$interval"

    genai-perf profile \
        -m Qwen/Qwen3-32B \
        --stability-percentage 95 \
        --measurement-interval "$interval" \
        --service-kind openai \
        --synthetic-input-tokens-mean 1000 \
        --synthetic-input-tokens-stddev 50 \
        --output-tokens-mean 1000 \
        --output-tokens-stddev 50 \
        --endpoint-type completions \
        --url http://10.38.7.167:8000 \
        --concurrency "$concurenc"
done
```

■ **Výpis kódu 5.1** Script testujúci rôznu záťaž pri inferencií modelu pomocou nástroju GenAI-Perf.

platforme ClusterFIT sa plánovanie a spúšťanie úloh realizuje pomocou plánovača Slurm. Preto nasledujúci script definuje konkrétne parametre, výpočetný uzol, časť výpočetného uzla, GPU ktoré sa má použiť atď. Script nasadí server s modelom Qwen3-32B s maximálnym možným vstupom 2048 s predvoleným nastavením presnosti váh FP16. Na použitie modelov z platformy hugging face je potreba autorizácia a na využitie potrebných knižníc bolo vytvorené virtuálne prostredie s názvom vllm. Monitoring pamäte GPU pri inferencií bolo spracované pomocou gpustat.

5.2 Testovacie parametre

Na kvalitné zhodnotenie výkonu jednotlivých jazykových modelov je potrebné najskôr definovať testovacie parametre, ktoré dokážu verne odzrkadliť rôzne typy záťaže. Na ich základe potom vieme určiť vhodnosť konkrétneho modelu

```
#!/bin/bash
#SBATCH --job-name=llm
#SBATCH --partition=gpu
#SBATCH --nodelist=cf-prod-node-037
#SBATCH --gres=gpu:a100_80
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --output=32B_output.log
#SBATCH --error=32B_error.log

source vllm/bin/activate
huggingface-cli login --token $HF_TOKEN --add-to-git-credential

gpustat -a --watch >> gpu_32B &
GPU_MONITOR_PID=$!

vllm serve Qwen/Qwen3-32B --trust-remote-code \
    --max-model-len 2048

kill $GPU_MONITOR_PID
```

■ **Výpis kódu 5.2** Script na nasadenie LLM serveru pomocou vLLM pomocou plánovača Slurm.

pre špecifické použitie (use-case). V rámci testovania sme sa rozhodli porovnať modely rôznych veľkostí podľa počtu parametrov (čo je zároveň súčasťou ich názvu): LLaMA-3.2-1B-Instruct, LLaMA-3.1-8B-Instruct a Qwen2.5-32B. Výkon modelov budeme testovať na rôznych parametroch inferencie a to konkrétne s veľkosťami dávky (batch size) 1, 4, 16, 32, 64 a 128. Batch size pri inferencii určuje počet požiadaviek, ktoré sa spracúvajú naraz v jednej dávke. Inými slovami, je to počet vstupov (napr. textových promptov), ktoré model paralelne generuje na výstupe počas jedného prechodu cez model. Simuluje to teda používanie modelu viacerými užívateľmi zároveň. Testovanie ďalej zahŕňa aj rôzne kombinácie počtu vstupných a výstupných tokenov, konkrétne hodnoty 100, 200 a 1000. Tieto kombinácie sme rozdelili do niekoľkých úrovní zataženia nasledovne:

- **Úroveň 1.1** - priemerný počet vstupných tokenov je 100 a výstupných 200.
- **Úroveň 1.2** - priemerný počet vstupných tokenov je 200 a výstupných 100.
- **Úroveň 2.1** - priemerný počet vstupných tokenov je 200 a výstupných 1000.

- **Úroveň 2.2** - priemerný počet vstupných tokenov je 1000 a výstupných 200.
- **Úroveň 3** - priemerný počet vstupných tokenov je 1000 a výstupných 1000.

Na vstupné sekvencie ponúka GenAI-Perf dve možnosti a použitie požiadaviek zo súboru alebo syntetické dáta. V tejto práci, kvôli sústredeniu sa na výkon modelu z pohľadu potrebných zdrojov, využívame syntetické dáta.

5.3 Metriky testovania

Testovacie metriky predstavujú základný pilier pri rozhodovaní o tom, ktorý jazykový model je vhodné nasadiť do produkčného prostredia. Práve tieto metriky nám umožňujú objektívne porovnať výkonnosť rôznych modelov a určiť, ktorý z nich najefektívnejšie využíva dostupné výpočtové zdroje, a to pri zachovaní požadovanej úrovne latencie, priepustnosti či kvality odpovedí. Na základe hodnôt týchto metrík dokážeme optimalizovať výber modelu pre konkrétny use-case, či už ide o aplikáciu, kde je rozhodujúca rýchla latencia, alebo o úlohy, kde je dôležitá vysoká priepustnosť pri spracovaní veľkého množstva požiadaviek. Aby sme sa vyhli nejednoznačnej interpretácii výsledkov a zároveň zabezpečili konzistentnosť testovania, v nasledujúcej časti podrobne definujeme používané výkonnostné metriky a ich význam v kontexte testovania jazykových modelov.

- **Time to First Token (TTFT)** je čas potrebný na vytvorenie prvého výstupného tokenu po prijatí vstupnej požiadavky. Predstavuje, ako rýchlo môžu používatelia vidieť výstup LLM po odoslaní svojej požiadavky.
- **Time to Second Token** je čas medzi prijatím prvej odpovede a prijatím druhej odpovede.
- **Inter Token Latency (ITL)** sa vzťahuje na priemerný časový interval medzi generovaním po sebe nasledujúcich tokenov. Meria ako rýchlo dokáže model vytvoriť každý nasledujúci token po predchádzajúcom.
- **Request Latency**, tiež známe ako End-to-End Latency je čas medzi odoslaním požiadavky a prijatím jeho konečnej odpovede.
- **Output Sequence Length** je celkový počet výstupných tokenov požiadavky.
- **Input Sequence Length** celkový počet vstupných tokenov požiadavky.
- **Output Token Throughput/Throughput** je celkový počet výstupných tokenov z benchmarku vydelený trvaním benchmarku. Priepustnosť meria počet úspešných operácií za jednotku času. Je dôležitým meradlom na určenie toho, ako dobre dokážu spracovať požiadavky používateľov súčasne. V prípade LLM sa priepustnosť meria pomocou tokenov za sekundu.

- **Request Throughput** je počet konečných odpovedí z benchmarku vydelený trvaním benchmarku.

Ďalej sa sústredíme hlavne na Request Latency a Output Token Throughput, kde rozoberieme trade-off medzi latenciou a priepustnosťou pri zväčšovaní záťaže. Sú to kľúčové metriky pri testovaní výkonu, aby sa dokázala určiť hranica pre vhodný use-case nasadenia modelu.

5.4 Výsledky

Spektrum premenných, ako je batch size, veľkosť modelu (počet parametrov), či dĺžka vstupných a výstupných sekvencií, vytvára rozsiahlu kombináciu možných scenárov záťaže, ktoré ovplyvňujú výsledný výkon jazykových modelov. Každá z týchto premenných má odlišný vplyv na metriky ako latencia, priepustnosť alebo spotreba pamäte, a preto je dôležité ich analyzovať v rôznych konfiguráciách. Porovnáme správanie sa jednotlivých modelov pri zmenách týchto parametrov a identifikujeme, v ktorých podmienkach dosahujú najlepší pomer medzi výkonom a efektivitou využitia výpočtových zdrojov. Tento prehľad nám umožní lepšie pochopiť silné a slabé stránky jednotlivých modelov v rôznych aplikačných scenároch.

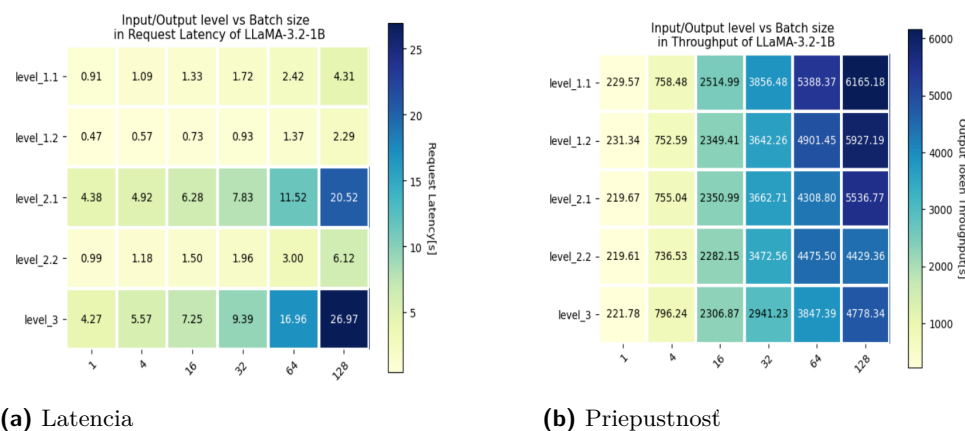
Ako prvé si zobrazíme pamäťovú náročnosť daných modelov pri nasadení s presnosťou FP16 (floating point). Naznačí nám to, koľko pamäte zaberú jednotlivé parametre modelu a napovie, koľko miesta zostane na výpočet predikcií výstupu pri zasielaní požiadaviek. Tabuľka 5.1 ukazuje pamäťovú náročnosť jednotlivých modelov, kde možno vidieť, že modely zaberajú približne dvakrát toľko pamäti, koľko majú parametrov. Tabuľka teda napovedá, že na nasadenie modelu je potreba približne dvojnásobnú pamäť, ako má parametrov.

Model	Memory Usage (GB)
LLaMA-3.2-1B-Instruct	2.31
LLaMA-3.1-8B-Instruct	15
Qwen3-32B	61.03

■ **Tabuľka 5.1** Pamäťová náročnosť jednotlivých modelov pri nasadení

5.4.1 LLaMA-3.2-1B-Instruct

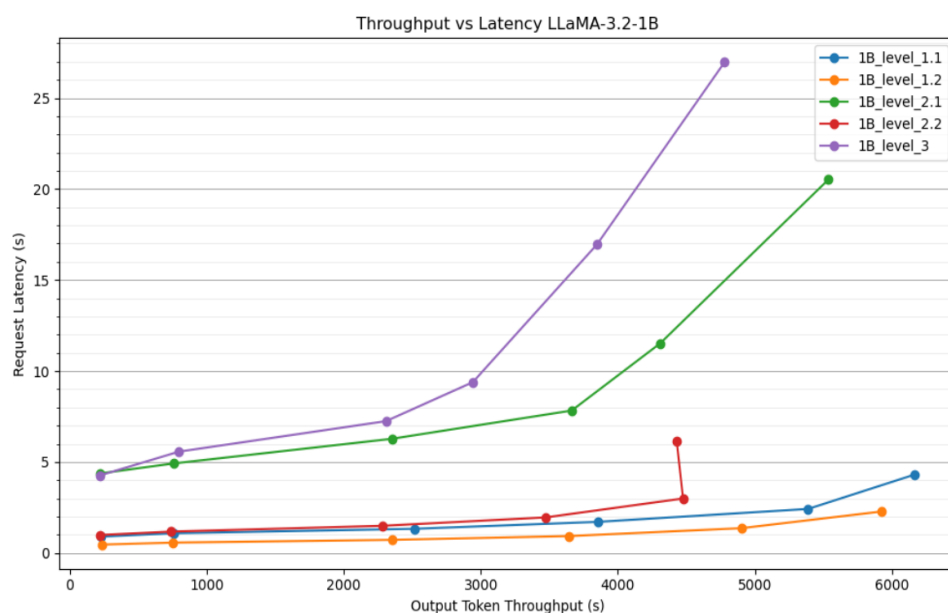
Pri nasadení modelu LLaMA-3.2-1B-Instruct pomocou vLLM a nasledným otestovaním, GenAI-Perf vyprodukoval výsledky, ktoré sme následne upravili a vykreslili grafy, tak aby boli jasné a čitateľné. Na obrázku 5.1a je zrejmé, že latencia požiadaviek stúpa s väčšujúcimi sa dávkami. Ďalej sa dá vypožorovať, že latencia stúpa zásadne pri úrovniach stupu, kde model musí generovať väčší počet tokenov. Napríklad porovnanie úrovni 2.1 (200 vstupných a 1000 výstupných tokenov) oproti 2.2 (1000 vstupných a 200 výstupných tokenov)



Obr. 5.1 Meranie latencie a priepustnosti modelu LLaMA-3.2-1B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.

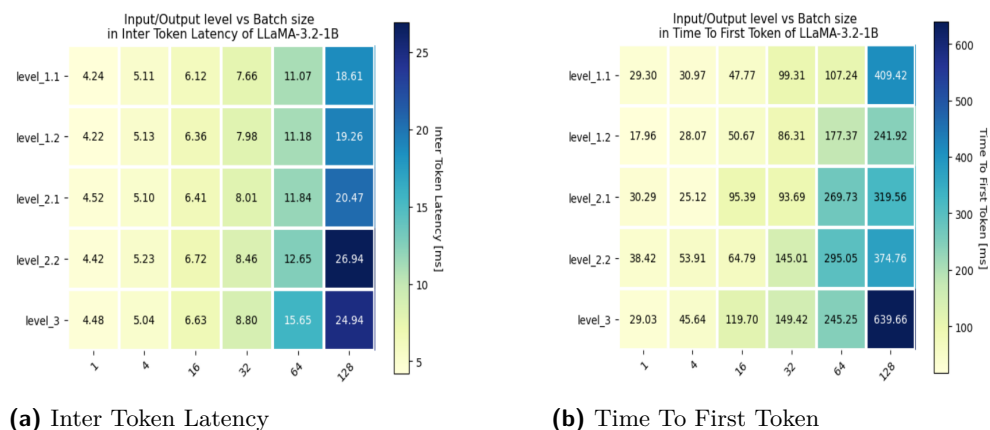
sa výrazne spomaluje vďaka počtu operácií na predikciu jedného tokenu. Keď pridáme porovnanie 2.1 s 1.2 (200 vstupných a 100 výstupných tokenov) nám potvrdzuje, že pri rovnakej dĺžke vstupov zaberá dlhší čas práve predikcia a generovanie nového tokenu oproti spracovaniu vstupu.

Na obrázku 5.1b pozorujeme, že priepustnosť modelu takisto stúpa s rastúcim počtom dávok. Znamená to, že pri zväčšovaní záťaže pridávaním užívateľov model paralelne spracováva požiadavky a generuje odpovede, tým pádom, kým je pamäť dostatočne veľká, dokáže spracovať aj väčšie množstvo požiadaviek a zvýši sa tým aj práve priepustnosť. Zaujímavé je pozorovať úroveň 1.2 a 2.1, kde je rovnaký počet vstupných parametrov, no pri väčších počtoch dávok má vyššiu priepustnosť úroveň 1.2, kde generuje v priemere 100 výstupných tokenov. Dôvodom vyššej priepustnosti pri nižšom počte tokenov generovaní môže byť práve zložitosť kontextu pri generovaní dlhšieho výstupu.



Obr. 5.2 Priepustnosť v porovnaní s latenciou pre LLaMA-3.2-1B

Trade-off medzi latenciou a priepustnosťou je znázornený na 5.2, kde jednotlivé body na krivkách označujú veľkosť dávok. Znázorňuje, že kým priepustnosť stúpa, tak latencia takisto stúpa. Nižšia latencia znamená rýchlejšia odpoveď modelu na požiadavku a preto je potrebné pri nasadzovaní určiť počet možných užívateľov, tak aby latencia aj pri vyššej záťaži ostávala rovnaká. Pri úrovniach 1.1, 1.2 a 2.2 model zafarží zvláda, teda že pri stúpajúcej priepustnosti nedochádza k významnému zhoršeniu latencie. Naopak pri úrovniach 2.1 a 3, kvôli generovaniu 1000 tokenov dochádza k výraznému zhoršeniu oproti predošlým úrovniam, avšak stále rastie vzhľadom na počet dávok sublineárne.



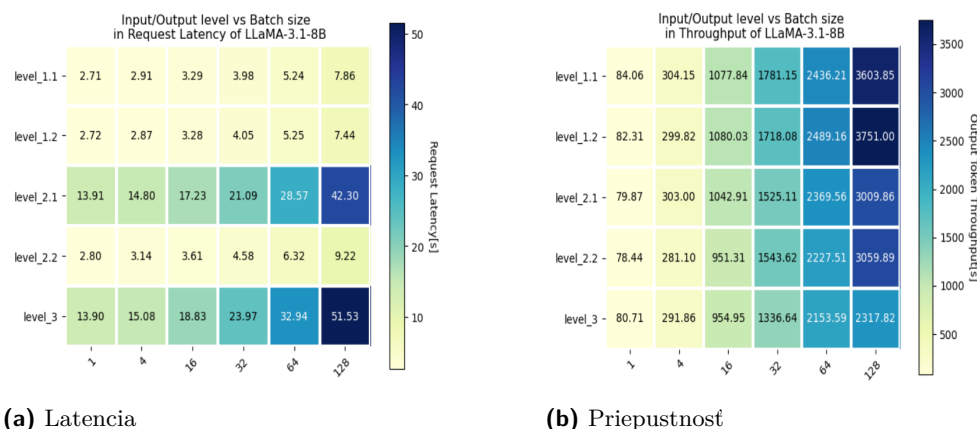
(a) Inter Token Latency

(b) Time To First Token

Obr. 5.3 Meranie ITL a TTFT modelu LLaMA-3.2-1B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.

Na obrázkoch 5.3 sme ako ďalšie metriky vykreslili Inter Token Latency a Time To First Token. Inter Token Latency označuje čas medzi generovaním tokenu a jeho následníka. Time To First Token, ako z názvu vyplýva, je čas od prijatia požiadavky k vygenerovaniu prvého výstupného tokenu. Podotýkame, že obidve metriky sú vykreslené v milisekundách na rozdiel od predošlých grafov, ktoré boli v sekundách. Na grafoch vidíme, že zvyšovaním dávky sa zvyšujú obe testované metriky a tým pádom aj výsledná latencia. Avšak v ITL sa jednotlivé úrovne vstupu a výstupu v konkrétnej dávke nelíšia tak, ako tomu bolo pri latenciách. Otvára sa preto otázka, prečo sú vo výslednej latenciách väčšie rozdiely, keď v ITL sa neobjavujú tak výrazné zmeny medzi úrovňami. Výsledná latencia totiž závisí od počtu generovaných tokenov, zatiaľ čo ITL ostáva podobná. Keď ale model generuje viac tokenov, zarátava viac ITL času a preto aj latencia rastie.

5.4.2 LLaMA-3.1-8B-Instruct

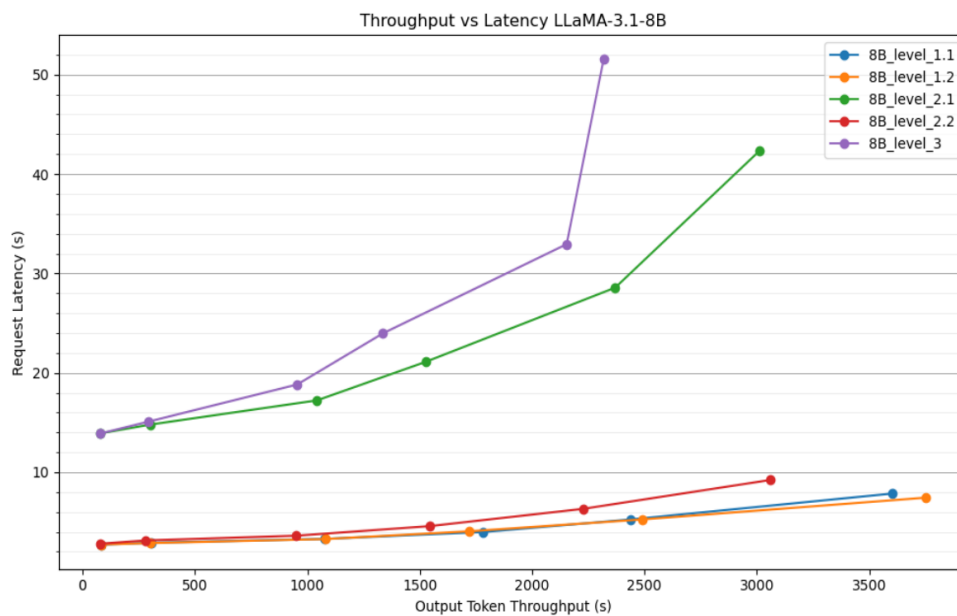


Obr. 5.4 Meranie latencie a priepustnosti modelu LLaMA-3.1-8B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.

Pri testovaní modelu LLaMA-3.1-8B-Instruct pozorujeme podobné rozdiely honôt, ako pri predošlom modele. Teda obrázok 5.4a ukazuje, že latencia požiadaviek stúpa s väčšujúcimi sa dávkami. Podobne, ako pri menšom modele sa dá vypočítavať, že latencia stúpa zásadne pri úrovniach stupu, kde model musí generovať väčší počet tokenov.

Obrázok 5.4b ukazuje, že priepustnosť sa správa obdobne ako v predošlom prípade, teda pri zväčšovaní záťaže pridávaním užívateľov model paralelne spracováva požiadavky a generuje odpovede, tým pádom, kým je pamäť dostatočne veľká, dokáže spracovať aj väčšie množstvo požiadaviek a zvýši sa tým aj práve priepustnosť. Nastáva ale rozdiel v poslednom riadku, kde sa priepustnosť aj s dvojnásobným zväčšením dávky zvýšila minimálne. Je to od-

razom toho, že grafickej karte dochádza pamäť a teda pri obsluhu viacerých užívateľov už nemá protriedky na výpočet a generovanie toľkých požiadaviek paralelne.

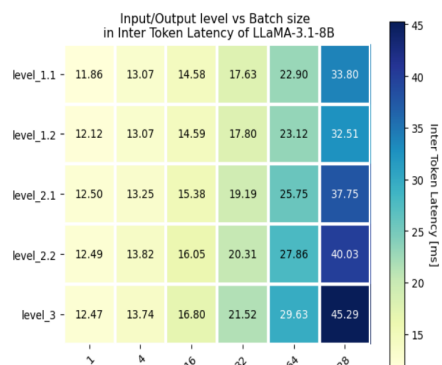


Obr. 5.5 Priepustnosť v porovnaní s latenciou pre LLaMA-3.1-8B

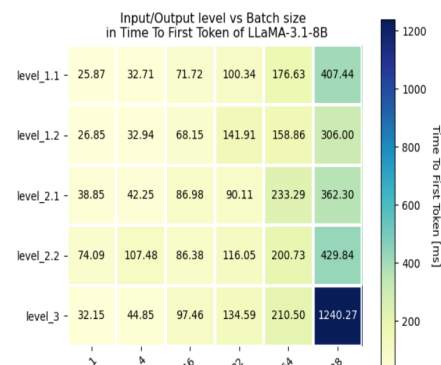
Pri pohľade na obrázok 5.5 sa objavujú už väčšie rozdiely medzi latenciou a priepustnosťou, ako tomu bolo v predšlom prípade. Pri úrovniach 1.1, 1.2 a 2.2 model zataž zvláda, teda že exponenciálnom zvyšovaní dávok latencia stúpa relatívne málo. Zatiaľ čo pri úrovniach 2.1 a 3, kvôli zvýšeniu nárokov na pamäť zväčšením počtu parametrov modelu prichádza k výraznému zhoršovaniu latencie pri zvyšovaní dávok, avšak takisto stále sublineárne.

Metriky ITL a TTFT 5.6 sa správajú podobne, ako tomu bolo pri LLaMA-3.2-1B-Instruct, kde sa hodnoty medzi jednotlivými úrovňami nelíšili tak výrazne, ako pri latencii. Avšak hodnoty obidvoch metrík sú vyššie, ako v predšlom prípade a preto aj latencia bude vyššia. Môže to byť spôsobené tým, že LLaMA-3.1-8B-Instruct má väčšiu pamäťovú náročnosť oproti 1B verzií. Čo sa výrazne zhoršilo je TTFT na úrovni 3 a počtom dávok 128, kde oproti ostatným hodnotám je výrazne odlišná, no v konečnom dôsledku je čas okolo jednej sekundy.

Je dobré si všimnúť, akým spôsobom metriky ovplyvňujú latenciu na základe dĺžky odpovedí. Pozrime sa detailne na úroveň 1.1 a 2.1 s počtami dávok 64 a 128, kde v prvom prípade je 200 výstupných tokenov a v druhom 1000. Ďalej rozdiel v ITL medzi dávkami sa pohybuje v obidvoch prípadoch okolo 11 ms a v TTFT vzrástla hodnota viac na úrovni 1.1, ako 2.1. Keď jednotlivé hodnoty zoberieme do úvahy a pozrieme sa na obrázok 5.5, tak vidíme že v prípade úrovne 2.1 došlo k výraznejšiemu rozdielu medzi počtom dávok 64 a 128, ako



(a) Inter Token Latency

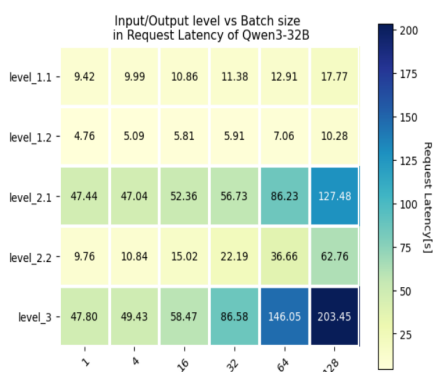


(b) Time To First Token

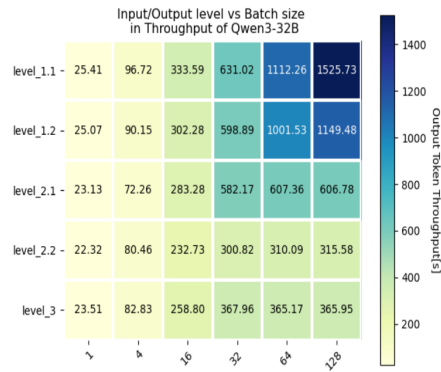
Obr. 5.6 Meranie ITL a TTFT modelu LLaMA-3.1-8B-Instruct na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.

v prípade úrovne 1.1, aj keď rozdiel medzi ITL bol skoro rovnaký a TTFT na strane 1.1 vzrástla výrazne viac, ako pri 2.1. Znamená to, že pri väčších dĺžkach odpovedí nemá TTFT až taký vplyv, ako ITL, ktoré sa opakovane prirátava pri každom výstupnom tokene a práve vďaka dĺžke výstupu vzrástla aj latencia.

5.4.3 Qwen3-32B



(a) Latencia



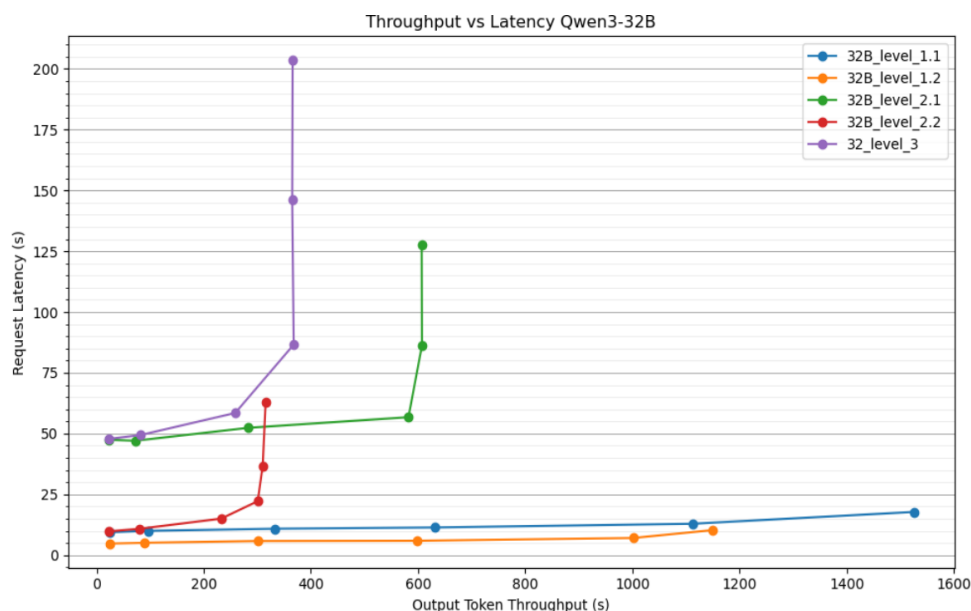
(b) Priepustnosť

Obr. 5.7 Meranie latencie a priepustnosti modelu Qwen3-32B na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.

Qwen3-32B vďaka počtu parametrov konzumuje veľký kus pamäte, ako tabuľka 5.1 znázorňuje. Ostáva tak menší pamäťový priestor na práve výpočet a generovanie. Následkom tejto skutočnosti sa zvýšil čas latencie a zmenšila priepustnosť. Obrázok 5.7a naznačuje podobnú skutočnosť, ako v predošlých prípadoch, no začína sa zväčšovať latencia aj pri úrovni 2.2, kde síce na vstupe

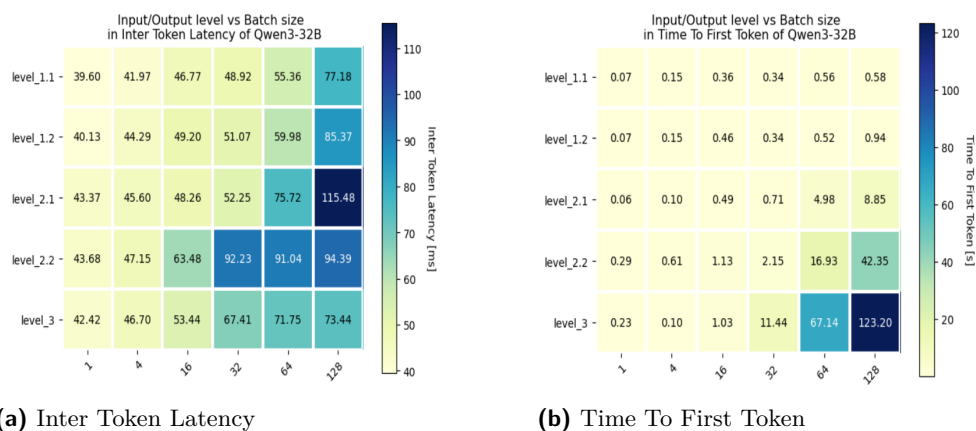
je 1000 tokenov, no generuje 200. Takisto, ako pri menšom modele sa dá vypo-
zorovať, že latencia stúpa zásadne pri úrovniach stupu, kde model musí genero-
vať väčší počet tokenov. Zároveň je viac výrazná skutočnosť, že pri úrovniach
2.1 (200 vstupných a 1000 výstupných tokenov) a 2.2 (1000 vstupných a 200
výstupných tokenov) je úroveň 2.1, už pri jednej dávke násobne pomalších v
latencií jednotlivých požiadaviek. Naznačuje to, že model (aj predošlé) bude
pri úlohách typu „Napíš rozprávku, ktorá obsahuje aspoň 1000 slov.“ výrazne
pomalší, ako pri úlohách sumarizácií textu.

S priepustnosťou sa jav, ktorý sa objavil pri LLaMA-3.1-8B zväčšil a na
obrázku 5.7b vidieť stagnáciu priepustnosti napriek zvyšovaniu dávok. Výraz-
nejšie a zreteľnejšie znázorňuje obrázok 5.8, kde model pri väčších vstupoch
(2.1, 2.2, 3), či výstupoch klesá na rýchlosti a dochádza k stagnácii priepust-
nosti pri zvyšovaní dávok.



■ Obr. 5.8 Priepustnosť v porovnaní s latenciou pre Qwen3-32B

Zmena vo vykreslení grafu 5.9b TTFT je v sekundách. Hodnoty ITL a
TTFT 5.9 vzrástli oproti predošlým prípadom a dochádza aj k väčším rozdie-
lom medzi jednotlivými úrovňami. Zväčšovaním dávok sa zväčšuje výpočtová
náročnosť a dochádza k stagnácii v prípadoch 2.2 a 3. Zaujímavé je pozorovať,
že obidve metriky ovplyvňujú výslednú latenciu. V prvom prípade na úrovni 2.1
pozorujeme výrazné zhoršenie ITL a zároveň TTFT pri väčšom počte dávok
a preto aj dochádza v výraznom zhoršení latencie, zatiaľ čo priepustnosť sa
výrazne nemení. Iný prípad je úroveň 2.2 a 3, kde takisto došlo k nárastu hod-
nôt, ale zároveň k stagnácii na strane ITL a preto k zhoršeniu latencie hlavne
zo strany TTFT, čo pri väčších odpovediach nemalo mať až taký vplyv, ako ITL,
no vďaka stagnácii je hlavný rozdiel vďaka TTFT.



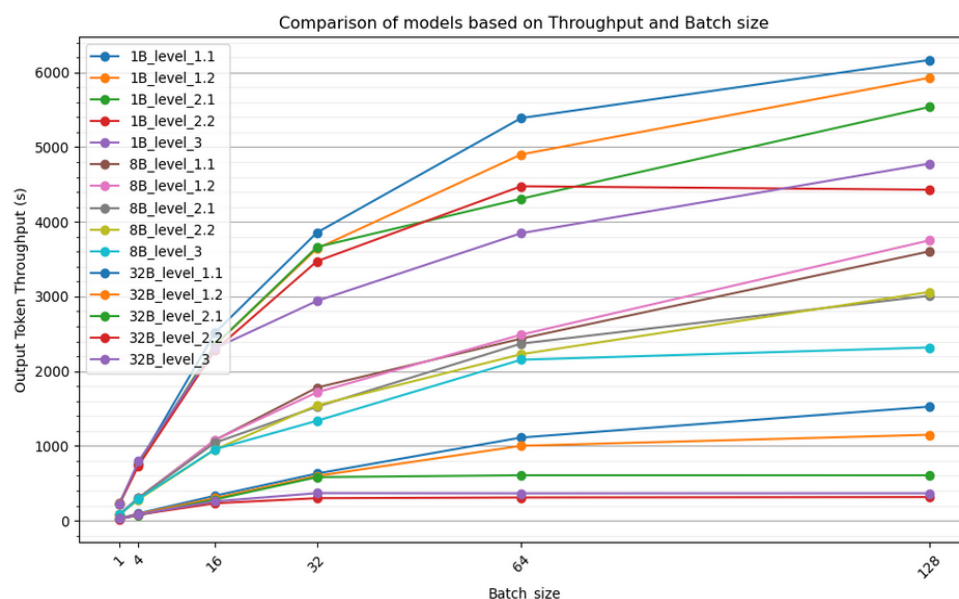
Obr. 5.9 Meranie ITL a TTFT modelu Qwen3-32B na základe dávok a vstupných/výstupných dát. Na ose x je batch size a na ose y úroveň vstupu/výstupu.

5.4.4 Porovnanie

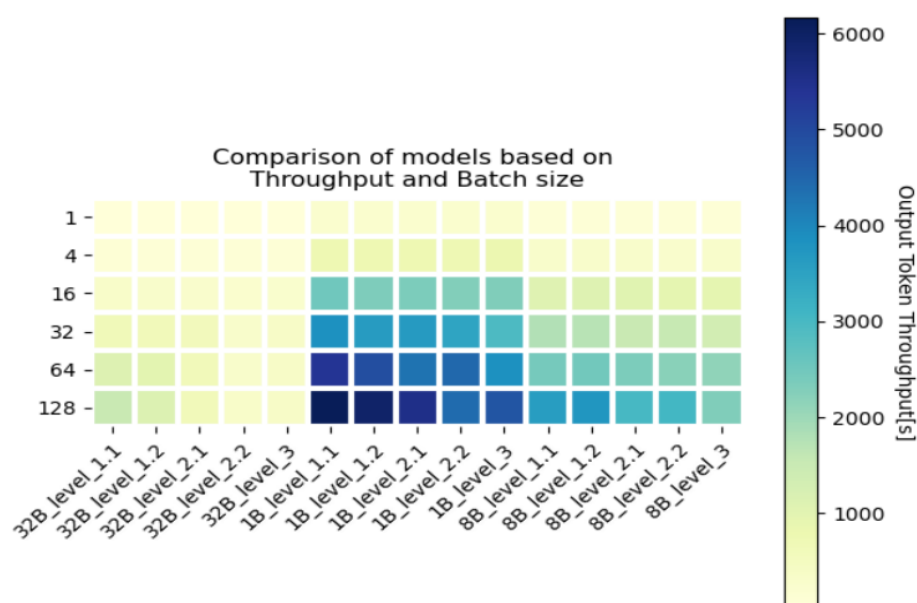
Zámerné sa testovali modely, ktoré boli rôznej veľkosti, teda s rôznym počtom parametrov. To prinieslo určité informácie o tom, ako veľká záťaž môže byť aplikovaná na daný model, ktorý beží na A100 80GB GPU, tak aby aplikácie boli schopné fungovať v reálnom svete. V tejto časti porovnáme a zhrnieme tieto zistenia a aby to bolo prehľadnejšie, tak to urobíme pomocou grafov.

Na obrázkoch 5.10, 5.11, 5.12 a 5.13 sú vyzobrazené všetky úrovne každého modelu. Aby sa predišlo zbytočnej dezinterpretácií, tak ponúkame dva grafy, ktoré zobrazujú jeden a ten istý výsledok. Teda obrázky 5.10 a 5.11 patria k sebe a 5.12 a 5.13 patria k sebe.

Priepustnosť modelu závisí jak na počte vstupných/výstupných tokenov, počte dávok tak, ako je znázornené na 5.10 a 5.11 tak aj robustnosti modelu. Jednotlivé modely sa s istou odchýlkou pohybujú blízko seba v závislosti na počte dávok a vstupných/výstupných tokenov. Prvých 5 kriviek reprezentuje rôzne úrovne modelu LLaMA-3.2-1B, kde ani jeden scénar modelu LLaMA-3.1-8B nedosahuje podobných výsledkov, ako jeho menší náprotivok. Takisto rovnaký rozdiel pozorujeme pri porovnávaní LLaMA-3.1-8B a Qwen3-32B, pri ktorom dochádza k stagnácii a minimálnemu rastu priepustnosti, kvôli nedostatočnej pamäti pri jednotlivých scénaroch.



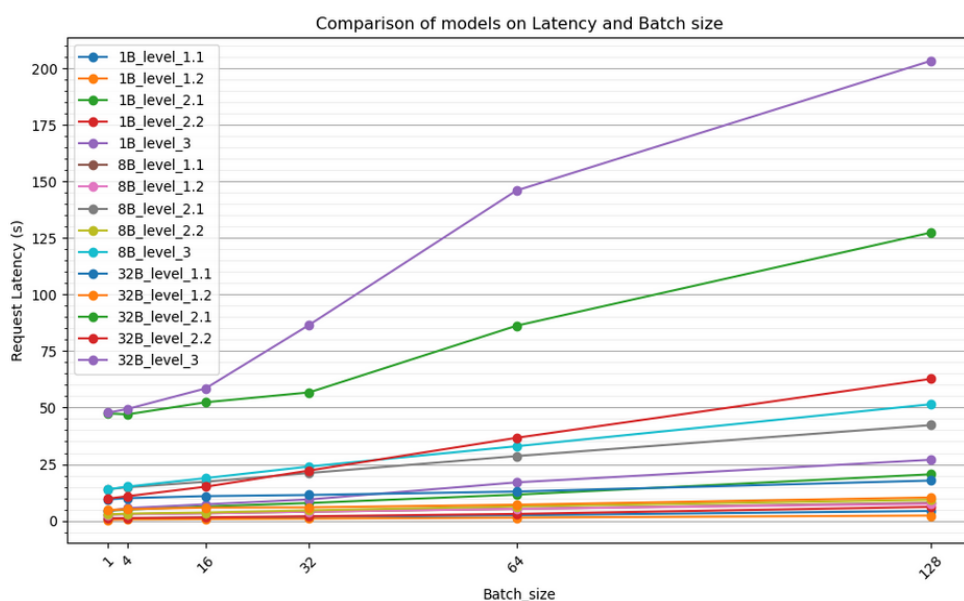
■ Obr. 5.10 Porovnanie modelov a ich úrovni v závislosti s priepustnosťou.



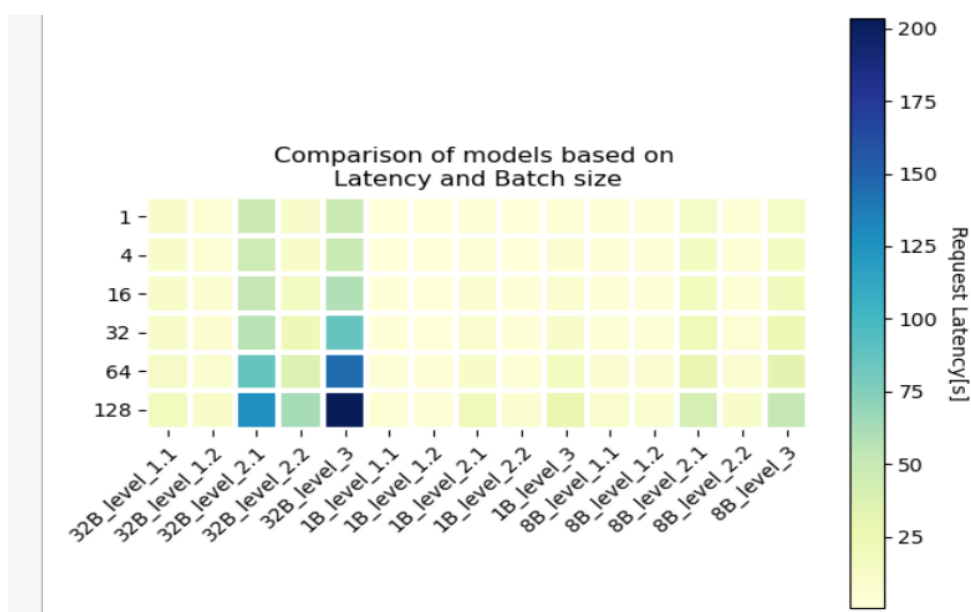
■ Obr. 5.11 Porovnanie modelov a ich úrovni v závislosti s priepustnosťou.

Z pohľadu latencie 5.10 a 5.11 sa modely výrazne líšia hlavne pri väčšom generovaní výstupu a pri vysokej záťaži. Oproti priepustnosti, kde sú väčšie rozdiely, dosahujú modely v latencii menšiu škálu rozdielov a postupne s väčšou záťažou a počtom tokenom reflektujú rozdiely medzi sebou. Modely ako Qwen32 na úrovniach 2.1, 3 (a 2.2 pri vysokej záťaži dávok) nepoužiteľné už od

malej záťaže, pretože na odpoveď sa čaká okolo minúty. Preto na tento model by bolo potrebné silnejšie GPU s väčšou kapacitou pamäte. Ostatné modely na rôznych úrovniach dosahujú s nižších počtách dávok uspokojivé výsledky, no podrobnejšia analýza jednotlivých modelov z predchádzajúceho textu by usprenila jednotlivé možnosti nasadenia.



Obr. 5.12 Porovnanie modelov a ich úrovní v závislosti s latenciou.



Obr. 5.13 Porovnanie modelov a ich úrovní v závislosti s latenciou.

Pre lepšie porovnanie a spriehľadnenie jednotlivých úrovní a výkonu modelov navzájom, vykresľujeme sadu grafov 5.14, kde rozdeľujeme grafy podľa jednotlivých úrovní. Výsledky neprekvapia a sumarizujú predošlé tvrdenie, že menšie modely majú nižšiu latenciu a tým pádom vyššiu priepustnosť vďaka menšej náročnosti váh na pamäť. Čo je zaujímavé pozorovať je, že pri porovnávaní úrovni 2.2 s úrovňou 2.1 modely s vyšším počtom parametrov na úrovni 2.2 majú v niektorých bodoch dávok nižšiu latenciu, ako menšie modely na úrovni 2.1. Konkrétne, keď porovnáваме LLaMA-3.1-8B-Instruct na úrovni 2.2. s LLaMA-3.2-1B-Instruct na úrovni 2.1., tak má nižšiu latenciu cez všetky počty dávok. Takisto, keď porovnáваме LLaMA-3.1-8B-Instruct 2.1. s Qwen3-32B, tak pri menších počtoch dávok dokáže model, ktorý zahŕňa váhami skoro celú pamäť GPU mať nižšiu latenciu, ako model s nižším počtom parametrov. Tento jav ukazuje, že generovanie textu a predikcia nového tokenu je výpočtetne náročnejšia a preto má vyššiu latenciu.

5.4.5 Záver merania

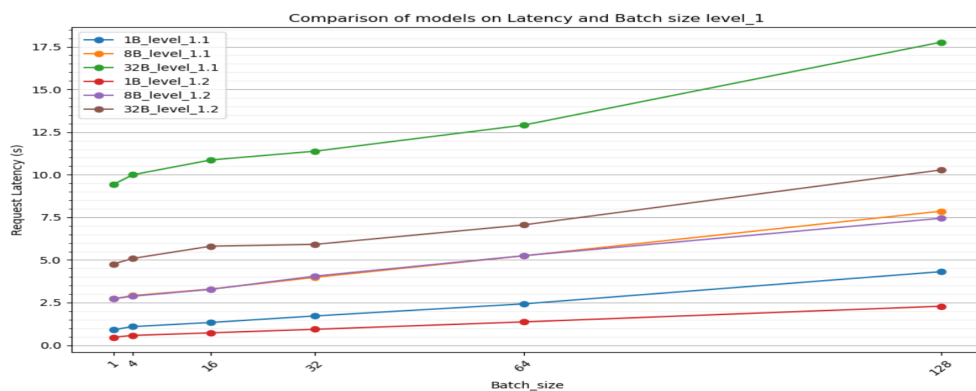
Na základe vykonanej analýzy je zrejmé, že výkonnosť veľkých jazykových modelov pri inferencii je výrazne ovplyvnená kombináciou viacerých faktorov, najmä veľkosťou modelu, počtom vstupných a výstupných tokenov, ako aj veľkosťou dávky požiadaviek. Menšie modely, ako napríklad LLaMA-3.2-1B, dosahujú lepší kompromis medzi latenciou a priepustnosťou, pričom zvládajú vyššie zaťaženie bez zásadného zhoršenia výkonu. Naopak, väčšie modely, ako Qwen3-32B, sú výrazne obmedzené dostupnou pamäťou, čo vedie k zníženej priepustnosti a výraznému nárastu latencie už pri strednej záťaži.

V prípade vplyvu ITL a TTFT na latenciu sme ukázali, že majú vplyv obe metriky. Pri krátkych odpovediach má väčší relatívny vplyv TTFT, keďže táto hodnota sa započítava len raz, zatiaľ čo ITL sa kumuluje až pri generovaní každého ďalšieho tokenu. Naopak, pri dlhších odpovediach začne mať výraznejší vplyv práve ITL, pretože sa opakovane prirátava pri každom ďalšom generovanom tokene.

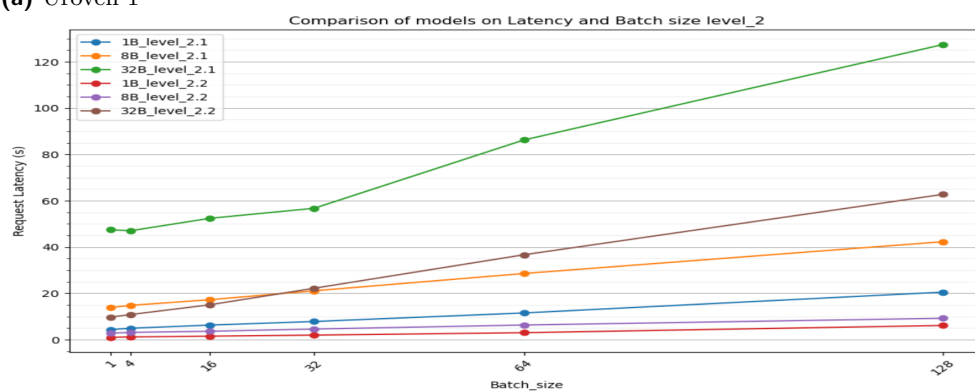
Zároveň sa ukazuje, že generovanie dlhších výstupov má podstatne väčší vplyv na zhoršenie latencie než spracovanie dlhších vstupov. Pre nasadenie modelu v konkrétnom aplikačnom scenári je preto nevyhnutné zvážiť, aký typ požiadaviek bude najčastejšie spracovávaný a aká záťaž sa očakáva. Výsledky tiež potvrdzujú, že správna voľba veľkosti modelu a jeho konfigurácie dokáže výrazne ovplyvniť efektivitu využitia výpočtových zdrojov. Táto analýza tak poskytuje cenný základ pre informované rozhodovanie pri výbere a nasadzovaní jazykových modelov v praxi.

Na základe vykonaných testov inferencie možno rozdeliť požiadavky na GPU pre jazykové modely podľa typu úlohy – interaktívnej a dávkovej – pričom každá z nich kladie odlišné nároky na výpočtový výkon, pamäť a spracovanie požiadaviek.

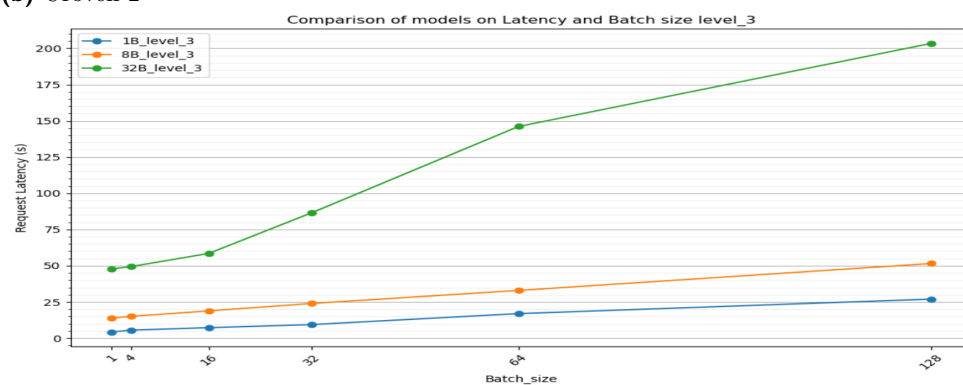
Pri interaktívnych úlohách (napr. chatboty, asistenčné systémy), kde po-



(a) Úroveň 1



(b) Úroveň 2



(c) Úroveň 3

Obr. 5.14 Meranie a porovnanie latencie všetkých modelov na rôznych úrovniach vstupných/výstupných dát. Na ose x je batch size a na ose y latencia.

užívateľ očakáva rýchlu odozvu, je kľúčová nízka latencia, teda hlavne TTFT (time to first token) a ITL (inter-token latency). V tomto prípade sa pracuje s nízkym počtom požiadaviek súčasne, a preto je dôležité, aby GPU poskytovalo vysoký výkon aj pri nízkom zaťažení. Postačuje pritom jedno GPU s dostatočne veľkou VRAM (napr. A100 80GB), ktoré zvládne bežať stredne veľké modely. Zároveň ale chceme využiť dostupné prostriedky naplno a určiť tak optimálny počet užívateľov tak, aby inferencia bola plynulá a užívatelia boli s dĺžkou inferencie spokojní. V súčasnej dobe sa ITL a TTFT modelov pohybuje v desatinách až jednotkách sekúnd, keď následne spojíme tieto informácie s celkovou latenciou, všimneme si, že pri testovaných modeloch s hodnotou batch size okolo 32 je latencia stále nižšia, ako dvojnásobok hodnoty batch size 1. Tým pádom usudzujeme, že optimálny počet užívateľov serveru s jedným GPU A100 je 32 a určuje kompromis medzi veľkosťou latencie a počtom užívateľov.

Naopak pri dávkovom spracovaní (napr. sumarizácia dokumentov), kde nie je podstatná odozva na jednotlivý vstup, ale celková priepustnosť systému, sa pracuje s veľkými dávkami požiadaviek (vysoký batch size). Práve kvôli tomu je potrebné, aby GPU disponovalo veľkým množstvom pamäte a výpočtových jednotiek, ktoré zvládnu naraz obslúžiť viacero vstupov. Preto dávkové spracovanie často vyžaduje väčšie GPU alebo viaceré GPU v paralelnej konfigurácii a práve tu sa ukazuje výhoda H100 80GB alebo multi-GPU systémov.



Kapitola 6

Záver

Výsledkom tejto práce je komplexný pohľad na problematiku veľkých jazykových modelov (LLM), vrátane ich architektúry, procesu tréovania, optimalizačných techník a kľúčových mechanizmov, ktoré ovplyvňujú ich presnosť a výkonnosť. Práca zároveň prináša porovnanie aktuálnych open-source modelov na úrovni state-of-the-art, pričom poukazuje na rozdiely v architektúre, použitých tréovacích dátach a tréovacích stratégiách. Vďaka tomu poskytuje jasný prehľad o ich rozdieloch a špecifikách pri implementácii.

Hlavným cieľom práce bolo otestovať výkonnosť rôznych otvorených LLM pomocou grafických kariet A100 80GB a získať tak predstavu o ich výpočtovej náročnosti a časovej efektivite pri rôznych typoch záťaže. Zamerali sme sa najmä na faktory ako dĺžka vstupných a výstupných sekvencií a počet súbežných požiadaviek, ktoré majú zásadný vplyv na výkon modelu.

V rámci merania sme sa sústredili na kľúčové metriky ako latencia, priepustnosť, pamäťová náročnosť a závislosti ITL (inter token latency) a TTFT (time to first token) na hodnotu výslednej latencie. Na základe týchto údajov sme dokázali identifikovať, pre ktoré typy aplikácií sú jednotlivé modely najvhodnejšie, aké majú limity a aké výhody či nevýhody so sebou prinášajú.

Táto práca tak poskytuje praktický prehľad o možnostiach nasadenia veľkých jazykových modelov v závislosti od dostupného hardvéru a očakávaného typu úloh, čím pomáha pri výbere vhodného modelu pre konkrétne využitie.

Bibliografia

1. ZHAO, Wayne Xin; ZHOU, Kun; LI, Junyi; TANG, Tianyi; WANG, Xiaolei; HOU, Yupeng; MIN, Yingqian; ZHANG, Beichen; ZHANG, Junjie; DONG, Zican et al. A survey of large language models. *arXiv preprint arXiv:2303.18223* [online]. 2023, roč. 1, č. 2 [cit. 2025-03-14]. Dostupné z : https://paper-notes.zhjwpku.com/assets/pdfs/llm_survey_2303.18223.pdf.
2. BRIGANTI, Giovanni. ChatGHow PT works: a mini review. *European Archives of Oto-Rhino-Laryngology* [online]. 2024, roč. 281, č. 3, s. 1565–1569 [cit. 2025-03-14]. Dostupné z : <https://link.springer.com/article/10.1007/s00405-023-08337-7>.
3. TUGGENER, Lukas; SAGER, Pascal; TAOUDI-BENCHEKROUN, Yasmine; GREWE, Benjamin F; STADELMANN, Thilo. So you want your private LLM at home?: a survey and benchmark of methods for efficient GPTs. In: *11th IEEE Swiss Conference on Data Science (SDS), Zurich, Switzerland, 30-31 May 2024* [online]. ZHAW Zürcher Hochschule für Angewandte Wissenschaften, 2024 [cit. 2025-03-14]. Dostupné z : <https://ieeexplore.ieee.org/abstract/document/10675845/>.
4. *Llama* [online]. Meta, 2025. [cit. 2025-03-20]. Dostupné z : <https://www.llama.com/>.
5. *Qwen LLMs* [online]. Alibaba Cloud, 2025. [cit. 2025-03-20]. Dostupné z : <https://www.alibabacloud.com/help/en/model-studio/developer-reference/what-is-qwen-llm>.
6. CHITTY-VENKATA, Krishna Teja; RASKAR, Siddhisanket; KALE, Bharat; FERDAUS, Farah; TANIKANTI, Aditya; RAFFENETTI, Ken; TAYLOR, Valerie; EMANI, Murali; VISHWANATH, Venkatram. LLM-Inference-Bench: Inference Benchmarking of Large Language Models on AI Accelerators. In: *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, s. 1362–1379.

7. GOYAL, Sagar; RASTOGI, Eti; RAJAGOPAL, Sree Prasanna; YUAN, Dong; ZHAO, Fen; CHINTAGUNTA, Jai; NAIK, Gautam; WARD, Jeff. Healai: A healthcare llm for effective medical documentation. In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 2024, s. 1167–1168.
8. LI, Yinheng; WANG, Shaofei; DING, Han; CHEN, Hang. Large language models in finance: A survey. In: *Proceedings of the fourth ACM international conference on AI in finance*. 2023, s. 374–382.
9. *Co je to Crowdsourcing?* [Online]. IT-Slovník.cz team. [cit. 2025-03-21]. Dostupné z : <https://it-slovník.cz/pojem/crowdsourcing>.
10. *Comparison of Models: Intelligence, Performance and Price Analysis* [online]. Artificial Analysis, 2024. [cit. 2025-03-21]. Dostupné z : <https://artificialanalysis.ai/models>.
11. MALEC, Melissa. *Open-Source LLMs vs Closed: Unbiased Guide for Innovative Companies* [online]. 2025. [cit. 2025-03-21]. Dostupné z : <https://hatchworks.com/blog/gen-ai/open-source-vs-closed-llms-guide/#open-source-vs-closed>.
12. *Pozor na „vendor lock-in“ aneb řídte bezpečně své IT projekty* [online]. Total Service, 2023. [cit. 2025-03-21]. Dostupné z : <https://www.total-service.cz/novinky/pozor-na-vendor-lock-in-aneb-ridte-bezpecne-sve-it-projekty-2023-03-10/>.
13. YANG, Jingfeng; JIN, Hongye; TANG, Ruixiang; HAN, Xiaotian; FENG, Qizhang; JIANG, Haoming; ZHONG, Shaochen; YIN, Bing; HU, Xia. Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data* [online]. 2024, roč. 18, č. 6, s. 1–32 [cit. 2025-03-14]. Dostupné z : <https://dl.acm.org/doi/full/10.1145/3649506>.
14. VASWANI, Ashish; SHAZEER, Noam; PARMAR, Niki; USZKOREIT, Jakob; JONES, Llion; GOMEZ, Aidan N; KAISER, Łukasz; POLOSUKHIN, Illia. Attention is all you need. *Advances in neural information processing systems*. 2017, roč. 30. Dostupné tiež z: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
15. *LLM Architecture: Exploring the Technical Architecture Behind Large Language Models* [online]. GreeksForGreeks, 2024. [cit. 2025-03-21]. Dostupné z : <https://www.geeksforgeeks.org/exploring-the-technical-architecture-behind-large-language-models/>.
16. *What is a Large Language Model (LLM)* [online]. GreeksForGreeks, 2025. [cit. 2025-03-21]. Dostupné z : <https://www.geeksforgeeks.org/large-language-model-llm/>.

17. JACOB MUREL PH.D., Joshua Noble. *What is an encoder-decoder model?* [Online]. IBM, 2024 [cit. 2025-03-23]. Dostupné z : <https://www.ibm.com/think/topics/encoder-decoder-model>.
18. SANSEVIERO, Omar; TUNSTALL, Lewis; SCHMID, Philipp; MANGRULKAR, Sourab; BELKADA, Younes; CUENCA, Pedro. *Mixture of Experts Explained* [online]. Hugging Face, 2023 [cit. 2025-04-04]. Dostupné z : <https://huggingface.co/blog/moe>.
19. PENEDO, Guilherme; MALARTIC, Quentin; HESSLOW, Daniel; COJOCARU, Ruxandra; CAPPELLI, Alessandro; ALOBEIDLI, Hamza; PANNIER, Baptiste; ALMAZROUEI, Ebtesam; LAUNAY, Julien. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.
20. NAM, Daye; MACVEAN, Andrew; HELLENDORRN, Vincent; VASILESCU, Bogdan; MYERS, Brad. Using an llm to help with code understanding. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* [online]. 2024, s. 1–13 [cit. 2025-03-14]. Dostupné z : <https://dl.acm.org/doi/abs/10.1145/3597503.3639187>.
21. KANDPAL, Nikhil; WALLACE, Eric; RAFFEL, Colin. Deduplicating training data mitigates privacy risks in language models. In: *International Conference on Machine Learning*. PMLR, 2022, s. 10697–10707.
22. *The Hugging Face Course, 2022* [<https://huggingface.co/course>]. Hugging Face, 2022. [cit. 2025-04-05]. [Online].
23. DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton; TOUTANOVA, Kristina. Bert: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, s. 4171–4186.
24. BA, Jimmy Lei; KIROU, Jamie Ryan; HINTON, Geoffrey E. Layer normalization. *arXiv preprint arXiv:1607.06450*. 2016.
25. ZHANG, Biao; SENNRICH, Rico. Root mean square layer normalization. *Advances in Neural Information Processing Systems*. 2019, roč. 32.
26. WANG, Hongyu; MA, Shuming; DONG, Li; HUANG, Shaohan; ZHANG, Dongdong; WEI, Furu. Deepnet: Scaling transformers to 1,000 layers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2024.
27. HENDRYCKS, Dan; GIMPEL, Kevin. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*. 2016.
28. SHAZEER, Noam. Glue variants improve transformer. *arXiv preprint arXiv:2002.05202*. 2020.

29. SHAW; PETER; USZKOREIT; JAKOB; VASWANI; ASHISH. Self-Attention with Relative Position Representations. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. New Orleans, Louisiana: Association for Computational Linguistics, 2018, s. 464–468. Dostupné z DOI: 10.18653/v1/N18-2074.
30. PRESS, Ofir; SMITH, Noah; LEWIS, Mike. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In: *International Conference on Learning Representations*. 2022. Dostupné tiež z: <https://openreview.net/forum?id=R8sQPpGCv0>.
31. CHILD, Rewon; GRAY, Scott; RADFORD, Alec; SUTSKEVER, Ilya. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*. 2019.
32. TOUVRON, Hugo; MARTIN, Louis; STONE, Kevin; ALBERT, Peter; ALMAHAIRI, Amjad; BABAEI, Yasmine; BASHLYKOV, Nikolay; BATRA, Soumya; BHARGAVA, Prajwal; BHOSALE, Shruti et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*. 2023.
33. BERGMANN, Dave. *What is fine-tuning?* [Online]. IBM, 2024 [cit. 2025-04-05]. Dostupné z : <https://www.ibm.com/think/topics/fine-tuning>.
34. GUU, Kelvin; LEE, Kenton; TUNG, Zora; PASUPAT, Panupong; CHANG, Mingwei. Retrieval augmented language model pre-training. In: *International conference on machine learning*. PMLR, 2020, s. 3929–3938.
35. LOSHCHILOV, Ilya; HUTTER, Frank. Fixing Weight Decay Regularization in Adam. 2018. Dostupné tiež z: <https://openreview.net/forum?id=rk6qdGgCZ>.
36. KINGMA, Diederik P; BA, Jimmy. Adam: A method for stochastic optimization. *ICLR 2015, International Conference on Learning Representations (ICLR)*. 2014.
37. WILT, Christopher; THAYER, Jordan; RUMMLER, Wheeler. A comparison of greedy search algorithms. In: *proceedings of the international symposium on combinatorial search*. 2010, zv. 1, s. 129–136. Č. 1.
38. WU, Yonghui; SCHUSTER, Mike; CHEN, Zhifeng; LE, Quoc V; NOUROZI, Mohammad; MACHEREY, Wolfgang; KRIKUN, Maxim; CAO, Yuan; GAO, Qin; MACHEREY, Klaus et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*. 2016.
39. *Pre-training in LLM Development* [online]. Toloka, Toloka Team, 2024 [cit. 2025-04-05]. Dostupné z : <https://toloka.ai/blog/pre-training-in-llm-development/>.

40. SHENG, Ying; ZHENG, Lianmin; YUAN, Binhang; LI, Zhuohan; RY-ABININ, Max; CHEN, Beidi; LIANG, Percy; RÉ, Christopher; STOICA, Ion; ZHANG, Ce. Flexgen: High-throughput generative inference of large language models with a single gpu. In: *International Conference on Machine Learning*. PMLR, 2023, s. 31094–31116.
41. GHOLAMI, Amir; KIM, Sehoon; DONG, Zhen; YAO, Zhewei; MAHONEY, Michael W; KEUTZER, Kurt. A survey of quantization methods for efficient neural network inference. In: *Low-power computer vision*. Chapman a Hall/CRC, 2022, s. 291–326.
42. OWENS, John D; HOUSTON, Mike; LUEBKE, David; GREEN, Simon; STONE, John E; PHILLIPS, James C. GPU computing. *Proceedings of the IEEE*. 2008, roč. 96, č. 5, s. 879–899.
43. CHOQUETTE, Jack; GANDHI, Wishwesh; GIROUX, Olivier; STAM, Nick; KRASHINSKY, Ronny. Nvidia a100 tensor core gpu: Performance and innovation. *IEEE Micro*. 2021, roč. 41, č. 2.
44. MITTAL, Sparsh; VAISHAY, Shraiys. A survey of techniques for optimizing deep learning on GPUs. *Journal of Systems Architecture*. 2019, roč. 99, s. 101635.
45. STEINKRAUS, Dave; BUCK, Ian; SIMARD, Patrice Y. Using GPUs for machine learning algorithms. In: *Eighth International Conference on Document Analysis and Recognition (ICDAR'05)*. IEEE, 2005, s. 1115–1120.
46. RECASENS, Pol G; AGULLO, Ferran; ZHU, Yue; WANG, Chen; LEE, Eun Kyung; TARDIEU, Olivier; TORRES, Jordi; BERRAL, Josep Ll. Mind the Memory Gap: Unveiling GPU Bottlenecks in Large-Batch LLM Inference. *arXiv preprint arXiv:2503.08311*. 2025.
47. BAI, Jinze; BAI, Shuai; CHU, Yunfei; CUI, Zeyu; DANG, Kai; DENG, Xiaodong; FAN, Yang; GE, Wenbin; HAN, Yu; HUANG, Fei et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*. 2023.
48. JIANG, Albert Q.; SABLAYROLLES, Alexandre; MENSCH, Arthur; BAMFORD, Chris; CHAPLOT, Devendra Singh; CASAS, Diego de las; BRESSAND, Florian; LENGUEL, Gianna; LAMPLE, Guillaume; SAULNIER, Lucile; LAVAUD, Léo Renard; LACHAUX, Marie-Anne; STOCK, Pierre; SCAO, Teven Le; LAVRIL, Thibaut; WANG, Thomas; LACROIX, Timothée; SAYED, William El. *Mistral 7B*. 2023. Dostupné z arXiv: 2310.06825 [cs.CL].
49. LIU, Aixin; FENG, Bei; WANG, Bin; WANG, Bingxuan; LIU, Bo; ZHAO, Chenggang; DENG, Chengqi; RUAN, Chong; DAI, Damai; GUO, Daya et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*. 2024.

50. YANG, An; YANG, Baosong; HUI, Binyuan; ZHENG, Bo; YU, Bowen; ZHOU, Chang; LI, Chengpeng; LI, Chengyuan; LIU, Dayiheng; HUANG, Fei; DONG, Guanting; WEI, Haoran; LIN, Huan; TANG, Jialong; WANG, Jialin; YANG, Jian; TU, Jianhong; ZHANG, Jianwei; MA, Jianxin; YANG, Jianxin; XU, Jin; ZHOU, Jingren; BAI, Jinze; HE, Jinzheng; LIN, Junyang; DANG, Kai; LU, Keming; CHEN, Keqin; YANG, Kexin; LI, Mei; XUE, Mingfeng; NI, Na; ZHANG, Pei; WANG, Peng; PENG, Ru; MEN, Rui; GAO, Ruize; LIN, Runji; WANG, Shijie; BAI, Shuai; TAN, Sinan; ZHU, Tianhang; LI, Tianhao; LIU, Tianyu; GE, Wenbin; DENG, Xiaodong; ZHOU, Xiaohuan; REN, Xingzhang; ZHANG, Xinyu; WEI, Xipin; REN, Xuancheng; LIU, Xuejing; FAN, Yang; YAO, Yang; ZHANG, Yichang; WAN, Yu; CHU, Yunfei; LIU, Yuqiong; CUI, Zeyu; ZHANG, Zhenru; GUO, Zhifang; FAN, Zhihao. *Qwen2 Technical Report*. 2024. Dostupné z arXiv: 2407.10671 [cs.CL].
51. QWEN; : YANG, An; YANG, Baosong; ZHANG, Beichen; HUI, Binyuan; ZHENG, Bo; YU, Bowen; LI, Chengyuan; LIU, Dayiheng; HUANG, Fei; WEI, Haoran; LIN, Huan; YANG, Jian; TU, Jianhong; ZHANG, Jianwei; YANG, Jianxin; YANG, Jiaxi; ZHOU, Jingren; LIN, Junyang; DANG, Kai; LU, Keming; BAO, Keqin; YANG, Kexin; YU, Le; LI, Mei; XUE, Mingfeng; ZHANG, Pei; ZHU, Qin; MEN, Rui; LIN, Runji; LI, Tianhao; TANG, Tianyi; XIA, Tingyu; REN, Xingzhang; REN, Xuancheng; FAN, Yang; SU, Yang; ZHANG, Yichang; WAN, Yu; LIU, Yuqiong; CUI, Zeyu; ZHANG, Zhenru; QIU, Zihan. *Qwen2.5 Technical Report*. 2025. Dostupné z arXiv: 2412.15115 [cs.CL].
52. *Qwen3: Think Deeper, Act Faster* [online]. Qwen, Qwen Team, 2025 [cit. 2025-05-07]. Dostupné z : <https://qwenlm.github.io/blog/qwen3/>.
53. JIANG, Albert Q.; SABLAYROLLES, Alexandre; ROUX, Antoine; MENSCH, Arthur; SAVARY, Blanche; BAMFORD, Chris; CHAPLOT, Devendra Singh; CASAS, Diego de las; HANNA, Emma Bou; BRESSAND, Florian; LENGUEL, Gianna; BOUR, Guillaume; LAMPLE, Guillaume; LAVAUD, L  lio Renard; SAULNIER, Lucile; LACHAUX, Marie-Anne; STOCK, Pierre; SUBRAMANIAN, Sandeep; YANG, Sophia; ANTONIAK, Szymon; SCAO, Teven Le; GERVET, Th  ophile; LAVRIL, Thibaut; WANG, Thomas; LACROIX, Timoth  e; SAYED, William El. *Mixtral of Experts*. 2024. Dostupné z arXiv: 2401.04088 [cs.LG].
54. GRATTAFIORI, Aaron et al. *The Llama 3 Herd of Models*. 2024. Dostupné z arXiv: 2407.21783 [cs.AI].
55. TOUVRON, Hugo; MARTIN, Louis; STONE, Kevin; ALBERT, Peter; ALMAHAIRI, Amjad; BABAEI, Yasmine; BASHLYKOV, Nikolay; BATRA, Soumya; BHARGAVA, Prajjwal; BHOSALE, Shruti; BIKEL, Dan; BLECHER, Lukas; FERRER, Cristian Canton; CHEN, Moya; CUCURULL, Guillem; ESIOBU, David; FERNANDES, Jude; FU, Jeremy; FU, Wenyin; FULLER, Brian; GAO, Cynthia; GOSWAMI, Vedanuj; GOYAL,

- Naman; HARTSHORN, Anthony; HOSSEINI, Saghar; HOU, Rui; INAN, Hakan; KARDAS, Marcin; KERKEZ, Viktor; KHABSA, Madian; KLOUMANN, Isabel; KORENEV, Artem; KOURA, Punit Singh; LACHAUX, Marie-Anne; LAVRIL, Thibaut; LEE, Jenya; LISKOVICH, Diana; LU, Yinghai; MAO, Yuning; MARTINET, Xavier; MIHAYLOV, Todor; MISHRA, Pushkar; MOLYBOG, Igor; NIE, Yixin; POULTON, Andrew; REIZENSTEIN, Jeremy; RUNGTA, Rashi; SALADI, Kalyan; SCHELTEN, Alan; SILVA, Ruan; SMITH, Eric Michael; SUBRAMANIAN, Ranjan; TAN, Xiaoqing Ellen; TANG, Binh; TAYLOR, Ross; WILLIAMS, Adina; KUANG, Jian Xiang; XU, Puxin; YAN, Zheng; ZAROV, Iliyan; ZHANG, Yuchen; FAN, Angela; KAMBADUR, Melanie; NARANG, Sharan; RODRIGUEZ, Aurelien; STOJNIC, Robert; EDUNOV, Sergey; SCIALOM, Thomas. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023. Dostupné z arXiv: 2307.09288 [cs.CL].
56. KUDO, Taku; RICHARDSON, John. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 2018. Dostupné z DOI: 10.18653/v1/D18-2012.
 57. BI, Xiao; CHEN, Deli; CHEN, Guanting; CHEN, Shanhuang; DAI, Damai; DENG, Chengqi; DING, Honghui; DONG, Kai; DU, Qiushi; FU, Zhe et al. Deepseek llm: Scaling open-source language models with long-termism. *arXiv preprint arXiv:2401.02954*. 2024.
 58. DAI, Damai; DENG, Chengqi; ZHAO, Chenggang; XU, RX; GAO, Huazuo; CHEN, Deli; LI, Jiashi; ZENG, Wangding; YU, Xingkai; WU, Yu et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. 2024. Dostupné z DOI: 10.18653/v1/2024.acl-long.70.
 59. WANG, Lean; GAO, Huazuo; ZHAO, Chenggang; SUN, Xu; DAI, Damai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*. 2024.
 60. *GenAI-Perf* [online]. NVIDIA Corporation. [cit. 2025-03-14]. Dostupné z : https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/genai-perf/README.html#command-line-options.
 61. KWON, Woosuk; LI, Zhuohan; ZHUANG, Siyuan; SHENG, Ying; ZHENG, Lianmin; YU, Cody Hao; GONZALEZ, Joseph E.; ZHANG, Hao; STOLICA, Ion. *Proceedings of the 29th Symposium on Operating Systems Principles*. Efficient Memory Management for Large Language Model Serving with PagedAttention. Koblenz, Germany: Association for Computing Machinery, 2023. SOSP '23. ISBN 9798400702297. Dostupné z DOI: 10.1145/3600006.3613165.

Obsah prílohy

impl.....	zdrojové kódy implementace
└─ graphs.ipynb.....	python notebook na vykreslenie grafov
└─ testerlevel11.sh.....	testovací script na úrovni 1.1
└─ testerlevel12.sh.....	testovací script na úrovni 1.2
└─ testerlevel21.sh.....	testovací script na úrovni 2.1
└─ testerlevel22.sh.....	testovací script na úrovni 2.2
└─ testerlevel3.sh.....	testovací script na úrovni 3
└─ slurm.sh.....	slurm script na nasadenie jobu s clusteri
└─ thesis.....	zdrojová forma práce ve formátu L ^A T _E X